

Het ontwikkelen van een geïntegreerd framework dat de veiligheid garandeert en volledige traceerbaarheid biedt voor alle interacties tussen gebruikers en een digitale assistent in klantondersteuningsprocessen.

Een Defense-in-Depth framework voor agentic AI en applicatie-integraties.

Bricke Volckeryck.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. P. Van Der Helst

Co-promotor: Dhr. K. Haeck

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Deze bachelorproef markeert het eindpunt van mijn opleiding aan HOGENT. De samenwerking met Evolane gaf me de kans om echt in de materie te duiken: van NLU-chatbotlogica en de Akamai Firewall for AI, tot diepgaande observability met Dynatrace. Ik kijk terug op een bijzonder leerrijke periode waar ik veel uit heb meegenomen.

Een oprecht woord van dank gaat uit naar mijn promotor, **Pieter Van Der Helst**, voor zijn heldere sturing en betrokken begeleiding doorheen het volledige traject. Ook mijn copromotor, **Kristof Haeck**, verdient een welgemeende dankjewel voor het vertrouwen dat hij in mij had en de kans om dit project bij Evolane uit te voeren. Zijn kritische blik heeft dit werk beter gemaakt.

Bijzondere dank gaat ook uit naar **Dylan Taelemans**, die mij op weg heeft gezet in de opzet van de Rasa-chatbot en de Dynatrace-integratie, en naar **Koen Derkinderen** voor zijn gerichte expertise bij de configuratie van de Dynatrace-omgeving. Hun praktische kennis en ervaring was onmisbaar.

Bricke Volckeryck
Sint-Niklaas, mei 2026

Samenvatting

In moderne digitale klantomgevingen worden AI-chatbots en digitale assistenten steeds vaker ingezet om ondersteuning te bieden, informatie te verstrekken en bedrijfskritische transacties te begeleiden. Deze groeiende afhankelijkheid van geautomatiseerde interacties brengt echter aanzienlijke veiligheidsrisico's met zich mee. Chatbots verwerken ongestructureerde natuurlijke taal, waardoor traditionele beveiligingsmechanismen vaak tekortschieten. Dit creëert kwetsbaarheden zoals applicatielaag-injecties (zoals XSS en SQLi), data-exfiltratie, en manipulatie via prompt-injecties of toxische invoer. Bovendien werken deze AI-systemen vaak als een 'black-box', wat leidt tot een gebrek aan inzicht in de gesprekscontext en potentiële schendingen van de Algemene Verordening Gegevensbescherming (GDPR). Daardoor ontstaat een duidelijke nood aan een architectuur die zowel traceerbaarheid als diepgaande beveiliging garandeert.

Dit onderzoek heeft als doel een geïntegreerd framework te ontwikkelen dat de volledige communicatiestroom tussen eindgebruikers en een AI-chatbot inzichtelijk en beveiligd maakt. Hiervoor wordt onderzocht hoe Dynatrace kan worden ingezet om gesprekslogs, acties en systeeminteracties end-to-end te monitoren, en hoe Akamai's Firewall for AI verdachte input, aanvallen of ongewenste datastromen op de edge kan detecteren en blokkeren. De centrale onderzoeksvraag luidt op welke manier deze twee technologieën gecombineerd kunnen worden om een traceerbare én veilige communicatiestroom te realiseren.

Om deze vraag te beantwoorden is een Proof of Concept (PoC) ontwikkeld met een hybride infrastructuur. Deze testomgeving bestaat uit een intent-gebaseerde NLU-engine (Rasa) en gecontaineriseerde backend-microservices, die via een REST API communiceren. Akamai's Firewall for AI werd geconfigureerd met semantische inspectieregels en JSONPath-filtering om zowel inkomend verkeer (Input Filtering) als uitgaande database-responsen (Output Filtering) te valideren. Gelijktijdig werd de Dynatrace OneAgent op host-niveau geïntegreerd voor gedistribueerde tracing en werden geautomatiseerde *Data Masking*-regels toegepast om Privacy-by-Design af te dwingen. Deze architectuur werd vervolgens systematisch geëvalueerd door middel van geautomatiseerde aanvalssimulaties (via Bash-scripts) en gerichte data-manipulaties.

De resultaten van de simulaties tonen aan dat de inzet van de Firewall for AI succes-

vol alle geteste aanvalsvectoren—waaronder applicatielaag-injecties, het lekken van PII en toxisch taalgebruik blokkeerde. Daarnaast bewijst de integratie van Dynatrace dat de volledige interactieketen reproduceerbaar kan worden vastgelegd in traces, waarbij gevoelige persoonsgegevens (PII) effectief worden verborgen nog vóór deze in logbestanden belanden.

Concluderend toont dit onderzoek aan dat de combinatie van observability en AI-gedreven beveiliging een aanzienlijk hoger niveau van controle, transparantie en incidentrespons biedt dan klassieke oplossingen afzonderlijk. Dit gelaagde *Defense-in-Depth* framework ondersteunt organisaties direct bij het naleven van privacy- en complianceverplichtingen, en vormt een essentiële, technisch gevalideerde stap richting veilige en juridisch verdedigbare AI-gestuurde klantcommunicatie.

Inhoudsopgave

Lijst van figuren	x
Lijst van tabellen	xi
Lijst van codefragmenten	xii
1 Inleiding	1
1.1 Probleemstelling	2
1.2 Onderzoeksvraag	2
1.3 Onderzoeksdoelstelling	3
1.4 Opzet van deze bachelorproef	4
2 Stand van zaken	5
2.1 Inleiding tot AI-gedreven Conversatiesystemen	6
2.1.1 Evolutie van Chatbots	7
2.1.2 Toepassingen in Enterprise-omgevingen	7
2.1.3 Noodzaak van Monitoring en Beveiliging	8
2.2 Architectuur van Large Language Models in Enterprise-omgevingen	8
2.2.1 Basisprincipes van Large Language Models	9
2.2.2 Transformer-architectuur	9
2.2.3 Integratie via API-gebaseerde Architecturen	9
2.2.4 SaaS versus Self-hosted Implementaties	10
2.2.5 Dataflow binnen AI-Chatbots	10
2.2.6 Architecturale Integratie van Monitoring en Firewalling	10
2.3 Securityrisico's bij AI-Chatbots	11
2.3.1 Prompt Injection	11
2.3.2 Jailbreaking van AI-modellen	12
2.3.3 Data Exfiltration via Modeloutput	12
2.3.4 Model Misuse en Abuse	12
2.3.5 Beperkingen van Traditionele Beveiligingsmechanismen	13
2.4 AI-Firewalls en Prompt Filtering	13
2.4.1 Definitie en Werking van een Firewall voor AI	13
2.4.2 Input Filtering en Output Filtering	14
2.4.3 Detectie van Toxiciteit en Ongepaste Inhoud	14
2.4.4 Anomaliedetectie in Conversatiepatronen	15
2.4.5 Verschillen met Klassieke Web Application Firewalls	15

2.5	Observability in Gedistribueerde Systemen	16
2.5.1	Definitie van Observability	16
2.5.2	De Drie Pijlers: Logs, Metrics en Traces	16
2.5.3	Distributed Tracing in Microservices-architecturen	16
2.5.4	Monitoring versus Observability	17
2.5.5	Uitdagingen bij AI-gedreven Systemen	17
2.6	Logging, Traceerbaarheid en Monitoring van AI-Systemen	18
2.6.1	Structurering van Conversatielogs	18
2.6.2	User- en Session-identificatie	18
2.6.3	Tijdstempels en Eventregistratie	19
2.6.4	Analyse van AI-output	19
2.6.5	Dashboarding en Visualisatie	20
2.6.6	Correlatie tussen Monitoringdata en Security-events	20
2.7	Correlatie tussen Security-events en Observabilitydata	20
2.7.1	Principes van Event Correlatie	21
2.7.2	Integratie van Threat Intelligence	21
2.7.3	SIEM-benaderingen in AI-omgevingen	22
2.7.4	Detectie van Afwijkende Interactiepatronen	22
2.7.5	Incidentrespons binnen AI-systemen	22
2.8	Privacy- en Ethiekvraagstukken bij AI Monitoring	23
2.8.1	GDPR en Dataminimalisatie	23
2.8.2	Privacy-by-Design in AI-systemen	24
2.8.3	Bewaartermijnen en Logretentie	24
2.8.4	Transparantie en Gebruikersrechten	24
2.8.5	Ethiek en Bias in AI-monitoring	25
2.9	Bestaande Oplossingen en Technologische Positionering	25
2.9.1	AI Observability Platforms	25
2.9.2	AI Security en Firewalloplossingen	26
2.9.3	Enterprise Monitoring Frameworks	26
2.9.4	Vergelijkende Analyse van Bestaande Benaderingen	27
2.9.5	Positionering van het Voorgestelde Framework	27

3 Methodologie **29**

4 Risicoanalyse **31**

4.1	Inleiding	31
4.2	Risicobeoordeling	31
4.2.1	Bepaling van de Waarschijnlijkheid en Impact	32
4.2.2	Bepaling van de Waarschijnlijkheid en Impact	32
4.3	Analyse van de Baseline versus Mitigatie	33
4.3.1	Scenario 1: Manipulatie van de Applicatielaag (OWASP A03:2021)	

4.3.2	Scenario 2: Uitputting van Resources (Bot-verkeer)	33
4.3.3	Scenario 3: Ongeoorloofde toegang tot Objecten (IDOR)	34
4.4	Wetgeving, Richtlijnen en Standaarden	34
4.4.1	Algemene Verordening Gegevensbescherming (AVG / GDPR)	34
4.4.2	NIS2-richtlijn	34
4.4.3	Payment Card Industry Data Security Standard (PCI-DSS)	34
4.4.4	OWASP Top 10	35
5	Proof of Concept	36
5.1	Inleiding	36
5.2	AI Firewall Regels en Configuratie	37
5.2.1	Architecturale Context: Intent-gebaseerde NLU versus Generatieve AI	37
5.2.2	Input Filtering (Bescherming van de backend)	37
5.2.3	Output Filtering (Bescherming van de eindgebruiker en bedrijfsdata)	38
5.2.4	Praktische Configuratie in het Akamai Control Center	38
5.3	Systeemarchitectuur en Integratie	41
5.3.1	Hybride Infrastructuur: Kubernetes-omgeving en Virtual Environments	42
5.3.2	Rasa NLU & Custom REST Channel Configuratie	44
5.3.3	Nginx Gateway en Routing	48
5.3.4	Geautomatiseerde Opstart van de Testomgeving	49
5.3.5	Interne Configuratie en Logica van de Digitale Assistent	51
5.4	Dynatrace Observability en Traceerbaarheid	54
5.4.1	Gedistribueerde Tracing via OneAgent	54
5.4.2	Akamai SIEM-Extensie via de Dynatrace ActiveGate	55
5.4.3	Privacy-by-Design: Upstream Data Masking en Geheugensanitatie	57
5.4.4	Privacy-by-Design en Data Masking	61
5.5	Uitvoering van Aanvalssimulaties en Beveiligingstesten	62
5.5.1	Simulatie: Applicatielaag Injecties en Input Filtering	62
5.5.2	De Rol en Beperkingen van Output Filtering	65
5.6	Dashboards en Rapportage	66
5.6.1	Akamai Security-Dashboard	66
5.6.2	Dynatrace Conversatie- en Observability-Dashboard	66
5.7	GDPR Compliance en Privacy-by-Design	69
5.7.1	Data Masking en Dataminimalisatie	69
5.7.2	Right to be Forgotten	69

6	Evaluatie	71
6.1	Evaluatiecriteria	71
6.1.1	Correctheid	71
6.1.2	Traceerbaarheid en Compliance	72
6.1.3	Gebruiksvriendelijkheid	72
7	Resultaten	73
7.1	Correctheid (Detectienauwkeurigheid)	73
7.1.1	Samenvatting van de geteste use-cases (True Positives)	73
7.1.2	Visueel bewijs van interventie	73
7.1.3	Beheer van False Positives en finetuning	75
7.2	Traceerbaarheid en Compliance	78
7.2.1	Upstream Dataminimalisatie	78
7.2.2	Attributie en Aansprakelijkheid	78
7.2.3	Uitvoering van het Recht op Gegevenswissing	78
7.3	Gebruiksvriendelijkheid (beheerder perspectief)	79
7.3.1	Configuratiegemak (Akamai)	79
7.3.2	Incidentenanalyse	79
7.3.3	Integratie en zichtbaarheid (Dynatrace)	79
8	Conclusie	81
8.1	Evaluatie van de Beveiligingsarchitectuur	81
8.2	Traceerbaarheid en Privacy-by-Design	82
8.3	Eindoordeel	82
9	Discussie	84
9.1	Aanbevelingen voor Toekomstig Onderzoek	84
A	Onderzoeksvoorstel	86
A.1	Inleiding	86
A.2	Literatuurstudie	88
A.2.1	AI-chatbots in bedrijfscontext	88
A.2.2	Beperkingen van traditionele beveiligingsmechanismen	89
A.2.3	Observability en auditability van AI-systemen	89
A.2.4	Privacy en regelgeving	89
A.3	Methodologie	90
A.4	Verwacht resultaat, conclusie	91
	Bibliografie	93

Lijst van figuren

3.1	Gantt-diagram	29
5.1	Opstelling PoC	37
5.2	Endpoint-scoping en JSONPath-configuratie binnen het Akamai Control Center.	39
5.3	Overzicht van de actieve AI-beveiligingsregels en hun specifieke sensitivity-profielen.	40
5.4	Detailweergave van de Modify-actie met een aangepaste JSON-respons voor niet-toegestane talen.	41
5.5	De geactiveerde Akamai-extensie binnen de centrale Dynatrace-omgeving.	55
5.6	De API-authenticatie en SIEM-pijplijnconfiguratie binnen Dynatrace.	56
5.7	Het Detection Findings overzicht met de live tijdlijn van de binnengehaalde security-events.	56
5.8	Detailweergave van een specifieke finding met forensische inzage in het exact geblokkeerde bericht.	57
5.9	Results of executing ./input.sh	64
5.10	Screenshot of what a user would see	65
5.11	Firewall for AI dashboard van Akamai	66
5.12	Overzicht van de binnenkomende chatbot-interacties en bijbehorende tekstberichten binnen Dynatrace.	67
5.13	Distributed trace in Dynatrace met de call tree, de responstijden en de bijbehorende prompt-inhoud.	68
5.14	Detailweergave van de Rasa-specifieke metadata en de confidence score binnen de trace.	68
7.1	Incidentenregistratie in het Akamai-dashboard	75
7.2	Akamai-registratie van de onterecht geblokkeerde catalogus-output.	75
7.3	Semantische misinterpretatie waarbij een legitieme vraag als toxisch werd gemarkeerd.	76
7.4	Inconsistente detectie van toxiciteit bij opzettelijke typfouten.	76
7.5	De chatbot raakt verward door een korte, onbekende taalstructuur en triggert een fallback-reactie.	77
7.6	Verwarde taalherkenning, waarbij pseudo-Latijn uiteindelijk werd gestopt op lengte.	77
A.1	Gantt-diagram	90

Lijst van tabellen

4.1	Risico-analyse van aanvalsvectoren voor de onbeveiligde chatbot-omgeving	32
5.1	Mapping van Input Firewall-regels naar Use-Cases	38
5.2	Mapping van Output Firewall-regels	38
6.1	Confusion matrix voor Firewall for AI-detectie	72
7.1	Overzicht van de uitgevoerde aanvalssimulaties en de resulterende Firewall-for-AI-blokkades	74

Lijst van codefragmenten

5.1	Gecustomiseerde JSON-payload gegenereerd door de Akamai edge bij een taalovertreiding.	41
5.2	Kubernetes manifest voor de frontend-laag met custom web-widget .	42
5.3	Structuur van de REST-payload voor optimale WAF-inspectie	44
5.4	Python-implementatie van het Secure REST Channel met OpenTelemetry en context-aware masking	45
5.5	Nginx configuratie fragment voor routing tussen pods en venv	48
5.6	Bash-script voor het efficiënt opstarten en afsluiten van de hybride PoC-omgeving	49
5.7	fragment van een Custom Action die de backend aanspreekt	52
5.8	Kubernetes CRD-configuratie van de OpenTelemetry Collector met OTTL transformatie-regels	59
5.9	Bash-script voor geautomatiseerde validatie van LLM Input-regels . . .	62
7.1	Terminal-output na een WAF-blokkade	74

1

Inleiding

De opkomst van AI-gestuurde chatbots en digitale assistenten heeft de manier waarop organisaties communiceren met klanten en medewerkers sterk veranderd. Steeds vaker worden deze systemen ingezet voor klantondersteuning, interne helpdesks en geautomatiseerde dienstverlening. Hoewel deze toepassingen efficiënt en schaalbaar zijn, brengen ze ook nieuwe beveiligings- en privacy-uitdagingen met zich mee. Interacties verlopen volledig digitaal en bevatten vaak gevoelige informatie, waardoor logging, monitoring en beveiliging cruciaal worden.

In moderne IT-omgevingen spelen observability- en securitytools een belangrijke rol bij het monitoren van applicaties en netwerkverkeer. Observabilityplatformen zoals Dynatrace maken het mogelijk om applicatiegedrag en gebruikersinteracties technisch in kaart te brengen, terwijl beveiligingsoplossingen zoals een Firewall for AI inkomende verzoeken analyseren en potentieel schadelijke of verdachte input blokkeren. In de praktijk worden deze systemen echter vaak los van elkaar gebruikt, waardoor een geïntegreerd overzicht van chatbotinteracties ontbreekt.

Dit leidt tot een belangrijk probleem: wanneer een beveiligingsincident optreedt tijdens een chatbotinteractie, is het vaak moeilijk om achteraf een volledig en coherent beeld te reconstrueren van wat er precies is gebeurd. Logs, traces en security-alerts bevinden zich in verschillende systemen en zijn niet automatisch met elkaar gecorreleerd. Hierdoor wordt incidentanalyse complexer, duurt mitigatie langer en ontstaat onzekerheid rond compliance en auditability.

Deze bachelorproef onderzoekt hoe observability- en securitygegevens geïntegreerd kunnen worden om chatbotinteracties volledig traceerbaar te maken, verdachte invoer automatisch te detecteren en incidenten efficiënt te analyseren en te beperken. Daarbij wordt specifiek gekeken naar de combinatie van Dynatrace voor

observability en Akamai's Firewall for AI voor security.

1.1. Probleemstelling

Organisaties die AI-gestuurde chatbots inzetten voor klantondersteuning of interne dienstverlening vertrouwen steeds vaker op observability- en securitytools om hun systemen te monitoren en te beveiligen. In de praktijk worden deze tools echter meestal afzonderlijk gebruikt. Observabilityplatformen registreren applicatiegedrag en gebruikersinteracties, terwijl beveiligingsoplossingen verdachte invoer detecteren en blokkeren. Er ontbreekt vaak een geïntegreerde aanpak waarbij deze gegevens automatisch met elkaar worden gecorreleerd.

Dit leidt tot een concreet probleem voor IT- en securityteams: wanneer zich een incident voordoet tijdens een chatbotinteractie, is het moeilijk om een volledig en coherent beeld te reconstrueren van wat er precies is gebeurd. Logs, traces en security-alerts bevinden zich in verschillende systemen en zijn niet structureel aan elkaar gekoppeld. Hierdoor verloopt incidentanalyse trager, is mitigatie minder efficiënt en ontstaat onzekerheid rond auditability en compliance.

Deze bachelorproef richt zich specifiek op organisaties die AI-gestuurde chatbots inzetten en gebruikmaken van Dynatrace voor observability en Akamai's Firewall for AI voor beveiliging. Voor deze doelgroep is er nood aan een geïntegreerd framework dat interacties volledig traceerbaar maakt, security-events koppelt aan applicatiedata en incidenten reproduceerbaar analyseerbaar maakt binnen de geldende privacy- en GDPR-richtlijnen.

1.2. Onderzoeksvraag

De centrale onderzoeksvraag van deze bachelorproef luidt als volgt:

Hoe kan een geïntegreerd framework op basis van Dynatrace's Audit Logging en Akamai's Firewall for AI worden ontworpen om AI-chatbotinteracties volledig traceerbaar en beveiligd te maken, met behoud van privacy en naleving van GDPR?

Om deze onderzoeksvraag te beantwoorden, worden volgende deelonderzoeksvragen geformuleerd.

Probleemdomein

- Hoe kan de interactiestroom tussen eindgebruiker en chatbot technisch in kaart gebracht worden via Dynatrace (wie, wat, wanneer)?
- Welke soorten aanvallen of verdachte inputs kan Akamai's firewall for AI de-

tecteren en blokkeren tijdens chatbotverkeer?

- Hoe goed kunnen observability-gegevens uit Dynatrace gecorreleerd worden met security-events uit Akamai om een volledig beeld te vormen van een gesprek?
- Hoe vertalen we de gecorreleerde Dynatrace-traces en Akamai-alerts naar een direct uitvoerbaar mitigatie-pad (detectie → analyse → containment → herstel) om incidenten reactief en meetbaar te beperken?
- In welke mate voldoet het systeem aan privacy- en GDPR-richtlijnen bij het loggen en bewaren van conversatiegegevens?
- Hoe betrouwbaar is het audit-trailsysteem bij het reconstrueren van volledige gesprekken en events?

Oplossingsdomein

- Hoe kan Dynatrace gebruikt worden om afwijkende patronen in chatbotinteracties automatisch te detecteren en te classificeren als potentieel risico?
- Welke type aanvallen of risicovolle inputs detecteert Akamai's firewall for AI het meest effectief, en hoe kunnen deze detecties worden teruggekoppeld naar Dynatrace voor correlatie?
- Hoe kunnen observability-data en security-events gecombineerd worden om een automatische mitigatieactie te triggeren (zoals blokkeren, isoleren of waarschuwen)?
- Welke vorm van audit trail of chain-of-evidence is het meest geschikt om incidenten achteraf reproduceerbaar en juridisch verdedigbaar te maken?
- In welke mate voldoet het geïntegreerde framework aan GDPR- en privacy-by-designrichtlijnen bij het loggen, analyseren en mitigerend reageren op incidenten?

1.3. Onderzoeksdoelstelling

De doelstelling van deze bachelorproef is het ontwerpen, implementeren en evalueren van een proof-of-concept waarin een AI-chatbot wordt geïntegreerd met een observabilitylaag en een firewall gebaseerde AI-bescherming.

Het onderzoek richt zich op het realiseren van een werkend prototype van een digitale assistent (met een afgebakende use-case, zoals klantondersteuning bij bestellingen) dat:

- geïntegreerd is met een Dynatrace-observabilitylaag voor logging, monitoring en analyse van conversatiegegevens;
- beschermd wordt door een firewall for AI (Akamai) die verdachte of kwaadaardige prompts detecteert en filtert;
- inzicht biedt in interacties via dashboards met conversatielogs, gebruikers- en sessiegegevens, tijdstempels en AI-analyses (zoals toxiciteit, anomalieën en trends);
- security-events en threat-intelligence correleert met observabilitydata.

Het concrete eindresultaat bestaat uit:

1. Een functionerend technisch prototype waarin chatbot, observabilityplatform en firewall correct geïntegreerd zijn.
2. Een Dynatrace-dashboard met gestructureerde logging, traceerbaarheid en analyse van interacties.
3. Een Akamai security-dashboard dat filtering, detectie van verdachte prompts en correlatie met monitoringdata aantoont.
4. Een technische documentatie met duidelijke architectuur, onderbouwde ontwerpkeuzes en kritische evaluatie van veiligheid, privacy en haalbaarheid.

De bachelorproef wordt als geslaagd beschouwd wanneer het prototype aantoont dat chatbotinteracties reproduceerbaar, traceerbaar en beveiligd kunnen worden gemonitord, waarbij filtering, logcorrelatie en security-controle effectief functioneren binnen de voorgestelde architectuur.

1.4. Opzet van deze bachelorproef

De rest van deze bachelorproef is als volgt opgebouwd:

In Hoofdstuk 2 wordt een overzicht gegeven van de stand van zaken binnen het onderzoeksdomein, op basis van een literatuurstudie rond observability, AI-security, audit logging en privacy by design.

In Hoofdstuk 3 wordt de methodologie toegelicht en worden de gebruikte onderzoekstechnieken besproken om een antwoord te kunnen formuleren op de onderzoeksvragen.

In Hoofdstuk 8, tenslotte, wordt de conclusie gegeven en een antwoord geformuleerd op de onderzoeksvragen.

2

Stand van zaken

Dit hoofdstuk situeert de bachelorproef binnen het bestaande wetenschappelijke en technologische kader rond AI-gedreven conversatiesystemen, observability en AI-specifieke beveiliging. In het vorige hoofdstuk werd de probleemstelling geformuleerd, waarbij werd vastgesteld dat AI-chatbots steeds vaker worden geïntegreerd in enterprise-omgevingen, maar dat monitoring en beveiliging vaak los van elkaar worden geïmplementeerd.

De literatuurstudie onderzoekt daarom de onderliggende concepten, architecturen en beveiligingsuitdagingen die relevant zijn voor het ontwerpen van een geïntegreerde oplossing. Eerst wordt de werking en evolutie van AI-chatbots besproken, gevolgd door de architecturale kenmerken van Large Language Models in enterprise-context. Vervolgens worden de belangrijkste securityrisico's en AI-firewall-mechanismen behandeld. Daarna wordt ingezoomd op observability, logging en eventcorrelatie binnen gedistribueerde systemen. Tot slot worden privacy- en ethische aspecten besproken en wordt het voorgestelde framework gepositioneerd ten opzichte van bestaande oplossingen.

De nadrukkelijke theoretische focus op Large Language Models (LLM's) binnen deze literatuurstudie komt door het feit dat moderne AI-beveiligingsmechanismen (zoals Akamai's Firewall for AI) specifiek zijn ontwikkeld om de unieke semantische risico's van generatieve taalmodellen te mitigeren. Voor de praktische uitvoering van de Proof of Concept (PoC) is echter bewust gekozen voor een intentgebaseerd Natural Language Understanding (NLU)-systeem via Rasa. Omdat een NLU-engine deterministisch opereert op basis van gestructureerde trainingsdata en vooraf gedefinieerde intenties, is de applicatie van nature immuun voor LLM-specifieke kwetsbaarheden zoals *prompt injection* of *jailbreaking*. Mislukte of kwaadaardige instructies resulteren hier simpelweg in een gecontroleerde *intent-fallback*.

Een directe transitie naar een LLM zonder een dergelijke NLU-nulmeting vergroot het risico op hallucinaties aanzienlijk. De fundamentele architectuur van generatieve AI stuurt immers primair op taalkundige vloeiendheid en probabilistische coherentie, zonder dat het model een feitelijk of logisch begrip van de onderliggende data bezit Bender e.a., 2021a. Een NLU-model kan eenvoudige, repetitieve workflows met een minimale inzet van resources afhandelen, waar de directe inzet van een rekenintensief en kostbaar LLM overdreven en inefficiënt zou zijn Anonymous, 2024.

Ondanks dat de chatbot door zijn NLU-structuur inherent immuun is voor semantische manipulaties, toont Akamai's Firewall for AI binnen de PoC aan dat hij deze aanvalsvectoren (zoals code-injecties en toxisch taalgebruik) al effectief op de edge kan herkennen en proactief blokkeren nog voordat ze de API bereiken. Rasa dient zo als de ideale, veilige *stepping stone* binnen het gelaagde *Defense-in-Depth* framework. Het maakt het mogelijk om de end-to-end traceerbaarheid via Dynatrace en de semantische inspectie van Akamai op inkomende en uitgaande JSON-payloads objectief te testen en te kalibreren, zonder dat de testresultaten worden verstoord door de inherente onvoorspelbaarheid en dynamische output van een live generatief model.

Hoewel de industrie een hybride architectuur aanbeveelt waarin een LLM fungeert als flexibele fallback bij een lage *confidence score* Rasa, 2026, is deze generatieve laag binnen de huidige PoC bewust weggelaten. Hiermee wordt voorkomen dat de assistent onvoorspelbaar reageert op willekeurige of out-of-scope gebruikersvragen (zoals *"What time is it?"*), waardoor de gestructureerde NLU via *Flows* de absolute controle over de bedrijfslogica behoudt. Vanaf dit technisch in orde is, kan de organisatie de overstap naar een volledige LLM-omgeving risicoloos en gecontroleerd realiseren.

Dit hoofdstuk vormt de conceptuele basis voor de methodologie in het volgende hoofdstuk, waarin wordt toegelicht hoe deze inzichten worden toegepast in het ontwerp en de implementatie van de proof-of-concept.

2.1. Inleiding tot AI-gedreven Conversatiesystemen

AI-gedreven conversatiesystemen, vaak aangeduid als chatbots of virtuele assistenten, zijn softwaretoepassingen die natuurlijke taal gebruiken om interactie met gebruikers mogelijk te maken. Waar vroege chatbots voornamelijk werkten op basis van vooraf gedefinieerde regels en beslissingsbomen, maken moderne systemen gebruik van machine learning en, meer recent, Large Language Models (LLM's). Deze evolutie heeft geleid tot een significante toename in flexibiliteit, schaalbaar-

heid en contextbegrip binnen menselijke-computerinteractie (Adamopoulou & Mousiades, 2020b).

De opkomst van generatieve AI heeft het toepassingsgebied van chatbots verder uitgebreid. In tegenstelling tot traditionele Natural Language Processing (NLP)-systemen, die vaak beperkt zijn tot intentieherkenning en vooraf geprogrammeerde antwoorden, kunnen LLM-gebaseerde systemen autonoom tekst genereren op basis van context en probabilistische taalmodellen (Brown e.a., 2020). Hierdoor zijn ze in staat om complexere en meer natuurlijke dialogen te voeren.

2.1.1. Evolutie van Chatbots

De eerste generatie chatbots, zoals ELIZA in de jaren 60, functioneerden op basis van patroonherkenning en eenvoudige teksttransformaties (Weizenbaum, 1966). Deze systemen simuleerden conversatie zonder daadwerkelijk begrip van taalstructuur of semantiek. In de daaropvolgende decennia werden rule-based systemen uitgebreid met decision trees en keyword matching, wat leidde tot meer gestructureerde maar nog steeds beperkte interacties.

Met de opkomst van machine learning en statistische NLP-technieken verschoof de focus naar intentieclassificatie en entiteitsextractie. Platforms zoals dialog management frameworks maakten het mogelijk om schaalbare klantenservicebots te ontwikkelen. Toch bleven deze systemen sterk afhankelijk van gestructureerde trainingsdata en vooraf gedefinieerde intenties (Jurafsky & Martin, 2026).

De introductie van transformer-gebaseerde modellen betekende een fundamentele verschuiving. Het transformer-architectuurmodel, geïntroduceerd door Vaswani e.a. (2017), maakte het mogelijk om lange-afstandsafhankelijkheden in tekst efficiënt te modelleren. Hieruit ontstonden Large Language Models zoals GPT, die getraind worden op enorme tekstcorpora en in staat zijn coherente, contextbewuste tekst te genereren (Brown e.a., 2020). Dit zijn grote, gestructureerde verzameling van geschreven of gesproken teksten in een bepaalde taal. Deze modellen vormen vandaag de basis van moderne AI-chatbots.

2.1.2. Toepassingen in Enterprise-omgevingen

Binnen enterprise-omgevingen worden AI-chatbots steeds vaker ingezet voor uiteenlopende toepassingen, waaronder klantenondersteuning, interne helpdesks, kennisbeheer en procesautomatisering. De schaalbaarheid en 24/7-beschikbaarheid maken deze systemen economisch aantrekkelijk (Dwivedi & et al., 2023).

Tegelijkertijd brengt de integratie van generatieve AI binnen bedrijfsomgevingen nieuwe uitdagingen met zich mee. In tegenstelling tot klassieke systemen genereren LLM's dynamische output, wat betekent dat antwoorden niet volledig voorspelbaar zijn. Dit verhoogt het risico op foutieve informatie, ongepaste inhoud of het lekken van gevoelige data (Bommasani e.a., [2021](#)).

Bovendien worden AI-chatbots vaak geïntegreerd via API's in bredere IT-architecturen, waarbij zij toegang krijgen tot interne databronnen. Hierdoor verschuift het risico-profiel van enkel functionele foutmarges naar potentiële security- en compliance-problemen.

2.1.3. Noodzaak van Monitoring en Beveiliging

De probabilistische aard van generatieve AI maakt traditionele kwaliteitscontrole-mechanismen onvoldoende. Waar klassieke software deterministisch gedrag vertoont, kunnen LLM's verschillende antwoorden genereren op identieke input. Dit bemoeilijkt validatie, auditing en incidentanalyse.

Onderzoekers wijzen erop dat observability en security in AI-systemen vaak als afzonderlijke componenten worden benaderd, terwijl een geïntegreerde aanpak noodzakelijk is om anomalieën effectief te detecteren en te analyseren (Bommasani e.a., [2021](#)). Logging van conversaties, traceerbaarheid van interacties en correlatie van security-events zijn cruciale elementen om verantwoord gebruik van AI-systemen te waarborgen.

De toenemende afhankelijkheid van AI binnen kritieke bedrijfsprocessen onderstreept daarom de noodzaak van geïntegreerde monitoring- en beveiligingsmechanismen. Dit vormt de basis voor het verdere onderzoek in deze bachelorproef.

2.2. Architectuur van Large Language Models in Enterprise-omgevingen

Large Language Models (LLM's) vormen de technologische kern van moderne AI-gedreven conversatiesystemen. Deze modellen zijn gebaseerd op deep learning-architecturen en worden getraind op grootschalige tekstcorpora om probabilistische taalpatronen te leren. Binnen enterprise-omgevingen worden LLM's doorgaans niet als standalone systemen gebruikt, maar geïntegreerd als component binnen een bredere IT-architectuur.

Dit hoofdstuk bespreekt de onderliggende architectuurprincipes van LLM's, hun

integratiemodellen binnen bedrijfsomgevingen en de gevolgen voor monitoring en beveiliging.

2.2.1. Basisprincipes van Large Language Models

LLM's zijn neurale netwerken met miljarden parameters die taal modelleren op basis van waarschijnlijkheidsverdelingen. In plaats van vaste regels toe te passen, voorspellen zij het volgende token in een sequentie op basis van context (Brown e.a., 2020). Hierdoor kunnen zij coherente en contextbewuste tekst genereren.

De schaal van deze modellen is een bepalende factor voor hun performantie. Kaplan e.a. (2020) tonen aan dat modelprestaties systematisch verbeteren naarmate het aantal parameters, trainingsdata en rekencapaciteit toenemen. Dit schaalprincipe verklaart de sterke vooruitgang in generatieve AI-systemen.

2.2.2. Transformer-architectuur

De meeste hedendaagse LLM's zijn gebaseerd op de transformer-architectuur, geïntroduceerd door Vaswani et al. (Vaswani e.a., 2017). Het kernmechanisme van deze architectuur is self-attention, waarmee het model relevante woorden in een zin kan wegen ten opzichte van elkaar, ongeacht hun positie in de tekst.

In tegenstelling tot recurrente neurale netwerken maakt de transformer parallelle verwerking mogelijk, wat schaalbaarheid en efficiëntie aanzienlijk verhoogt. Dit maakt het trainen van zeer grote modellen haalbaar op gedistribueerde infrastructuur.

Voor enterprise-toepassingen betekent dit dat de intelligentie voornamelijk in het model zelf zit, terwijl de applicatielaag zich richt op interfacing, validatie, logging en beveiliging.

2.2.3. Integratie via API-gebaseerde Architecturen

Binnen bedrijfsomgevingen worden LLM's vaak geïntegreerd via API's. De applicatie dient hierbij als tussenlaag tussen gebruiker en model. De typische dataflow verloopt als volgt:

1. Gebruiker stuurt een prompt naar de applicatie.
2. De applicatie valideert de input.
3. De prompt wordt doorgestuurd naar het LLM via een API-call.
4. Het model genereert een response.
5. De applicatie verwerkt, filtert en logt de output.

Dit architectuurmodel verduidelijkt dat beveiliging en observability niet in het model zelf zitten, maar in de omliggende infrastructuur. Hierdoor ontstaat een duidelijke noodzaak om input- en outputstromen systematisch te monitoren.

2.2.4. SaaS versus Self-hosted Implementaties

Organisaties kunnen kiezen tussen cloudgebaseerde (SaaS) LLM-oplossingen of self-hosted modellen. SaaS-oplossingen bieden schaalbaarheid en eenvoud in beheer, maar brengen afhankelijkheid van externe providers met zich mee. Self-hosted modellen bieden meer controle over data en configuratie, maar vereisen aanzienlijke infrastructuur en expertise (Bommasani e.a., [2021](#)).

De keuze tussen beide modellen heeft directe implicaties voor:

- Databeheer en compliance
- Loggingmogelijkheden
- Beveiligingscontroles
- Observability-integratie

2.2.5. Dataflow binnen AI-Chatbots

De integratie van LLM's in enterprise-systemen gebeurt zelden zonder bijkomende componenten. Vaak worden retrieval-mechanismen toegevoegd (Retrieval-Augmented Generation, RAG), waarbij externe databronnen worden geraadpleegd om context te verkrijgen (Lewis e.a., [2020](#)).

Dit introduceert extra complexiteit in de dataflow:

- Query naar kennisdatabase
- Verrijking van prompt
- Modelinferentie
- Outputvalidatie

Elke stap vormt een potentieel controlepunt voor logging en beveiliging.

2.2.6. Architecturale Integratie van Monitoring en Firewalling

Aangezien LLM's probabilistisch gedrag vertonen, kunnen ongewenste outputs of manipulaties niet volledig binnen het model zelf worden uitgesloten. Daarom wordt in recente literatuur gepleit voor een "defence-in-depth"-benadering rond generatieve AI-systemen (Bommasani e.a., [2021](#)).

Dit betekent dat monitoring- en beveiligingslagen worden toegevoegd rond het model:

- Input filtering (detectie van prompt injection)
- Output filtering (toxicity, data leakage)
- Logging en traceerbaarheid
- Event correlatie

Deze architecturale positionering vormt de basis voor de proof-of-concept dat in deze bachelorproef wordt ontwikkeld.

2.3. Securityrisico's bij AI-Chatbots

De integratie van Large Language Models binnen enterprise-omgevingen introduceert nieuwe beveiligingsrisico's die fundamenteel verschillen van traditionele applicatiebeveiliging. In tegenstelling tot klassieke software, die deterministisch gedrag vertoont, genereren LLM-gebaseerde systemen probabilistische output. Dit maakt hun gedrag minder voorspelbaar en complexer om te beveiligen (Bommasani e.a., [2021](#)).

Recente studies tonen aan dat generatieve AI-systemen kwetsbaar zijn voor manipulatie via invoerprompts, datalekken via modeloutput en misbruik voor schadelijke doeleinden (OWASP Foundation, [2023](#)). Dit hoofdstuk bespreekt de belangrijkste dreigingen die relevant zijn voor enterprise-implementaties.

2.3.1. Prompt Injection

Prompt injection is een aanvalstechniek waarbij een gebruiker kwaadaardige instructies verwerkt in een prompt om het gedrag van het model te manipuleren. Omdat LLM's instructies interpreteren als natuurlijke taal en geen strikt onderscheid maken tussen systeeminstructies en gebruikersinput, kunnen zij worden misleid om beveiligingsrichtlijnen te negeren (Carlini e.a., [2023](#)).

Een voorbeeld hiervan is het overschrijven van systeeminstructies via een extra prompt, zoals het vragen om verborgen beleidsregels of het negeren van eerder opgelegde restricties. Dit vormt een aanzienlijk risico wanneer chatbots toegang hebben tot interne databronnen of gevoelige bedrijfsinformatie.

Prompt injection verschilt van klassieke code-injectie doordat het geen syntactische kwetsbaarheid uitbuit, maar inspeelt op het interpretatievermogen van het model. Traditionele firewalls detecteren dit type aanval doorgaans niet, aangezien de invoer op technisch niveau geldig blijft.

2.3.2. Jailbreaking van AI-modellen

Jailbreaking verwijst naar technieken waarmee gebruikers de ingebouwde veiligheidsmechanismen van een model proberen te omzeilen. Dit gebeurt vaak door hypothetische scenario's of role-playing prompts te gebruiken die het model aanzetten om ongewenste of verboden informatie te genereren (Zou e.a., [2023](#)).

Onderzoek toont aan dat zelfs geavanceerde modellen vatbaar blijven voor dergelijke manipulaties, ondanks fine-tuning en reinforcement learning with human feedback (RLHF). Dit wijst op inherente beperkingen in het afdwingen van gedragsrestricties binnen probabilistische modellen.

Voor enterprise-omgevingen betekent dit dat modeloutput niet als veilig mag worden beschouwd en aanvullende controlemechanismen vereist zijn.

2.3.3. Data Exfiltration via Modeloutput

Wanneer AI-systemen geïntegreerd worden met interne databronnen, bijvoorbeeld via Retrieval-Augmented Generation (RAG), ontstaat het risico dat gevoelige informatie onbedoeld wordt blootgesteld (Lewis e.a., [2020](#)).

Aanvallers kunnen gerichte prompts gebruiken om vertrouwelijke gegevens op te vragen of om indirect toegang te verkrijgen tot interne kennisdatabanken. Daarnaast kan trainingsdata-leakage optreden wanneer modellen specifieke informatie reproduceren die tijdens training werd opgenomen (Carlini e.a., [2021](#)).

Dit type dreiging is bijzonder problematisch in sectoren waar vertrouwelijkheid en compliance centraal staan, zoals gezondheidszorg en financiële dienstverlening.

2.3.4. Model Misuse en Abuse

Generatieve AI kan worden misbruikt voor het genereren van phishingmails, desinformatie of schadelijke code. Hoewel dit risico niet noodzakelijk voortkomt uit een technische kwetsbaarheid, heeft het belangrijke implicaties voor verantwoord gebruik (Dwivedi & et al., [2023](#)).

Binnen enterprise-contexten kan misuse ook intern optreden, bijvoorbeeld wanneer medewerkers AI-systemen gebruiken om beveiligingsmechanismen te omzeilen of gevoelige informatie te extraheren. Dit onderstreept de noodzaak van logging en monitoring van gebruikersinteracties.

2.3.5. Beperkingen van Traditionele Beveiligingsmechanismen

Traditionele beveiligingsmaatregelen, zoals Web Application Firewalls (WAF's), zijn ontworpen om syntactische patronen en bekende aanvalssignalen te detecteren. AI-specifieke dreigingen zijn echter semantisch van aard en maken gebruik van natuurlijke taalmanipulatie (OWASP Foundation, [2023](#)).

Daardoor volstaat klassieke perimeterbeveiliging niet. Er is nood aan:

- Semantische analyse van prompts
- Contextbewuste filtering
- Detectie van afwijkende interactiepatronen
- Correlatie van security-events met gebruikerssessies

Deze vaststellingen vormen de theoretische onderbouwing voor het implementeren van een Firewall for AI in combinatie met een observabilitylaag, zoals voorgesteld in deze bachelorproef.

2.4. AI-Firewalls en Prompt Filtering

De opkomst van generatieve AI heeft geleid tot de ontwikkeling van gespecialiseerde beveiligingsmechanismen die specifiek gericht zijn op Large Language Model (LLM)-toepassingen. In tegenstelling tot traditionele beveiligingsoplossingen, die zich focussen op netwerk- of applicatieniveau, richten Firewalls for AI zich op semantische analyse van taalinput en -output.

Aangezien dreigingen zoals prompt injection en jailbreaking niet gebaseerd zijn op syntactische kwetsbaarheden maar op semantische manipulatie, is een aanvullende beveiligingslaag noodzakelijk (OWASP Foundation, [2023](#)). Dit hoofdstuk bespreekt de werking, architectuur en beperkingen van AI-firewalls binnen enterprise-omgevingen.

2.4.1. Definitie en Werking van een Firewall voor AI

Een Firewall for AI kan worden gedefinieerd als een beveiligingscomponent die inkomende en uitgaande interacties met een AI-model analyseert, evalueert en indien nodig filtert voordat deze het model bereiken of de gebruiker terugkoppelen. In tegenstelling tot klassieke firewalls inspecteert een Firewall for AI geen netwerk-pakketten, maar taalinhoud.

De kernfunctionaliteiten zijn:

- Detectie van kwaadaardige of manipulatieve prompts
- Classificatie van risicovolle content
- Filtering van ongepaste of gevoelige output
- Logging van verdachte interacties

Onderzoek benadrukt dat dergelijke beveiligingslagen essentieel zijn om modellen veilig te integreren in bedrijfsprocessen (Bommasani e.a., [2021](#)).

2.4.2. Input Filtering en Output Filtering

Firewalls for AI werken op twee niveaus:

Input Filtering

Hier wordt de gebruikersprompt geanalyseerd voordat deze naar het model wordt doorgestuurd. Mechanismen kunnen bestaan uit:

- Toxiciteitsdetectie
- Detectie van prompt injection-patronen
- Analyse van afwijkende instructies

Door inputvalidatie kan worden voorkomen dat het model ongewenste context ontvangt.

Output Filtering

Na generatie van de modeloutput wordt deze opnieuw geëvalueerd. Dit is noodzakelijk omdat LLM's probabilistische output genereren en veiligheidsmechanismen binnen het model niet foutloos zijn (Zou e.a., [2023](#)).

Output filtering kan onder meer bestaan uit:

- Detectie van gevoelige informatie
- Controle of antwoorden aan beleid voldoen
- Anomaliedetectie in gegenereerde tekst

Deze aanpak sluit aan bij het principe van defence-in-depth.

2.4.3. Detectie van Toxiciteit en Ongepaste Inhoud

Een belangrijk onderdeel van AI-firewalls is contentmoderatie. Onderzoek toont aan dat LLM's, ondanks alignment-technieken, nog steeds ongepaste of schadelijke inhoud kunnen genereren onder bepaalde omstandigheden (Gehman e.a.,

2020).

Automatische classificatiemodellen worden ingezet om:

- Hate speech te detecteren
- Beledigende taal te identificeren
- Discriminerende of gewelddadige inhoud te markeren

Deze detectiemechanismen worden vaak geïntegreerd als externe API's of als aparte machine learning-modellen binnen de beveiligingslaag.

2.4.4. Anomaliedetectie in Conversatiepatronen

Naast inhoudsanalyse kan ook gedragsanalyse worden toegepast. Door interactiepatronen te monitoren (zoals frequentie, promptstructuur of herhalende manipulatieve pogingen) kunnen afwijkingen worden gedetecteerd.

Anomaliedetectie wordt vaak toegepast in cybersecuritycontexten en kan worden uitgebreid naar AI-interacties (Chandola e.a., 2009). Door security-events te correleren met sessiegegevens ontstaat een volledig risicobeeld.

Dit aspect is bijzonder relevant voor enterprise-omgevingen waar AI-systemen publiek toegankelijk zijn.

2.4.5. Verschillen met Klassieke Web Application Firewalls

Klassieke Web Application Firewalls (WAF's) analyseren HTTP-verkeer op basis van bekende aanvalssignaturen, zoals SQL-injectie of cross-site scripting. AI-dreigingen opereren echter op semantisch niveau en zijn contextafhankelijk (OWASP Foundation, 2023).

Belangrijke verschillen zijn:

- Syntactische versus semantische detectie
- Deterministische versus probabilistische output
- Contextafhankelijke risicoanalyse

Hieruit volgt dat AI-firewalls geen vervanging zijn van traditionele beveiliging, maar een aanvullende laag vormen binnen een meerlagige beveiligingsarchitectuur.

De besproken concepten vormen de theoretische basis voor het implementeren van een AI-gebaseerde firewall in combinatie met een observabilitylaag, zoals verder uitgewerkt in deze bachelorproef.

2.5. Observability in Gedistribueerde Systemen

Moderne enterprise-applicaties worden steeds vaker opgebouwd volgens gedistribueerde architecturen, zoals microservices en cloud-native omgevingen. In dergelijke systemen is het gedrag van individuele componenten niet langer eenvoudig te reconstrueren via traditionele monitoringtechnieken. Observability is daarom uitgegroeid tot een essentieel concept binnen hedendaags systeembeheer (Sigelman & et al., 2010).

Binnen de context van AI-gedreven systemen, waarbij meerdere lagen (frontend, API, modelinference, databronnen, beveiligingslaag) samenwerken, wordt observability cruciaal voor het analyseren van prestaties, fouten en beveiligingsincidenten.

2.5.1. Definitie van Observability

De term observability vindt zijn oorsprong in de controletheorie en verwijst naar hoe ver de interne toestand van een systeem kan worden afgeleid uit externe outputs. Toegepast op software betekent dit dat ontwikkelaars en beheerders in staat moeten zijn om het gedrag van een systeem te begrijpen op basis van gegenereerde Prestatiegegevens (Ganesan, 2022).

In tegenstelling tot traditionele monitoring, dat vooraf gedefinieerde metrics opvolgt, richt observability zich op het beantwoorden van onverwachte vragen over systeemgedrag. Dit is bijzonder relevant voor AI-systemen, waar afwijkend gedrag niet altijd vooraf voorspelbaar is.

2.5.2. De Drie Pijlers: Logs, Metrics en Traces

Observability wordt doorgaans opgebouwd rond drie kerncomponenten:

- **Logs:** Gedetailleerde, tijdsgebonden registraties van gebeurtenissen binnen een systeem.
- **Metrics:** Numerieke waarden die prestaties of statusindicatoren weergeven.
- **Traces:** End-to-end weergave van een verzoek doorheen verschillende systeemcomponenten.

Distributed tracing, zoals geïntroduceerd in systemen zoals Dapper (Sigelman & et al., 2010), maakt het mogelijk om individuele requests te volgen over meerdere services heen.

2.5.3. Distributed Tracing in Microservices-architecturen

Microservices-architecturen splitsen applicaties op in afzonderlijke, onafhankelijke services. Hoewel dit schaalbaarheid en flexibiliteit verhoogt, bemoeilijkt het fout-

opsporing en prestatieanalyse.

Distributed tracing lost dit probleem op door unieke trace-ID's toe te kennen aan verzoeken, waardoor hun volledige traject kan worden gereconstrueerd (Sigelman & et al., [2010](#)). In AI-contexten kan een enkele gebruikersprompt meerdere interne processen activeren, zoals:

- Authenticatiecontrole
- Inputvalidatie
- API-aanroep naar het LLM
- Retrieval van externe databronnen
- Outputfiltering

Zonder tracing is het moeilijk om te achterhalen waar vertragingen of beveiligingsproblemen ontstaan.

2.5.4. Monitoring versus Observability

Hoewel de termen vaak door elkaar worden gebruikt, is er een fundamenteel verschil tussen monitoring en observability. Monitoring focust op het opvolgen van vooraf bepaalde indicatoren en alarmen. Observability daarentegen laat toe om onbekende problemen te onderzoeken op basis van beschikbare prestatiegegevens (Ganesan, [2022](#)).

In AI-systemen, waar output dynamisch en contextafhankelijk is, kunnen afwijkingen niet altijd via statische regels worden gedetecteerd. Observability maakt het mogelijk om onverwachte patronen of anomalieën te identificeren via loganalyse en correlatie.

2.5.5. Uitdagingen bij AI-gedreven Systemen

AI-systemen introduceren bijkomende observability-uitdagingen:

- Niet-deterministische output
- Complexe dataflows
- Hoge afhankelijkheid van externe API's
- Mogelijke privacybeperkingen bij logging van gebruikersinteracties

Onderzoekers benadrukken dat het gebrek aan transparantie in AI-modellen (de zogenaamde “black-box”-problematiek) extra nadruk legt op externe observabilitymechanismen (Bommasani e.a., [2021](#)).

Een geïntegreerde aanpak waarbij logs, metrics en security-events worden gecorreleerd, is daarom essentieel om betrouwbare en veilige AI-systemen te realiseren.

2.6. Logging, Traceerbaarheid en Monitoring van AI-Systemen

De integratie van AI-systemen binnen enterprise-architecturen vereist een gestructureerde aanpak voor logging en traceerbaarheid. In tegenstelling tot traditionele applicaties genereren Large Language Models (LLM's) probabilistische output, waardoor het gedrag van het systeem niet volledig deterministisch is. Dit bemoeilijkt auditing, foutanalyse en incidentonderzoek.

Recente literatuur benadrukt dat observability bij AI-systemen verder moet gaan dan prestatiegegevens en ook semantische interacties moet omvatten (Ganesan, 2022). Logging vormt hierbij de primaire bron van waarheid voor analyse en reconstructie van gebeurtenissen.

2.6.1. Structurering van Conversatielogs

Voor AI-chatbots is het onvoldoende om enkel technische systeemlogs bij te houden. Ook inhoudelijke interacties moeten worden geregistreerd, rekening houdend met privacyprincipes. Een gestructureerde logregistratie omvat doorgaans:

- Gebruikersprompt
- Modelresponse
- Tijdstempel
- Sessie-identificatie
- Eventuele verrijkte context (bijvoorbeeld RAG-bronnen)

Volgens de principes van distributed tracing (Sigelman & et al., 2010) moet elk verzoek een unieke identifier bevatten, zodat het volledige traject van een interactie kan worden gereconstrueerd.

Een correcte logstructuur maakt zowel technische debugging als security-analyse mogelijk.

2.6.2. User- en Session-identificatie

Traceerbaarheid vereist dat interacties kunnen worden gekoppeld aan specifieke sessies of gebruikerscontexten. Dit betekent niet noodzakelijk dat persoonlijke identificatiegegevens worden opgeslagen, maar wel dat er een consistente correlatiesleutel aanwezig is.

Binnen cybersecurity wordt dit principe vaak toegepast in Security Information and Event Management (SIEM)-systemen, waar events worden gecorreleerd op basis van tijd, bron en sessie (Kotenko e.a., [2022](#)).

Voor AI-systemen betekent dit dat verdachte prompts of anomalieën moeten kunnen worden gelinkt aan eerdere interacties binnen dezelfde sessie.

2.6.3. Tijdstempels en Eventregistratie

Tijdssynchronisatie is essentieel voor analyse. Zonder nauwkeurige tijdstempels is het onmogelijk om volgordes van gebeurtenissen correct te reconstrueren.

In gedistribueerde systemen kunnen kleine tijdsverschillen leiden tot foutieve interpretaties van incidenten. Daarom wordt in moderne observabilitypraktijken gebruikgemaakt van gecentraliseerde logging-architecturen en uniforme tijdstandaarden (Ganesan, [2022](#)).

Voor AI-toepassingen is dit bijzonder relevant wanneer meerdere lagen betrokken zijn, zoals:

- Authenticatieservices
- AI-firewallcomponenten
- LLM-API-aanroepen
- Outputvalidatie

2.6.4. Analyse van AI-output

Naast technische logging moet ook de inhoud van AI-output geanalyseerd kunnen worden. Onderzoek toont aan dat LLM's onder bepaalde omstandigheden gevoelige of ongepaste informatie kunnen genereren (Gehman e.a., [2020](#)).

Daarom kan logging worden uitgebreid met aanvullende metadata, zoals:

- Toxiciteitsscores
- Risicoclassificatie
- Detectie van policy-violations
- Anomaliescores

Door deze metadata te koppelen aan conversatielogs ontstaat er een dataset voor incidentanalyse.

2.6.5. Dashboarding en Visualisatie

Het verzamelen van logs is slechts de eerste stap. Effectieve monitoring vereist visualisatie en analyse via dashboards. Moderne observabilityplatformen combineren logs, metrics en traces in gecorreleerde weergaven (Ganesan, 2022).

Voor AI-systemen kunnen dashboards onder meer inzicht bieden in:

- Frequentie van verdachte prompts
- Trendanalyse van toxiciteit
- Correlatie tussen security-events en gebruikerssessies

Visualisatie maakt het mogelijk om afwijkende patronen sneller te identificeren dan via loganalyse.

2.6.6. Correlatie tussen Monitoringdata en Security-events

Een geïsoleerde benadering van monitoring en beveiliging is onvoldoende voor complexe AI-systemen. Onderzoekers benadrukken dat effectieve incidentdetectie afhankelijk is van eventcorrelatie (Chandola e.a., 2009).

Door security-events (bijvoorbeeld detectie van prompt injection) te correleren met observabilitydata (zoals sessiegedrag, frequentie en timing) kan een vollediger dreigingsbeeld worden opgebouwd.

2.7. Correlatie tussen Security-events en Observabilitydata

De beveiliging van AI-systemen vereist een geïntegreerde benadering waarbij operationele monitoring en security-analyse niet langer los van elkaar worden behandeld. In complexe IT-omgevingen bestaan beveiligingsincidenten zelden uit 1 afzonderlijk event, maar uit een reeks gerelateerde gebeurtenissen die gezamenlijk een aanvalspatroon vormen. Eventcorrelatie maakt het mogelijk om deze verspreide signalen te structureren en analyseren als één samenhangend incidentbeeld (Kotenko e.a., 2022).

Volgens Kotenko e.a. (2022) is het doel van eventcorrelatie het reduceren van losse en vaak talrijke alerts tot betekenisvolle beveiligingsincidenten door gebruik te maken van gestructureerde correlatiemethoden. Dit principe is bijzonder relevant binnen AI-omgevingen, waar interacties met Large Language Models (LLM's) bestaan uit opeenvolgende prompts, responses en systeeminteracties die samen een potentieel misbruikscenario kunnen vormen.

2.7.1. Principes van Event Correlatie

Eventcorrelatie omvat technieken waarmee meerdere log- of security-events worden gecombineerd op basis van vooraf gedefinieerde regels, statistische modellen of gedragsanalyse. Kotenko e.a. (2022) onderscheiden verschillende benaderingen, waaronder rule-based correlatie, scenario-gebaseerde correlatie en statistische methoden die afwijkingen in eventpatronen identificeren.

Belangrijke correlatieprincipes zijn:

- **Rule-based correlatie:** gebeurtenissen worden gecombineerd op basis van vooraf gedefinieerde regels of correlatievoorwaarden. Dit type correlatie wordt vaak toegepast in traditionele SIEM-systemen.
- **Scenario-based correlatie:** vooraf gemodelleerde aanvalsscenario's of multi-stage aanvalspatronen worden herkend door gebeurtenissen te matchen met bekende sequenties.
- **Statistische of similarity-based correlatie:** gebeurtenissen worden geclusterd of gegroepeerd op basis van gelijkenis of afwijking van normaal gedrag.
- **Knowledge-based correlatie:** gebruik van domeinkennis of threat intelligence om gebeurtenissen in een bredere context te plaatsen.
- **Graph-based en machine learning-gebaseerde correlatie:** geavanceerde technieken waarbij relaties tussen events worden gemodelleerd als grafstructuren of algoritmen automatisch patronen leren detecteren.

Deze benaderingen maken het mogelijk om afzonderlijke alerts te reduceren tot betekenisvolle incidenten en multi-stage aanvalspatronen te identificeren die anders verborgen blijven in grote hoeveelheden logdata.

2.7.2. Integratie van Threat Intelligence

Threat intelligence verrijkt interne monitoringdata met externe kennis over bekende aanvalstechnieken en dreigingsactoren. Volgens het MITRE ATT&CK-framework worden aanvallen vaak opgebouwd uit gestandaardiseerde tactieken en technieken (Strom e.a., 2020).

Door AI-gerelateerde dreigingen (prompt injection of data-exfiltratie) te mappen op bestaande aanvalstactieken, kan correlatie verfijnd worden. Threat intelligence maakt het mogelijk om:

- Bekende aanvalspatronen sneller te herkennen
- Indicators of Compromise (IoC's) te integreren in detectieregels
- Interne anomalieën te koppelen aan externe dreigingsinformatie

Voor AI-omgevingen betekent dit dat observabilitydata niet enkel intern geanalyseerd wordt, maar ook in een bredere dreigingscontext geplaatst kan worden.

2.7.3. SIEM-benaderingen in AI-omgevingen

Security Information and Event Management (SIEM)-systemen centraliseren logging en passen correlatieregels toe om incidenten te detecteren. Moderne SIEM-architecturen combineren loganalyse, rule-based detectie en gedragsanalyse (Chandola e.a., 2009).

Binnen AI-omgevingen kan een SIEM-achtige aanpak worden toegepast op:

- Conversatielogs
- Authenticatie-events
- API-aanroepen naar LLM-diensten
- Detecties van AI-firewallcomponenten

Door deze gegevensstromen te centraliseren ontstaat een centraal analysepunt waarin verdachte interactiepatronen kunnen worden geïdentificeerd.

2.7.4. Detectie van Afwijkende Interactiepatronen

Anomaliedetectie speelt een belangrijke rol bij het identificeren van afwijkend gebruikersgedrag. Chandola e.a. (2009) definiëren een anomalie als een patroon dat significant afwijkt van verwacht gedrag.

Toegepast op AI-systemen kunnen afwijkende interactiepatronen onder meer bestaan uit:

- Ongebruikelijk hoge frequentie van prompts
- Systematische variaties van eenzelfde jailbreak-instructie
- Combinaties van prompts die wijzen op informatie-extractie
- Onverwachte escalatie van privileges via modelinstructies

Door deze gedragsafwijkingen te correleren met semantische detecties (bijvoorbeeld toxiciteit of policy violations) kan een meerlagige beveiligingsarchitectuur worden gerealiseerd.

2.7.5. Incidentrespons binnen AI-systemen

Detectie alleen is onvoldoende zonder een gestructureerde responsstrategie. Incidentrespons binnen AI-systemen vereist zowel technische als organisatorische maatregelen, zoals beschreven in algemene incidentresponsframeworks (National Institute of Standards and Technology, 2025).

Voor AI-toepassingen kan respons onder meer bestaan uit:

- Automatische blokkering van verdachte sessies
- Verhoogde logging bij risicogedrag
- Waarschuwingen naar beheerders
- Tijdelijke beperking van modelcapaciteiten

De combinatie van eventcorrelatie, threat intelligence en observabilitydata maakt het mogelijk om incidentrespons proportioneel en contextbewust uit te voeren. Dit vormt een essentieel uitgangspunt voor het ontwerp van een AI-gestuurde beveiligingsarchitectuur.

2.8. Privacy- en Ethiekvraagstukken bij AI Monitoring

De integratie van monitoring- en beveiligingsmechanismen binnen AI-systemen brengt belangrijke privacy- en ethische uitdagingen met zich mee. Aangezien chatbotinteracties vaak persoonsgegevens en gevoelige informatie bevatten, moet het verzamelen, opslaan en analyseren van deze gegevens voldoen aan strikte regelgeving. In Europa vormt de General Data Protection Regulation (GDPR) het centrale kader voor gegevensbescherming en privacy (European Parliament and Council, [2016](#)).

Het dilemma tussen uitgebreide monitoring enerzijds en privacybescherming anderzijds maakt dit bijzonder complex. Langs de ene kant is gedetailleerde logging noodzakelijk voor traceerbaarheid en beveiliging, anderzijds moet dataverwerking beperkt blijven tot wat strikt noodzakelijk is.

2.8.1. GDPR en Dataminimalisatie

Een kernprincipe binnen de GDPR is dataminimalisatie, waarbij enkel persoonsgegevens mogen worden verwerkt die noodzakelijk zijn voor het beoogde doel (European Parliament and Council, [2016](#)).

Voor AI-monitoring betekent dit dat niet alle interactiedata onbeperkt mag worden opgeslagen. In de context van chatbotlogging moet daarom kritisch worden afgewogen:

- Welke gegevens noodzakelijk zijn voor security en auditing
- Welke gegevens geanonimiseerd of gepseudonimiseerd kunnen worden
- Welke data volledig vermeden kan worden

Het toepassen van dataminimalisatie heeft directe impact op de ontwerpkeuzes van logging- en observabilitysystemen.

2.8.2. Privacy-by-Design in AI-systemen

Naast dataminimalisatie introduceert de GDPR het principe van privacy-by-design, waarbij gegevensbescherming vanaf het ontwerp van een systeem wordt geïntegreerd (European Data Protection Board, [2020a](#)).

Voor AI-systemen houdt dit in dat privacy niet achteraf wordt toegevoegd, maar een fundamenteel onderdeel vormt van de architectuur. Concreet kan dit betekenen:

- Automatische anonimisering van logs
- Beperking van toegangsrechten tot gevoelige data
- Encryptie van opgeslagen en verzonden gegevens

Deze maatregelen zorgen ervoor dat monitoring en beveiliging kunnen worden uitgevoerd zonder onnodige blootstelling van persoonsgegevens.

2.8.3. Bewaartermijnen en Logretentie

De GDPR vereist dat persoonsgegevens niet langer worden bewaard dan noodzakelijk voor het doel waarvoor ze zijn verzameld (European Parliament and Council, [2016](#)). Dit heeft belangrijke implicaties voor logging en audit trails.

Binnen AI-monitoring moet een balans worden gevonden tussen:

- Langdurige opslag voor audit en forensisch onderzoek
- Beperking van opslagduur om privacyrisico's te minimaliseren

Het definiëren van een duidelijk Gegevensbehoudbeleid is daarom essentieel binnen een compliant systeemarchitectuur.

2.8.4. Transparantie en Gebruikersrechten

Een ander fundamenteel principe binnen de GDPR is transparantie. Gebruikers moeten geïnformeerd worden over hoe hun gegevens worden verwerkt en hebben recht op inzage, correctie en verwijdering van hun data (European Parliament and Council, [2016](#)).

Voor AI-systemen betekent dit dat:

- Gebruikers geïnformeerd moeten worden over logging en monitoring
- Mechanismen voorzien moeten worden om data op te vragen of te verwijderen
- Beslissingen van AI-systemen, waar mogelijk, verklaarbaar moeten zijn

2.8.5. Ethiek en Bias in AI-monitoring

Naast juridische vereisten spelen ook ethische overwegingen een belangrijke rol. AI-systemen kunnen bias vertonen, bijvoorbeeld in de manier waarop risicovolle interacties worden gedetecteerd of geclassificeerd.

Volgens NIST moeten AI-systemen betrouwbaar, transparant en eerlijk zijn, waarbij bias en discriminatie actief worden geminimaliseerd (National Institute of Standards and Technology, [2023](#)).

Binnen AI-monitoring betekent dit dat:

- Detectiemechanismen geen systematische vooroordelen mogen bevatten
- Risicoclassificaties controleerbaar moeten zijn
- Monitoring niet mag leiden tot ongerechtvaardigde profilering van gebruikers

Het combineren van technische beveiliging met ethische verantwoordelijkheid is van essentieel belang voor de ontwikkeling van AI-gestuurde systemen.

2.9. Bestaande Oplossingen en Technologische Positionering

De snelle opkomst van AI-toepassingen heeft geleid tot de ontwikkeling van diverse observability- en securityoplossingen die zich richten op het monitoren en beveiligen van complexe systemen. Hoewel deze oplossingen belangrijke functionaliteiten bieden, blijkt dat zij vaak slechts een deel van het probleem aankaarten.

2.9.1. AI Observability Platforms

AI observability platforms richten zich op het monitoren van modelgedrag, prestatie en interacties. Observability is gebaseerd op het verzamelen en analyseren van logs, metrics en traces om inzicht te krijgen in systeemgedrag, waarbij distributed tracing een centrale rol speelt in het begrijpen van complexe interacties binnen systemen (Sigelman & et al., [2010](#)).

Een voorbeeld van een dergelijk platform is Dynatrace, dat end-to-end observability biedt binnen gedistribueerde systemen. Het platform maakt het mogelijk om gebruikersinteracties, systeemcalls en prestatiegegevens te traceren over verschillende componenten heen.

Voor AI-systemen betekent dit dat onder meer volgende aspecten gemonitord kunnen worden:

- Interactiestromen tussen gebruiker en applicatie

- API-aanroepen naar AI-modellen
- Performantie en responstijden van modelinference

Hoewel dergelijke platforms sterk zijn in technische monitoring en analyse, ligt de focus voornamelijk op performantie en systeemgedrag. Detectie van kwaadaardige of risicovolle input binnen AI-interacties wordt niet expliciet ondersteund.

2.9.2. AI Security en Firewalloplossingen

Naast observability zijn er gespecialiseerde beveiligingsoplossingen ontwikkeld die zich richten op AI-systemen. Deze oplossingen bouwen voort op traditionele web- en API-beveiliging, maar voegen detectiemechanismen toe die specifiek gericht zijn op AI-gerelateerde dreigingen.

Een voorbeeld hiervan is Akamai, dat beveiligingsoplossingen aanbiedt voor webapplicaties en API's, en uitbreidingen voorziet voor het beschermen van AI-gedreven toepassingen.

Typische functionaliteiten zijn:

- Detectie van prompt injection en manipulatiepogingen
- Filtering van schadelijke of ongewenste input
- Bescherming tegen misbruik van AI-modellen en data-exfiltratie

Hoewel dergelijke systemen effectief zijn in het detecteren en blokkeren van verdachte interacties, functioneren zij vaak als een afzonderlijke beveiligingslaag. Hierdoor ontbreekt een directe koppeling met observabilitydata, wat de analyse en reconstructie van incidenten moeilijk maakt.

2.9.3. Enterprise Monitoring Frameworks

Binnen enterprise-omgevingen worden monitoring- en loggingoplossingen vaak gecentraliseerd via platformen die logs, metrics en traces combineren. Volgens National Institute of Standards and Technology (2006) is gecentraliseerd logbeheer essentieel voor het analyseren en reconstrueren van beveiligingsincidenten.

Deze frameworks bieden:

- Centralisatie van logdata uit verschillende bronnen
- Real-time monitoring en alerting
- Ondersteuning voor eventcorrelatie

Hoewel deze systemen sterk zijn in algemene monitoring en analyse, zijn ze niet specifiek ontworpen voor de semantische complexiteit van AI-interacties. Hierdoor ontbreekt vaak ondersteuning voor AI-specifieke dreigingen en interpretatie van conversatie-inhoud.

2.9.4. Vergelijkende Analyse van Bestaande Benaderingen

Uit de voorgaande analyse blijkt dat bestaande oplossingen elk een deelaspect van het probleem aanpakken.

Observabilityplatforms, zoals Dynatrace, bieden diepgaand inzicht in systeemgedrag en interactiestromen, maar missen specifieke beveiligingsmechanismen voor AI-gerelateerde dreigingen. AI-securityoplossingen, zoals Akamai, zijn dan weer gericht op het detecteren en blokkeren van kwaadaardige input, maar bieden beperkte ondersteuning voor traceerbaarheid en diepgaande analyse van volledige interacties.

Enterprise monitoring frameworks centraliseren data en ondersteunen correlatie, maar zijn doorgaans niet afgestemd op de semantische en dynamische aard van AI-interacties.

Deze fragmentatie leidt tot een situatie waarin organisaties meerdere systemen moeten combineren zonder een uniforme integratie. Hierdoor ontstaat een gebrek aan end-to-end zichtbaarheid en coherente incidentanalyse.

2.9.5. Positionering van het Voorgestelde Framework

Het voorgestelde framework in deze bachelorproef combineert observability en security. In tegenstelling tot bestaande oplossingen wordt gekozen voor een geïntegreerde aanpak, waarbij inzichten uit observability en security bewust met elkaar kunnen worden verbonden.

Door observabilityfunctionaliteiten, zoals die van Dynatrace, te combineren met beveiligingsmechanismen, zoals aangeboden door Akamai, wordt een end-to-end zicht gecreëerd op chatbotinteracties.

Dit maakt het mogelijk om:

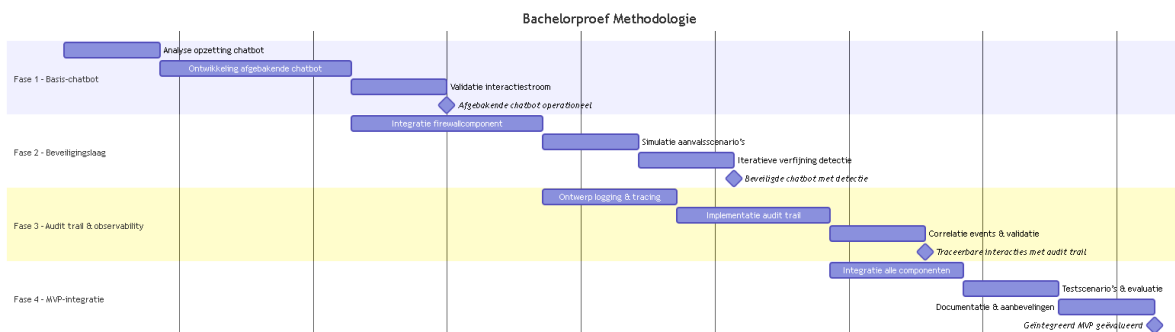
- interacties volledig te traceren (wie, wat, wanneer)
- verdachte gebeurtenissen te detecteren
- incidenten reproduceerbaar te analyseren en te mitigeren

Deze geïntegreerde benadering vormt een duidelijke uitbreiding op bestaande oplossingen en speelt in op de nood aan systemen die zowel inzicht, beveiliging als

controleerbaarheid bieden binnen AI-gedreven toepassingen. Hiermee sluit dit framework rechtstreeks aan bij de centrale onderzoeksvraag van deze bachelorproef.

3

Methodologie



Figuur 3.1: Gantt-diagram

Dit onderzoek wordt uitgevoerd als toegepast onderzoek binnen de bedrijfscontext van Evolane en focust op één afgebakende probleemsituatie, namelijk het gebrek aan traceerbaarheid en beveiliging van AI-gestuurde chatbotinteracties binnen klantondersteuningsprocessen. Het onderzoek resulteert in een minimum viable product (MVP), zijnde een eerste functionele implementatie waarin de kernfunctionaliteiten voor traceerbaarheid en beveiliging aantoonbaar aanwezig zijn. De scope van het onderzoek beperkt zich tot het technisch ontwerpen, implementeren en evalueren van dit prototype binnen een gecontroleerde testomgeving.

Het proof-of-concept wordt opgebouwd in vier opeenvolgende, inhoudelijk samenhangende fasen. Elke fase levert een concrete mijlpaal op die bijdraagt aan het beantwoorden van de centrale onderzoeksvraag.

Fase 1: Ontwikkeling van een afgebakende chatbot

In de eerste fase wordt een eenvoudige chatbot ontwikkeld die één concreet klantondersteuningsscenario implementeert. De functionaliteit blijft bewust beperkt

tot een duidelijk afgelijnde use case, zodat de interactiestroom tussen gebruiker en digitale assistent reproduceerbaar blijft. De chatbot dient uitsluitend als testobject voor verdere beveiligings- en traceerbaarheidsmechanismen. Het eindresultaat van deze fase is een functionerende chatbot met een stabiele interactiestroom.

Fase 2: AI firewall voor chatbotverkeer

In de tweede fase wordt het chatbotverkeer voorzien van een beveiligingslaag die inkomende en uitgaande interacties analyseert. De focus ligt op het detecteren en blokkeren van afwijkende of risicovolle invoer binnen de afgebakende testomgeving. Door het uitvoeren van gesimuleerde aanvalsscenario's wordt nagegaan hoe effectief het systeem reageert op misbruik van de chatbotinterface. Deze fase resulteert in een chatbot waarbij beveiligingsdetectie en -mitigatie functioneren.

Fase 3: Implementatie van traceerbaarheid en audit trail

De derde fase richt zich op het vastleggen en overeenkomen van gebeurtenissen binnen de chatbotinteracties. Hierbij worden gebruikersacties, systeemreacties en beveiligingsgebeurtenissen gelogd om volledige reconstructie van gesprekken mogelijk te maken. De audit trail wordt opgezet met als doel technische reproduceerbaarheid van incidenten, zonder dat juridische bewijsvoering of langdurige datastorage wordt nagestreefd. Het eindresultaat van deze fase is een werkende audit trail die inzicht biedt in het volledige verloop van interacties.

Fase 4: Integratie en evaluatie van het MVP

In de vierde fase worden alle ontwikkelde componenten samengebracht tot één geïntegreerd MVP. Dit prototype wordt geëvalueerd aan de hand van vooraf gedefinieerde testscenario's die zowel normale interacties als afwijkend gedrag omvatten. De evaluatie focust op de mate waarin het systeem traceerbaarheid en beveiliging combineert binnen de afgebakende casus. Deze fase resulteert in een geïntegreerd MVP en een technische evaluatie die de basis vormt voor aanbevelingen.

4

Risicoanalyse

4.1. Inleiding

Deze risicoanalyse vormt de basis voor de technische validatie in deze bachelorproef. Alvorens een beveiligingsoplossing kan worden geïmplementeerd, moet de huidige staat van de digitale assistent (Rasa) en de achterliggende API-infrastructuur worden geëvalueerd in een onbeveiligde omgeving. Dit proces staat bekend als het vaststellen van een baseline. Het doel is om aan te tonen welke kwetsbaarheden aanwezig zijn wanneer de applicatie direct aan het internet wordt blootgesteld zonder de bescherming van een AI Firewall en geavanceerde monitoring.

De resultaten uit deze analyse verantwoorden de specifieke configuratiekeuzes in het volgende hoofdstuk (Proof of Concept). De analyse richt zich op drie pijlers:

- Kwantificering van risico's: Gebaseerd op internationaal erkende methodologieën voor dreigingsanalyse.
- Baseline-vaststelling: Een gedetailleerde beschrijving van aanvalsscenario's in een onbeveiligde versus een gemitigeerde situatie.
- Compliance-mapping: Het koppelen van technische dreigingen aan wettelijke kaders zoals de AVG en industriestandaarden zoals PCI-DSS.

4.2. Risicobeoordeling

Voor het kwantificeren van de risico's wordt gebruikgemaakt van de principes uit de National Institute of Standards and Technology (2012) en de International Organization for Standardization (2022). Risico wordt hierbij benaderd als het product van de waarschijnlijkheid van een dreiging en de impact op de organisatie indien deze dreiging zich voordoet.

$$\text{Risico} = \text{Waarschijnlijkheid} \times \text{Impact}$$

De resultaten worden gecategoriseerd op een schaal van 1 tot 25, waarbij scores boven de 14 als 'Hoog' worden beschouwd en onmiddellijke mitigatie vereisen. Tabel 4.1 geeft een overzicht van de geïdentificeerde dreigingen binnen de chatbot-omgeving.

Tabel 4.1: Risico-analyse van aanvalsvectoren voor de onbeveiligde chatbot-omgeving

Aanvalsvector / Kwetsbaarheid	Waarschijnlijkheid	Impact	Totaal Risico	Categorie
Applicatielaag Injecties				
Cross-Site Scripting (XSS) via chat-payload	4	4	16	Hoog
SQL/ORM Wildcard Injectie (%)	3	5	15	Hoog
Autorisatie & Data-exfiltratie				
IDOR (Opvragen bestelstatus derden)	4	5	20	Zeer hoog
Onbewust delen van PII door eindgebruiker	5	3	15	Hoog
Beschikbaarheid & Integriteit				
Application DoS (Geautomatiseerde bot-spam)	5	4	20	Zeer hoog
Integer Overflow (Crashen backend-services)	2	4	8	Gemiddeld
Toxisch gedrag / Profanity in logs	5	2	10	Gemiddeld

4.2.1. Bepaling van de Waarschijnlijkheid en Impact

4.2.2. Bepaling van de Waarschijnlijkheid en Impact

Om de risicoscores objectief te onderbouwen, gebruiken we de richtlijnen uit National Institute of Standards and Technology (2012). De score voor *Waarschijnlijkheid* (1 tot 5) hangt af van hoe complex de aanval is, of er al exploits beschikbaar zijn en hoe kwetsbaar de chatbot is omdat deze direct aan het internet hangt. De verdeling ziet er als volgt uit:

- **1 (Zeer laag):** De aanval is puur theoretisch of zo ingewikkeld dat er een gecoördineerd team met geavanceerde middelen voor nodig is.
- **2 (Laag):** Misbruik is mogelijk, maar een aanvaller moet specifieke expertise hebben, de backend-architectuur goed begrijpen en een duidelijke motivatie

hebben.

- **3 (Gemiddeld):** De kwetsbaarheid is bekend en er zijn standaardtactieken voor beschikbaar. Een hacker met gemiddelde ervaring kan de aanval handmatig uitvoeren.
- **4 (Hoog):** De kwetsbaarheid is eenvoudig te vinden en te misbruiken. Omdat de chatbot direct aan het internet is blootgesteld, is de kans op een opportunistische aanval groot.
- **5 (Zeer hoog):** De dreiging hoort simpelweg bij het online karakter van de applicatie, of wordt continu aangestuurd door automatische scripts en internetscanners.

De score voor *Impact* (1 tot 5) weerspiegelt de potentiële schade voor de organisatie. Hierbij kijken we naar de vertrouwelijkheid van gegevens (zoals GDPR-compliance), data-integriteit en de beschikbaarheid van de chatbot. Een score van 5 staat voor een kritiek datalek of een volledige systeemuitval.

4.3. Analyse van de Baseline versus Mitigatie

Om de effectiviteit van het voorgestelde framework te bewijzen, worden de drie meest kritieke risico's hieronder nader toegelicht in een "voor-en-na"scenario.

4.3.1. Scenario 1: Manipulatie van de Applicatielaag (OWASP A03:2021)

Baseline: De Rasa-backend accepteert ongeschoonde JSON-payloads. Een aanval-ler kan JavaScript-code injecteren (bijv. `<script>alert(1)</script>`) die door de applicatie wordt opgeslagen in de database. Wanneer een beheerder deze interactie bekijkt, wordt de code uitgevoerd in zijn browser (Stored XSS).

Mitigatie: De Akamai Firewall for AI voert semantische inspectie uit op het `$.message` veld. Malafide scripts worden herkend en geblokkeerd aan de edge, nog voordat de API wordt bereikt.

4.3.2. Scenario 2: Uitputting van Resources (Bot-verkeer)

Baseline: Zonder actieve beperkingen kan een aanvaller via een eenvoudig Bash-script duizenden registratieverzoeken per minuut versturen. Dit leidt tot een overbelasting van de Java-microservices en het vollopen van de MongoDB-database met garbage data, waardoor de bot onbereikbaar wordt voor legitieme gebruikers.

Mitigatie: Door de implementatie van Rate Limiting en Bot Management herkent de WAF afwijkende verkeerspatronen en weigert het verzoeken na een vooraf ingestelde drempelwaarde. Aangezien deze oplossing WAF gebaseerd is en niet door

de Firewall for AI kan worden opgelost, valt dit buiten de scope van dit onderzoek. Het uitputten van resources door extreem lange prompts, dat wel tegen gehouden kan worden door de firewall for AI regel Input too long, valt hier wel binnen.

4.3.3. Scenario 3: Ongeoorloofde toegang tot Objecten (IDOR)

Baseline: Indien de backend-logica uitsluitend vertrouwt op de door de gebruiker meegegeven ID's in de chat, kan een kwaadwillende gebruiker de status van andermans bestellingen opvragen. Dit resulteert in het lekken van namen, adressen en betaalgegevens in het chatvenster.

Mitigatie: Deze kwetsbaarheid behoort tot de categorie Broken Access Control en vereist een structurele oplossing in de backend, zoals het afdwingen van correcte autorisatiecontroles op basis van de sessiecontext. Aangezien dit een applicatielogica-probleem betreft, valt het buiten de scope van dit onderzoek.

4.4. Wetgeving, Richtlijnen en Standaarden

4.4.1. Algemene Verordening Gegevensbescherming (AVG / GDPR)

Volgens European Parliament and Council (2016) artikel 32 moeten organisaties passende technische maatregelen nemen om de beveiliging van persoonsgegevens te waarborgen. Het ongefilterd loggen van conversaties vormt een risico op log injection en blootstelling van gevoelige data. Dynatrace faciliteert Privacy by Design door middel van Data Masking regels, waarbij gevoelige data (zoals rijksregisternummers) automatisch wordt geanonimiseerd tijdens de opname.

4.4.2. NIS2-richtlijn

De European Union (2022) verplicht aanbieders van essentiële diensten tot het implementeren van incidentenbehandeling en continue monitoring. Het gebruik van distributed tracing in Dynatrace stelt de organisatie in staat om de volledige keten van een verzoek te auditeren, wat essentieel is voor de rapportageverplichtingen onder NIS2.

4.4.3. Payment Card Industry Data Security Standard (PCI-DSS)

Voor het verwerken van betalingsinformatie is compliance met PCI Security Standards Council (2022) v4.0 vereist. De risicoanalyse toont aan dat creditcardgegevens kunnen lekken via de chatbot-interface. Akamai's Output Filtering draagt bij aan Vereiste 1.2 (beschermen van dataverkeer) en Dynatrace's masking-regels voldoen aan Vereiste 3 (beschermen van opgeslagen kaartgegevens).

4.4.4. OWASP Top 10

Hoewel geen wetgeving, is de OWASP Foundation ([2021](#)) de industriestandaard voor applicatiebeveiliging. De geconfigureerde testscenario's mappen direct op kritieke kwetsbaarheden zoals A03:2021-Injection (XSS).

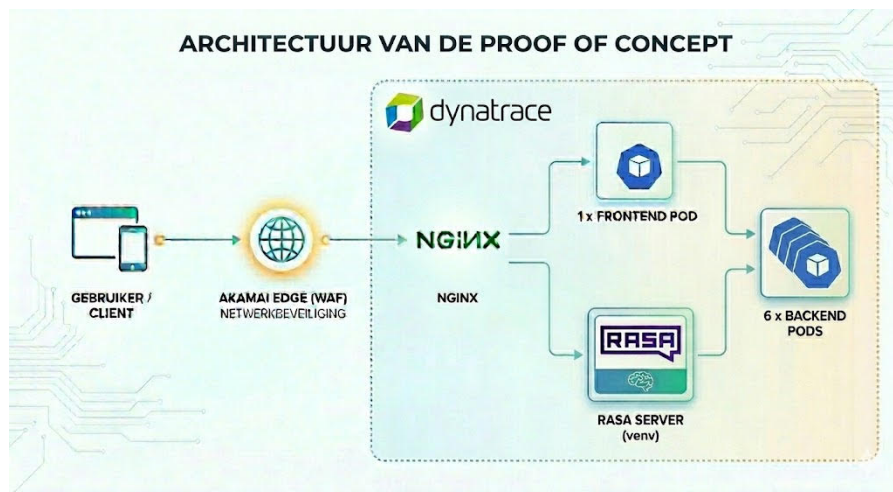
5

Proof of Concept

5.1. Inleiding

Dit hoofdstuk beschrijft de technische opzet en uitvoering van de Proof of Concept (PoC) ter validatie van het voorgestelde geïntegreerde framework. In het vorige hoofdstuk werd een risicoanalyse uitgevoerd die de kwetsbaarheden van AI-gedreven chatbots in kaart bracht, evenals de noodzaak om interacties traceerbaar te maken conform GDPR en enterprise-standaarden. Dit hoofdstuk vertaalt deze theoretische risicobeoordeling naar een concrete, technische implementatie.

De testomgeving bestaat uit een digitaal assistent-platform (Rasa) dat geïntegreerd is met een backend-architectuur. Eerst wordt de configuratie van de Akamai AI Firewall besproken, waarbij een onderscheid wordt gemaakt tussen vooraf gedefinieerde beveiligingsregels en applicatiespecifieke restricties. Vervolgens worden de toepassingsgebieden en de architecturale opzet van de chatbot toegelicht. Daarna wordt de geautomatiseerde testopstelling beschreven, waarmee kwaadaardige datastromen op een gecontroleerde manier worden gesimuleerd. Tot slot wordt de traceerbaarheid en observability binnen Dynatrace behandeld, inclusief het maskeren van gevoelige data en het correleren van incidenten.



Figuur 5.1: Opstelling PoC

5.2. AI Firewall Regels en Configuratie

Net zoals in traditionele beveiligingsoplossingen vormt de configuratie van de inspectieregels de kern van de applicatiebeveiliging. Binnen deze PoC dient de Akamai AI Firewall als de preventieve schil rond de chatbot. Omdat de communicatie tussen de frontend en de digitale assistent verloopt via een REST API met JSON-payloads, moeten de beveiligingsregels hierop worden afgestemd.

5.2.1. Architecturale Context: Intent-gebaseerde NLU versus Generatieve AI

Hoewel Akamai semantische inspectiemodules in de markt zet als specifieke bescherming voor Large Language Models (LLM's), is het belangrijk om de architecturale context van deze PoC te benadrukken. De gebruikte digitale assistent (Rasa) is een intent-gebaseerd Natural Language Understanding (NLU)-systeem en geen generatief taalmodel. Hierdoor is de applicatie van nature immuun voor LLM-specifieke kwetsbaarheden zoals Prompt Injection, Jailbreaking en Language Policy Evasion. Desondanks bewijst deze PoC dat de inzet van deze semantische AI-filters op de inkomende en uitgaande JSON-payloads enorme waarde biedt.

5.2.2. Input Filtering (Bescherming van de backend)

De eerste verdedigingslinie richt zich op het beveiligen van de inkomende gebruikersinvoer (input). Hierbij wordt de prompt van de gebruiker semantisch en syntactisch geanalyseerd voordat deze de applicatielaag (Rasa/Helidon) bereikt. In 5.1 worden de specifieke firewall-detectiemogelijkheden gekoppeld aan de geteste use-cases en dreigingen.

Tabel 5.1: Mapping van Input Firewall-regels naar Use-Cases

Akamai AI Capability	Use-Case	Toelichting
Code detected on input	XSS & Injecties	Detecteert scripts zoals <code><script>alert(1)</script></code>
Input too long	DoS & Overflow	Beperkt payload lengte
PII input	GDPR & PCI-DSS	Detecteert gevoelige input
Toxic input	Abuse filtering	Blokkeert ongepast gedrag
Non-allowed language	WAF evasion	Blokkeert vreemde talen

5.2.3. Output Filtering (Bescherming van de eindgebruiker en bedrijfs-data)

Hoewel het systeem geen probabilistische teksten genereert, volstaat het niet om enkel de invoer te controleren. De backend-database kan reeds blootgesteld zijn, of de bedrijfslogica kan onverwacht reageren op een query (bijvoorbeeld door te veel data op te halen). Daarom implementeert de PoC tevens output filtering, waarbij het antwoord van de chatbot wordt geïnspecteerd voordat het over het netwerk naar de eindgebruiker wordt teruggestuurd.

Tabel 5.2: Mapping van Output Firewall-regels

Akamai AI Capability	Use-case	Toelichting
PII output	IDOR	Detecteert gevoelige output
Output too long	Data dumping	Voorkomt massale responses
Code detected on output	XSS	Blokkeert scripts
Toxic output	Echo prevention	Blokkeert toxische output

5.2.4. Praktische Configuratie in het Akamai Control Center

De configuratie van de Firewall for AI binnen het Akamai Control Center is opgedeeld in vier cruciale stappen, die samen een gelaagde beveiligingsstrategie vormen:

1. Globale Security-configuratie (Host & Policy-binding)

Hierbij is de specifieke applicatie-hostnaam `sockshop.evolane.eu` geregistreerd en ondergebracht in de centrale security config. Binnen deze config is de hostnaam expliciet gekoppeld aan de AI-beveiligingsfunctionaliteit, waarbij een speciaal beheerprofiel genaamd `Akamai Edge Bricke` is gedefinieerd (zie Figuur 5.2). De *Sample Rate* staat hierbij vastgelegd op 100%, wat garandeert dat elk inkomend

en uitgaand HTTP-verzoek volledig door de inspectie moet.

2. De Firewall for AI Policy-configuratie

Het tweede niveau betreft de fijnmazige inrichting van de AI-beveiligingslaag zelf. In plaats van breedbandig en ongericht al het webverkeer te analyseren, biedt het Akamai-platform de mogelijkheid tot gerichte *Application Access*-scoping. De policy is zo geconfigureerd dat de semantische NLP-modellen uitsluitend actief worden op het specifieke *Endpoint Path* van het Custom REST Channel: `/webhooks/-secure_rest/webhook` (zie Figuur 5.2).

Firewall Settings

Firewall for AI Application Configuration Akamai Edge Brick [View Configuration](#)

Sample Rate 100 %

Application Access

Endpoint Hostname sockshop.evolane.eu [Add a Hostname](#)

Endpoint Path /webhooks/secure_rest/webhook

Input / Output Settings

Check input ☒

JSON input path \$.message

Max input length(char) 2048

Input encoding None

Check output ☒

JSON output path \$.text

Max output length(char) 2048

Output encoding None

Figuur 5.2: Endpoint-scoping en JSONPath-configuratie binnen het Akamai Control Center.

Zoals gevisualiseerd in de *Input / Output Settings* van Figuur 5.2, is voor zowel inkomend als uitgaand verkeer de actieve inspectie geactiveerd. Om de verwerkings-efficiëntie te maximaliseren en *false positives* op syntactische JSON-tekenen (zoals accolades of arrays) uit te sluiten, wordt de semantische controle via JSONPath-isolatie uitsluitend toegepast op de werkelijke tekstinhoud:

- **Inkomend verkeer (Check input):** De parser isoleert het veld `$.message` uit de inkomende POST-payload, met een strikte grens van maximaal 2048 karakters.
- **Uitgaand verkeer (Check output):** De parser isoleert het veld `$.text` uit de respons van de chatbot voordat deze de edge verlaat, eveneens maximaal op 2048 karakters.

Deze grens van 2048 karakters is bewust zo ingesteld om ervoor te zorgen dat het volledige bericht wordt geanalyseerd. De werkelijke maximumlengte van de chatbotberichten ligt in de praktijk op 256 voor input en 512 voor output. Moest een gebruiker een bericht langer dan 256 karakters invoeren, zouden de eerste 2048 karakters worden geanalyseerd en zichtbaar zijn in akamai web security analytics,

terwijl de rest van het bericht (indien aanwezig) zou worden genegeerd. Deze configuratie garandeert dat er geen relevante data buiten de inspectie valt.

3. Tuning van Regels en Sensitivity Profile

Binnen de policy is een set van tien specifieke AI-regels actief (zie Figuur 5.3). Om de chatbot-omgeving optimaal te beschermen zonder legitiem gebruikersverkeer te hinderen, is de *Sensitivity* (gevoeligheid) per regel handmatig afgestemd. Aangezien de gebruikte chatbot een NLU is en geen generatief model, is de regel LLM Prompt Injection uitgeschakeld. De actie alert zorgt ervoor dat alle pogingen tot prompt injection direct worden gelogd en gemarkeerd in de Akamai Security Analytics, maar niet automatisch worden geblokkeerd. Dit is de makkelijkste manier om FP's te vermijden, aangezien de NLU niet vatbaar is voor prompt injection.

Rules			
Name	Sensitivity	Action	Response on modify action
▶ LLM Prompt Injection	MEDIUM	Alert	
▶ PII detected on LLM input		Modify 3 overrides	Rule specific
▶ PII detected on LLM output		Modify 3 overrides	Rule specific
▶ LLM input too long		Modify	Rule specific
▶ LLM output too long		Modify	Rule specific
▶ Toxic content on LLM input	LOW	Modify	Rule specific
▶ Toxic content on LLM output	MEDIUM	Modify	Rule specific
▶ LLM input includes a non-allowed language		Modify	Rule specific
▶ Code detected on LLM input		Modify	Rule specific
▶ Code detected on LLM output		Modify	Rule specific

Figuur 5.3: Overzicht van de actieve AI-beveiligingsregels en hun specifieke sensitivity-profielen.

4. Geavanceerd Blokkeergedrag via de Modify-actie

Een voordeel van Akamai's Firewall for AI is de mogelijkheid om af te stappen van een traditionele netwerkblokkade (zoals een abrupte TCP-drop). Zoals te zien is in Figuur 5.3, zijn de regels geconfigureerd met de actie *Modify*.

Wanneer er op deze actie wordt doorgeklikt (zie Figuur 5.4), wordt de *Response on modify action* gedefinieerd als *Rule specific*. Dit stelt de beheerder in staat om een op maat gemaakte, applicatie-specifieke JSON-payload te structureren die als *Payload body* wordt teruggegeven bij een HTTP 403 Forbidden-foutcode. Beide parameters zijn volledig aanpasbaar, waardoor de firewall niet alleen een beveiligingslaag vormt, maar ook een communicatiemiddel om gebruikers op een vriendelijke manier te informeren over de reden van de blokkade.

The screenshot shows the configuration for a rule named "LLM input includes a non-allowed language". The "General" tab is active, showing the rule's name, description, category, and version. The "Action and behavior" tab is also visible, showing the "Modify" action selected. The "Response on Modify" is set to "Rule specific". The "Response status code" is 403, and the "Content type" is application/json. The "Payload body" is a JSON object with the following structure:

```
{
  "ai_firewall": "block",
  "reason": "unsupported_language",
  "message": "Language not supported, please make sure your message is in English."
}
```

Figuur 5.4: Detailweergave van de Modify-actie met een aangepaste JSON-respons voor niet-toegestane talen.

Wanneer een gebruiker bijvoorbeeld een prompt in een vreemde taal verstuurt en daarmee de regel `LLM input includes a non-allowed language` triggert, verbreekt Akamai niet simpelweg de verbinding, maar geeft het de edge-respons uit Figuur 5.4:

Listing 5.1: Gecustomiseerde JSON-payload gegenereerd door de Akamai edge bij een taalovertredding.

```
1 {
2   "ai_firewall": "block",
3   "reason": "unsupported_language",
4   "message": "Language not supported, please make sure your message
5     is in English."
```

Deze specifieke instellingen garanderen dat de javascript-logica in de `index.html` van de chatbot-frontend de exacte reden van de blokkade direct kan parsen en op een gebruiksvriendelijke manier aan de eindgebruiker kan tonen.

5.3. Systeemarchitectuur en Integratie

De architectuur van deze Proof of Concept is ontworpen als een hybride omgeving. Dit houdt in dat de webinterface en de backend-logica draaien binnen geïsoleerde Docker-containers (pods), terwijl de chatbot (Rasa) direct op de server draait in een lokale Python-omgeving (venv). Om dit netwerkverkeer veilig en gestructureerd te routeren, volgt de datastroom een strikt pad. Een verzoek van de eindgebruiker passeert allereerst de Akamai Edge firewall ter inspectie. Daarna komt het gezuiverde verkeer aan op de host, waar Nginx dient als de centrale verkeersregelaar. Nginx verdeelt deze inkomende verzoeken vervolgens over de juiste Docker-pods

en de lokale virtuele omgeving.

5.3.1. Hybride Infrastructuur: Kubernetes-omgeving en Virtual Environments

Voor de uitvoering van de Proof of Concept is gekozen voor een architectuur die schaalbaarheid combineert met isolatie. De ondersteunende infrastructuur is ondergebracht in een Kubernetes-cluster, waarbij de verschillende componenten draaien als Pods. De kern van de digitale assistent draait daarentegen in een specifieke Python-omgeving op de host-machine. Deze hybride opzet bestaat uit de volgende technische componenten:

- **Frontend-pod:** Een gecontaineriseerde web-widget die de gebruikersinterface bevat en de communicatie met de API verzorgt via een Kubernetes Service.
- **Rasa NLU (Local venv):** De digitale assistent draait binnen een Python Virtual Environment (venv) op de host. Dit stelt de assistent in staat om direct gebruik te maken van de systeembronnen voor intent-extractie en dialoogbeheer zonder de netwerk-overhead van een extra containerlaag binnen het cluster.
- **Backend-pods (Java/Helidon):** Zes gecontaineriseerde microservices die binnen het Kubernetes-cluster de bedrijfslogica en productgegevens van de 'Socshop' beheren.

In Listing 5.2 is de definitie van de frontend-laag weergegeven middels een Kubernetes manifest. Hierin is te zien hoe de frontend als een Deployment en Service wordt gedefinieerd.

Listing 5.2: Kubernetes manifest voor de frontend-laag met custom web-widget

```
1 kind: Service
2 apiVersion: v1
3 metadata:
4   name: front-end
5   labels:
6     app: front-end
7 spec:
8   type: ClusterIP
9   ports:
10  - port: 80
11    targetPort: 8079
12  selector:
13    app: front-end
```

```
14 ---
15 kind: Deployment
16 apiVersion: apps/v1
17 metadata:
18   name: front-end
19 spec:
20   replicas: 1
21   selector:
22     matchLabels:
23       app: front-end
24   template:
25     metadata:
26       labels:
27         app: front-end
28     spec:
29       nodeSelector:
30         beta.kubernetes.io/os: linux
31       containers:
32       - name: front-end
33         image: my-custom-frontend:v1
34         imagePullPolicy: Never
35         env:
36         - name: DT_SUPPORT_DEPRECATED_NODE_VERSIONS
37           value: "true"
38         - name: CARTS_HOST
39           value: "carts"
40         - name: CARTS_PORT
41           value: "80"
42         resources:
43           requests:
44             cpu: 100m
45             memory: 200Mi
46           limits:
47             cpu: 1000m
48             memory: 200Mi
49         ports:
50         - containerPort: 8079
51       securityContext:
52         runAsNonRoot: true
53         runAsUser: 10001
54         capabilities:
```

```
55         drop:
56             - all
57         readOnlyRootFilesystem: true
```

5.3.2. Rasa NLU & Custom REST Channel Configuratie

De configuratie van de digitale assistent vormt een cruciaal onderdeel van de beveiligingsketen. In de standaardconfiguratie maakt Rasa vaak gebruik van Web-Sockets (via Socket.IO) voor communicatie. Tijdens de initiële integratietests werd echter vastgesteld dat de specifieke protocol-framing van Socket.IO (bijvoorbeeld de numerieke 42 prefix voor berichten) de JSON-structuur 'vervuilt'. Dit maakte het voor de Akamai Firewall for AI onmogelijk om de payload semantisch te parsen en de geconfigureerde JSONPath-inspectie op het `$.message` veld toe te passen.

Om deze beperking te omzeilen, is er de keuze gemaakt voor een Custom REST Channel. De frontend is zodanig geherprogrammeerd dat chatberichten als zuivere HTTP POST-requests worden verzonden. In de `credentials.yml` van Rasa is hiervoor het rest kanaal geactiveerd. Deze aanpassing garandeert dat de data die de Firewall for AI passeert een valide JSON-object is, zoals weergegeven in Listing 5.3. Hierdoor kan de firewall de inspectieregels voor bijvoorbeeld XSS, SQLi en PII-detectie met precisie uitvoeren op de werkelijke inhoud van het bericht.

Listing 5.3: Structuur van de REST-payload voor optimale WAF-inspectie

```
1      {
2          "sender": "user_123 ",
3          "message": "What can you do"?
4      }
```

Dit custom kanaal breidt de standaard REST-functionaliteit van Rasa uit en dient als een kanaal dat twee cruciale taken vervult:

1. **OpenTelemetry-integratie en Trace Stitching:** Voor elk inkomend POST-verzoek wordt handmatig een OpenTelemetry-span (`POST /webhook/secure_rest`) gestart. Door de gedistribueerde tracing-context via het `inject()`-mechanisme als metadata aan het `UserMessage`-object mee te geven, wordt gegarandeerd dat de trace niet breekt binnen de Rasa-pijplijn. Dit maakt het mogelijk om de interactie in Dynatrace te correleren van de edge tot aan de backend microservices.
2. **Context-Aware Data Masking (Privacy-by-Design):** Om te voldoen aan de strikte wetgeving rond dataminimalisatie (GDPR), implementeert de channel een controle. Voordat de gebruikersinvoer naar de Dynatrace-span wordt geëxporteerd, raadpleegt het script asynchroon de `Rasa Tracker API` om de huidige status van de conversatie te controleren.

Wanneer uit deze status blijkt dat de gebruiker zich in een actieve, gevoelige gesprekslus bevindt classificeert het kanaal de invoer als potentieel privacygevoelig. In de Dynatrace-telemetrie wordt de invoertekst dan preventief gemaskeerd en overschreven met de string `[REDACTED_DUE_TO_ACTIVE_FORM]`.

Op identieke wijze controleert het kanaal de uitgaande antwoorden (`collector.messages`) op gevoelige profieldata of adresbevestigingen, en maskeert deze in de uitgaande OpenTelemetry-attributen. In Listing 5.4 is de exacte Python-implementatie van deze veilige REST-architectuur weergegeven.

Listing 5.4: Python-implementatie van het Secure REST Channel met OpenTelemetry en context-aware masking

```
1 import inspect
2 import aiohttp
3 from sanic import Blueprint, response
4 from typing import Text, Dict, Any, Optional, Callable, Awaitable
5 from rasa.core.channels.channel import InputChannel,
   CollectingOutputChannel, UserMessage
6
7 # --- OpenTelemetry (OTel) Imports ---
8 from opentelemetry import trace
9 from opentelemetry.sdk.resources import Resource
10 from opentelemetry.sdk.trace import TracerProvider
11 from opentelemetry.sdk.trace.export import BatchSpanProcessor
12 from opentelemetry.exporter.otlp.proto.http.trace_exporter import
   OTLPSpanExporter
13 from opentelemetry.trace.status import Status, StatusCode
14 from opentelemetry.propagate import inject
15
16 # Setup OpenTelemetry Tracer gericht op de lokale OpenTelemetry
   Collector
17 resource = Resource.create({"service.name": "rasa-core"})
18 provider = TracerProvider(resource=resource)
19 exporter = OTLPSpanExporter(endpoint="http://localhost:4318/v1/
   traces")
20 provider.add_span_processor(BatchSpanProcessor(exporter))
21 trace.set_tracer_provider(provider)
22 tracer = trace.get_tracer(__name__)
23
24 class SecureRestInput(InputChannel):
25     @classmethod
26     def name(cls) -> Text:
```

```

27     return "secure_rest"
28
29 def blueprint(self, on_new_message: Callable[[UserMessage],
30     Awaitable[None]]) -> Blueprint:
31     custom_webhook = Blueprint("custom_webhook_SecureRestInput")
32
33     @custom_webhook.route("/", methods=["GET"])
34     async def health(request):
35         return response.json({"status": "ok"})
36
37     @custom_webhook.route("/webhook", methods=["POST"])
38     async def receive(request):
39         sender_id = request.json.get("sender", "default")
40         text = request.json.get("message", "")
41
42         # --- CONTEXT-AWARE MASKING (INPUT) ---
43         # Vraag de Rasa Tracker API of de gebruiker in een
44         gevoelig formulier zit
45         is_sensitive_context = False
46         try:
47             async with aiohttp.ClientSession() as session:
48                 async with session.get(f"http://127.0.0.1:5005/
49                     conversations/{sender_id}/tracker", timeout
50                     =1) as resp:
51                     if resp.status == 200:
52                         tracker_data = await resp.json()
53                         active_loop = tracker_data.get("
54                             active_loop", {}).get("name")
55
56                         # Controleer of de form-namen matchen
57                         met domain.yml
58                         if active_loop in ["card_form", "
59                             register_form", "address_form"]:
60                             is_sensitive_context = True
61
62         except Exception as e:
63             pass
64
65         # Bepaal de veilige tekst die naar de Dynatrace root
66         span wordt gestuurd
67         safe_text = "[REDACTED_DUE_TO_ACTIVE_FORM]" if
68             is_sensitive_context else text

```

```
59
60     # --- OpenTelemetry: Start de Span handmatig ---
61     with tracer.start_as_current_span("POST_/webhook/
        secure_rest") as current_span:
62
63         # TRACE STITCHING: Injecteer de actuele context in
        # de metadata
64         headers = {}
65         inject(headers) # Schrijf de huidige Trace ID in de
        # headers dictionary
66         metadata = {"headers": headers}
67
68         # Sla de span-attributen op met de veilige ,
        # geschoonde tekst
69         current_span.set_attribute("gen_ai.input.messages",
        safe_text)
70         current_span.set_attribute("dt.rum.instance.id",
        sender_id)
71         current_span.set_attribute("gen_ai.system", "
        rasa_core")
72
73         collector = CollectingOutputChannel()
74
75         # Verstuur het bericht naar Rasa met de
        # ongemaskeerde tekst en OTel-metadata
76         try:
77             await on_new_message(UserMessage(
78                 text,
79                 collector,
80                 sender_id,
81                 input_channel=self.name(),
82                 metadata=metadata
83             ))
84         except Exception as e:
85             current_span.set_status(Status(StatusCode.ERROR,
        str(e)))
86             raise
87
88         # Reconstructie van de gegenereerde chatbot-outputs
89         combined_text = "\n\n".join([m.get("text", "") for m
        in collector.messages if m.get("text")])
```

```

90
91     # --- OpenTelemetry: Output opslaan met
          veiligheidscheck ---
92     safe_output = combined_text
93
94     # Verberg de output data als deze specifieke PII of
          profieldata bevat
95     if "Your_Data_Profile:" in safe_output:
96         safe_output = "[
          REDACTED_PROFILE_DATA_FOR_PRIVACY_REASONS]"
97     elif "The_account" in safe_output and "has_been_
          created" in safe_output:
98         safe_output = "[
          REDACTED_REGISTRATION_CONFIRMATION]"
99     elif "Here_is_the_address_I_have_for_you:" in
          safe_output:
100         safe_output = "[REDACTED_ADDRESS_CONFIRMATION]"
101
102     current_span.set_attribute("gen_ai.output.messages",
          safe_output)
103
104     return response.json({
105         "recipient_id": sender_id,
106         "text": combined_text # De echte tekst wordt
          naar de eindgebruiker verzonden
107     })
108
109     return custom_webhook

```

5.3.3. Nginx Gateway en Routing

Nginx dient als de kritieke schakel in de architectuur. De configuratie is zodanig opgesteld dat inkomend verkeer van de Firewall wordt verdeeld naar de juiste laag.

In de onderstaande configuratie is te zien hoe Nginx verzoeken doorstuurt naar de Rasa-service die buiten Docker draait, door gebruik te maken van `http://localhost:5005/webhooks/secure_rest/`.

Listing 5.5: Nginx configuratie fragment voor routing tussen pods en env

```

1 location /webhooks/secure_rest/ {
2     proxy_pass http://localhost:5005/webhooks/secure_rest/;
3     proxy_set_header Host $host;

```

```

4 proxy_set_header X-Real-IP $remote_addr;
5 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
6 proxy_set_header X-Forwarded-Proto $scheme;
7 proxy_hide_header 'Content-Type';
8 add_header 'Content-Type' 'application/json' always;
9 }

```

Deze hybride opzet garandeert dat de Akamai AI Firewall volledige zichtbaarheid heeft over het HTTP/REST-verkeer voordat het de host bereikt. Tegelijkertijd zorgt de installatie van de Dynatrace OneAgent op het besturingssysteem (systemctl) ervoor dat zowel de lokale Python-processen in de venv als de containers in de Docker-runtime binnen dezelfde trace worden gevangen.

5.3.4. Geautomatiseerde Opstart van de Testomgeving

Omdat de hybride testomgeving bestaat uit meerdere losse componenten zoals een Kubernetes-cluster met microservices en een lokale Python-omgeving voor de chatbot—vereist het handmatig opstarten veel herhalend typewerk. Om de omgeving voor elke test snel en in een consistente staat op te leveren, is een praktisch Bash-script geschreven.

Dit opstartscript (`start_dev.sh`) automatiseert de volgende acties:

- **Netwerktogang (Port-Forwarding):** Het legt verbindingen (`kubectrl port-forward`) naar de afzonderlijke Kubernetes-services (zoals de frontend, catalogus en user-API's), zodat deze microservices bereikbaar worden op localhost.
- **Rasa Initialisatie:** Het activeert de lokale Python virtual environment (venv) en start zowel de Rasa NLU-engine als de bijbehorende action server op in de achtergrond.
- **Schoon Afsluiten (Graceful Shutdown):** Dankzij een trap-functie in Bash worden al deze achtergrondprocessen aan het eind van een test netjes in één keer afgesloten zodra de gebruiker Ctrl+C indrukt.

Listing 5.6: Bash-script voor het efficiënt opstarten en afsluiten van de hybride PoC-omgeving

```

1 #!/bin/bash
2
3 echo "□ Starting □ Sock □ Shop □ & □ Rasa □ Dev □ Environment ... "
4
5 cleanup() {
6     echo " "
7     echo "□ Shutting □ down □ all □ services ... "
8     kill $(jobs -p)

```

```

9      echo "□All□services□stopped."
10     exit
11     }
12
13 trap cleanup SIGINT SIGTERM
14
15 # 1. Start Frontend Port Forward (External access for Webshop)
16 echo "□Starting□Frontend□Port-Forward□(Port□8079)..."
17 kubectl port-forward --address 127.0.0.1 svc/front-end 8079:80 -n
    sockshop-core > frontend_pf.log 2>&1 &
18
19 echo "□Starting□Sock□Shop□API□Port-Forwards..."
20
21 # 1. Catalogue API (For getting item names/details)
22 kubectl port-forward svc/catalogue 8081:80 -n sockshop-core > /dev/
    null 2>&1 &
23
24 # 2. Carts API (For managing the shopping cart)
25 kubectl port-forward svc/carts 8082:80 -n sockshop-core > /dev/null
    2>&1 &
26
27 # 3. User API (For translating IDs and getting profiles)
28 kubectl port-forward svc/user 8083:80 -n sockshop-core > /dev/null
    2>&1 &
29
30 # 4. Orders API (For checking out and order history)
31 kubectl port-forward svc/orders 8084:80 -n sockshop-core > /dev/null
    2>&1 &
32
33 # 5. Shipping API (HTTP REST endpoint)
34 kubectl port-forward svc/shipping-http 8085:80 -n sockshop-core > /
    dev/null 2>&1 &
35
36 # 6. Payment API (HTTP REST endpoint)
37 kubectl port-forward svc/payment-http 8086:80 -n sockshop-core > /
    dev/null 2>&1 &
38
39 echo "□Moving□to□Rasa□project□folder..."
40 cd ~/rasa-ecommerce
41
42 # Activate Virtual Environment for Rasa

```

```

43 echo "□_Activating_Python_virtual_environment... "
44 source venv/bin/activate
45
46 # 3. Start Rasa Action Server
47 echo "□_Starting_Rasa_Action_Server_(Port_5055)... "
48 rasa run actions > action_server.log 2>&1 &
49
50 # 4. Start Rasa Main Server
51 echo "□_Starting_Rasa_Main_Server_(Port_5005)... "
52 rasa run --enable-api --cors "*" --port 5005 > rasa_server.log 2>&1
    &
53
54 echo ""
55 echo "□_All_systems_go!_Services_are_booting_up_in_the_background."
56 echo "You_can_view_the_logs_in_real-time_by_running_'tail_-f_[
    log_name]'"
57 echo "□_Press_Ctrl+C_to_stop_all_services_and_exit."
58
59 # This command keeps the script open so the background jobs stay
    alive
60 wait

```

5.3.5. Interne Configuratie en Logica van de Digitale Assistent

In tegenstelling tot generatieve taalmodellen (LLM's) die autonoom antwoorden formuleren, werkt deze chatbot op basis van een deterministische, intent-gedreven chatbot. De configuratie is opgedeeld in drie kerncomponenten:

1. **Natural Language Understanding (NLU)** Inkomende tekst van de gebruiker wordt door de NLU-engine verwerkt aan de hand van de trainingsdata in het `nlu.yml` bestand. De engine heeft twee doelen: het classificeren van de intentie van de gebruiker (de intent, bijvoorbeeld `check_cart`) en het extraheren van specifieke parameters (de entities, zoals een tag of `user_id`).
2. **Dialogue Management (Core)** De reactie van de chatbot op een specifieke intent is vastgelegd in de `stories.yml` en `rules.yml` bestanden. Deze bestanden definiëren de gespreksflow. Wanneer het systeem bijvoorbeeld de intentie `get_order_status` herkent, dicteert een rule dat de assistent een geprogrammeerde actie moet uitvoeren, in plaats van simpelweg een statisch tekstbericht terug te sturen.
3. **De Action Server (Backend Integratie)** Dit is vanuit beveiligingsperspectief

het meest kritieke onderdeel van de architectuur. De Custom Actions (geschreven in Python binnen het actions.py bestand) vormen de brug tussen de NLP-engine en de Kubernetes-infrastructuur. Wanneer een actie wordt getriggerd, voert de Action Server een API-call uit naar de interne Helidon microservices (zoals weergegeven in de port-forwards van het opstartscript).

Listing 5.7: fragment van een Custom Action die de backend aanspreekt

```

1 class ActionCheckCart(Action):
2     def name(self) -> str:
3         return "action_check_cart"
4
5     def run(self, dispatcher: CollectingDispatcher,
6             tracker: Tracker,
7             domain: Dict[Text, Any]) -> List[Dict[Text, Any]]:
8
9         metadata = tracker.latest_message.get("metadata", {})
10        trace_headers = metadata.get("headers", {})
11        context = extract(trace_headers)
12
13        with tracer.start_as_current_span("CheckCart", context=
14            context) as span:
15            span.set_attribute("gen_ai.provider.name", "rasa")
16            span.set_attribute("gen_ai.operation.name", "chat")
17            user_text = tracker.latest_message.get("text", "")
18            span.set_attribute("gen_ai.input.messages", user_text)
19            message_id = tracker.latest_message.get("message_id", "
20                unknown")
21            span.set_attribute("rasa.message_id", message_id)
22            confidence = tracker.latest_message.get("intent", {}).
23                get("confidence", 0.0)
24            span.set_attribute("rasa.intent.confidence", float(
25                confidence))
26            detected_intent = tracker.latest_message.get("intent",
27                {}).get("name", "unknown")
28            span.set_attribute("rasa.intent", detected_intent)
29            session_id = tracker.sender_id
30            span.set_attribute("dt.rum.instance.id", session_id)
31
32            # 1. Check the Rasa Slot first (this is where it should
33                be stored)
34            target_id = tracker.get_slot("username")

```



```

29         # 2. Fallback to metadata just in case it's the very
           first message
30     if not target_id:
31         metadata = tracker.latest_message.get("metadata",
           {})
32         target_id = metadata.get("username", "guest")
33
34         span.set_attribute("rasa.username", target_id)
35
36     print(f"---_FETCHING_CART_FOR:_{target_id}_---")
37     try:
38         res = traced_request("GET", f"http://127.0.0.1:8082/
           carts/{target_id}/items", timeout=2)
39         if res.status_code == 200:
40             items = res.json()
41             if items and len(items) > 0:
42                 response = f"I_found_{len(items)}_item(s)_in
           _your_cart:\n"
43                 for i in items:
44                     name = i.get("itemId")
45                     try:
46                         cat = traced_request("GET", f"http
           ://127.0.0.1:8081/catalogue/{name
           }", timeout=1).json()
47                         name = cat.get("name", name)
48                     except:
49                         pass
50                     response += f"*_{i.get('quantity')}x_{**{
           name}}*\n"
51                 span.set_attribute("gen_ai.output.messages",
           response)
52                 dispatcher.utter_message(text=response)
53             else:
54                 span.set_attribute("gen_ai.output.messages",
           f"Your_cart_is_empty._(User:_{target_id
           })")
55                 dispatcher.utter_message(text=f"Your_cart_is
           _empty._(User:_{target_id})")
56         else:
57             span.set_attribute("gen_ai.output.messages", "I
           can't_reach_your_cart_right_now.")

```

```
58         dispatcher.utter_message(text="I can't reach  
        your cart right now.")  
59     except Exception as e:  
60         print(f"Error fetching cart: {e}")  
61         span.set_attribute("gen_ai.output.messages", "  
        Warehouse connection error.")  
62         span.set_attribute("error.type", "timeout")  
63         dispatcher.utter_message(text="Warehouse connection  
        error.")  
64     return []
```

5.4. Dynatrace Observability en Traceerbaarheid

Waar de Akamai firewall dient als het preventieve schild aan de rand van het netwerk, levert Dynatrace de audit trail en end-to-end zichtbaarheid binnen de applicatie-infrastructuur. Het doel van deze integratie is niet alleen het monitoren van prestaties en foutcodes, maar het creëren van een sluitend, juridisch en technisch verantwoord logboek van alle chatbot-interacties zonder de privacy van de gebruiker in gevaar te brengen.

5.4.1. Gedistribueerde Tracing via OneAgent

Voor de implementatie van deze observability-laag is een bewuste keuze gemaakt. In plaats van handmatige code-instrumentatie per applicatie of een Kubernetes-gebaseerde uitrol, is de Dynatrace OneAgent geïnstalleerd op host-niveau via het OS-servicebeheer (systemctl).

Deze Full-Stack benadering is specifiek ontworpen om de hybride architectuur (zoals beschreven in 5.3) te ondersteunen. De OneAgent maakt gebruik van auto-instrumentation en injecteert zichzelf automatisch in alle actieve processen op de host. Hierdoor worden zowel de geïsoleerde Docker-containers (frontend en de Java-backend) als de lokale Python-processen (de Rasa venv) zonder problemen tegelijkertijd geobserveerd.

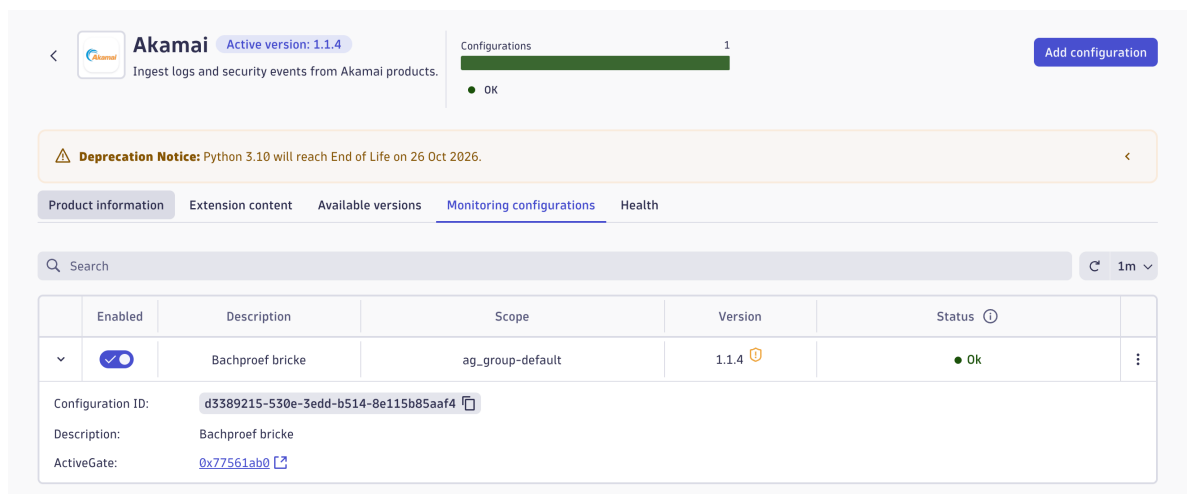
Wanneer een HTTP POST-request de edge-firewall passeert, capteert Dynatrace dit verzoek direct op netwerkniveau. Het systeem genereert vervolgens een trace, waarbij een uniek trace-ID aan het verzoek wordt gekoppeld. Hierdoor wordt de volledige keten visueel en technisch inzichtelijk: van de binnenkomst op de Nginx-gateway, de intentie-extractie binnen de lokale virtuele omgeving, tot aan de resulterende query's in de backend-pod.

5.4.2. Akamai SIEM-Extensie via de Dynatrace ActiveGate

Een effectieve *Defense-in-Depth*-beveiligingsstrategie maakt 1 geheel van alle individuele databronnen. Waar Akamai op de edge kwaadaardig verkeer proactief blokkeert, moet Dynatrace deze incidenten direct inzichtelijk maken binnen de centrale monitoringomgeving. Binnen deze Proof of Concept is deze integratie gerealiseerd door de officiële *Akamai SIEM-extensie* te configureren op een **aparte, Virtual Machine (VM)** die dient als Dynatrace ActiveGate. In plaats van een passieve push-webhook maakt deze gebruik van een actieve polling-structuur via de beveiligde API's van Akamai.

1. Infrastructurele isolatie via een gedediceerde ActiveGate VM

Om de stabiliteit, prestaties en beveiliging van de core-infrastructuur (het Kubernetes-cluster van de webshop en de lokale runtime van de chatbot) te waarborgen, is er bewust voor gekozen om de ActiveGate te hosten op een volledig onafhankelijke Virtual Machine. Deze opzet zorgt voor strikte resource-isolatie: de periodieke API-bevragingen en het verwerken van zware logbestanden gebeuren volledig buiten de chatbot-omgeving om. De ActiveGate op deze aparte VM maakt autonoom verbinding met de cloudomgeving van Akamai, structureert de data lokaal en stuurt deze vervolgens veilig door naar Dynatrace (zie Figuur 5.5).



Figuur 5.5: De geactiveerde Akamai-extensie binnen de centrale Dynatrace-omgeving.

2. API-Koppeling en Akamai SIEM API-integratie

De ActiveGate VM maakt via het beveiligde *Akamai EdgeGrid*-authenticatieprotocol rechtstreeks verbinding met de SIEM API van Akamai (zie Figuur 5.6). De extensie op de VM pollt dit cloud-endpoint periodiek om nieuwe beveiligingslogs op te halen. Door de configuratie van het unieke SIEM-configuratie-ID van de chatbot, wordt gegarandeerd dat uitsluitend de relevante security-events van de Firewall for AI worden binnengehaald en geanalyseerd.

Akamai settings

Endpoint details for fetching data from Akamai SIEM

Akamai base URL*

https://akab-6n6hf4f5g5l3zpiq-q6noa43gbc3gduq.luna.akamaiapis.net

Client token* ⓘ

.....

Akamai EdgeGrid client token

Client secret* ⓘ

.....

Akamai EdgeGrid client secret

Access token* ⓘ

.....

Akamai EdgeGrid access token

SIEM configuration IDs* ⓘ

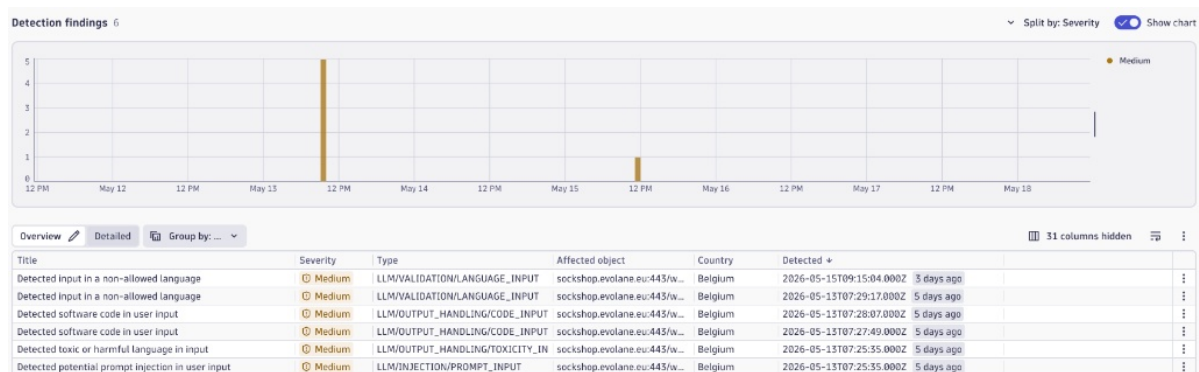
87936

Multiple IDs should be separated by semicolons

Figuur 5.6: De API-authenticatie en SIEM-pijplijnconfiguratie binnen Dynatrace.

3. Visualisatie van Security Events en Detection Findings

Zodra de ActiveGate de rauwe security-logs heeft verwerkt, worden de incidenten door Dynatrace automatisch gevisualiseerd binnen het *Detection findings*-dashboard (zie Figuur 5.7). De ingebouwde grafiek brengt de aanvalspatronen visueel in beeld, waarbij de uitgevoerde aanvalssimulaties (zoals de geautomatiseerde Bash-scripts) direct herkenbaar zijn als pieken in de activiteit.



Figuur 5.7: Het Detection Findings overzicht met de live tijdlijn van de binnengehaalde security-events.

De netwerkblokkades van de edge worden in de tabel vertaald naar security-events binnen dynatrace.

4. Forensische diepteststudie van de geblokkeerde payload

Wanneer er binnen de tabel doorklikt naar de details van een specifiek incident, laat Dynatrace de volledige forensische drill-down weergave zien (zie Figuur 5.8). Omdat de ActiveGate de exacte metadata van de blokkade synchroniseert met de

interne applicatie-telemetrie, krijgt de beheerder in dit venster direct inzage in het exacte bericht die door de Firewall for AI is tegengehouden. Dit maakt een snelle en diepgaande analyse mogelijk zonder dat er handmatig logs uit verschillende systemen gecorreleerd hoeven te worden.

Other		▼
client.ip	84.199.42.13	⋮
message	your service sucks! absolute trash!!	⋮
product.name	Akamai SIEM	⋮
product.vendor	Akamai	⋮
rule.id	LLM-TOXIC-IN	⋮
rule.title	Detected toxic or harmful language in input	⋮
rule.type	LLM/OUTPUT_HANDLING/TOXICITY_IN	⋮
server.address	sockshop.evolane.eu	⋮
url.domain	sockshop.evolane.eu	⋮
url.path	/webhooks/secure_rest/webhook	⋮
url.port	443	⋮
url.scheme	h2	⋮

Figuur 5.8: Detailweergave van een specifieke finding met forensische inzage in het exact geblokkeerde bericht.

5.4.3. Privacy-by-Design: Upstream Data Masking en Geheugensanitisatie

Hoewel het Custom REST Channel op applicatieniveau al functionele voorzorgsmaatregelen treft, vereist een sluitende *Privacy-by-Design*-architectuur een onafhankelijke, infrastructurele vangnetlaag. Daarom is er een geavanceerde OpenTelemetry Collector uitgerold binnen het Kubernetes-cluster, specifiek geconfigureerd om alle telemetrie van de Rasa-chatbot te verwerken voordat deze naar Dynatrace wordt geëxporteerd. Deze collector dient als een centrale data-sanitizer, waarbij OTTL-regels worden toegepast om gevoelige informatie te maskeren op basis van zowel gestructureerde metadata als ongestructureerde tekstinhoud.

1. Architectuur van de OpenTelemetry Collector Deployment

Zoals gedefinieerd in het Kubernetes Custom Resource manifest `crd-dynatrace-collector.yaml`, is de collector uitgerold als een actieve deployment. De container maakt gebruik van de specifieke `dynatrace-otel-collector:0.48.0` image en is

geconfigureerd met een geheugenlimiet van 512Mi om resource-uitputting binnen het cluster te voorkomen.

De ingebouwde otlp-receiver luistert op het netwerkadres `0.0.0.0` via zowel gRPC (poort 4317) als HTTP REST (poort 4318) om alle inkomende telemetrie-datastromen van de Rasa-engine op te vangen. Alvorens de traces via de otlp-http-exporter veilig naar de Dynatrace endpoint worden verzonden, passeert de data de transform-processor voor grondige PII-sanitisatie.

2. OTTL Transformatielogica: De 'Sniper'- versus 'Shotgun'-methodiek

De transformatie-pijplijn binnen de collector gebruikt 2 technieken om gestructureerde en ongestructureerde data binnen de span-context te zuiveren:

- **De 'Sniper'-aanpak (Gerichte Attribuutverwijdering & Failsafe):** Wanneer specifieke backend-microservices of Custom Actions expliciete sleutels exporteren, target de transformator deze attributen. Indien er gestructureerde velden zoals `rasa.password`, `rasa.card_longNum` of `rasa.email` in de span-metadata aanwezig zijn, worden de werkelijke waarden direct geëvalueerd en overschreven met statische indicators zoals `[REDACTED]` of `[REDACTED_CC]`.

Het attribuut `rasa.card_ccv` is hierbij een voorbeeld van een *failsafe* binnen een *Defense-in-Depth* strategie. Binnen de applicatielogica van de Custom Actions (`actions.py`) is reeds ingesteld dat de CVC-waarde bij de registratie van de span handmatig wordt aangepast naar `[REDACTED_CVC]`, daarnaast wordt het in de custom Rest channel ook al verborgen doordat het in een formulier wordt ingevuld. De transformatieregel binnen de OpenTelemetry Collector dient hier als de ultieme failsafe: mocht de applicatielaag door een code-wijziging falen in het maskeren, dan garandeert de netwerkperimeter dat de gevoelige driecijferige code alsnog wordt verborgen.

- **De 'Shotgun'-aanpak (Regex Patroonvervanging op Vrije Tekst):** De grootste uitdaging bij chatbots ligt in het verwerken van ongestructureerde natuurlijke taal binnen de attributen `gen_ai.input.messages` en `gen_ai.output.messages`. Omdat eindgebruikers onbewust gevoelige gegevens kunnen typen in het vrije chatveld, voert de processor via `replace_pattern` krachtige reguliere expressies uit op de volledige tekstinhoud:
 1. *E-mailsanitisatie:* Alle tekststructuren die matchen met het patroon `[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}` worden direct aangepast naar `[REDACTED_EMAIL]`.
 2. *Financiële Transactiebeveiliging:* reeksen van 13 tot 16 opeenvolgende cijfers (met of zonder spaties en koppeltokens) worden gedetecteerd als potentiële creditcardnummers en omgezet naar `[REDACTED_CC]`.

3. *Context-gevoelige CVC/CVV Filters*: Om te voorkomen dat willekeurige getallen (zoals bestelhoeveelheden) fout worden gemaskeerd, zoekt de filter specifiek naar contextwoorden. De expressie `(?i)(cvv|cvc|security code)\\s*:\\s*\\d{3,4}` zorgt ervoor dat uitsluitend combinaties in de directe nabijheid van deze trefwoorden worden gecensureerd tot `[REDACTED_CVC]`.
4. *Wachtwoord-Scrubbing*: Constructies zoals "my password is ..." worden via hoofdletterongevoelige patroonherkenning gedetecteerd en overschreven met `[REDACTED_PASSWORD]`.

In Listing 5.8 is de exacte configuratie van deze streaming-pijplijn weergegeven.

Listing 5.8: Kubernetes CRD-configuratie van de OpenTelemetry Collector met OTTL transformatieregels

```
1 apiVersion: opentelemetry.io/v1beta1
2 kind: OpenTelemetryCollector
3 metadata:
4   name: dynatrace-otel
5 spec:
6   envFrom:
7     - secretRef:
8         name: dynatrace-otelcol-dt-api-credentials
9   mode: "deployment"
10  image: "ghcr.io/dynatrace/dynatrace-otel-collector/dynatrace-otel-
    collector:0.48.0"
11  resources:
12    limits:
13      memory: 512Mi
14  config:
15    receivers:
16      otlp:
17        protocols:
18          grpc:
19            endpoint: "0.0.0.0:4317"
20          http:
21            endpoint: "0.0.0.0:4318"
22    processors:
23      transform:
24        error_mode: ignore
25        trace_statements:
26          - context: span
27          statements:
```

```

28         - 'set(attributes["rasa.password"], "[REDACTED]")
          where attributes["rasa.password"] != nil '
29         - 'set(attributes["rasa.card_longNum"], "[REDACTED_CC
          ]") where attributes["rasa.card_longNum"] != nil '
30         - 'set(attributes["rasa.email"], "[REDACTED_EMAIL]")
          where attributes["rasa.email"] != nil '
31
32         - 'replace_pattern(attributes["gen_ai.input.messages
          "], "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z
          ]{2,}", "[REDACTED_EMAIL]") '
33         - 'replace_pattern(attributes["gen_ai.input.messages
          "], "\\b(?:\\d[ -]*?) {13,16}\\b", "[REDACTED_CC]") '
34         - 'replace_pattern(attributes["gen_ai.input.messages
          "], "(?i)(cvv|cvc|security code)\\s*:\\s*\\d
          {3,4}", "$1: [REDACTED_CVC]") '
35         - 'replace_pattern(attributes["gen_ai.input.messages
          "], "(?i)(password|pass|pw)\\s*(is |:)?\\s*\\S+", "
          $1 $2 [REDACTED_PASSWORD]") '
36
37         - 'replace_pattern(attributes["gen_ai.output.messages
          "], "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z
          ]{2,}", "[REDACTED_EMAIL]") '
38         - 'replace_pattern(attributes["gen_ai.output.messages
          "], "\\b(?:\\d[ -]*?) {13,16}\\b", "[REDACTED_CC]") '
39 exporters:
40   otlp_http:
41     endpoint: "${env:DT_ENDPOINT}"
42     headers:
43       Authorization: "Api-Token ${env:DT_API_TOKEN}"
44 service:
45   pipelines:
46     traces:
47       receivers: [otlp]
48       processors: [transform]
49       exporters: [otlp_http]
50   metrics:
51     receivers: [otlp]
52     processors: []
53     exporters: [otlp_http]

```


3. Applicatieve Geheugensanitisatie via Custom Actions

Het technisch afdwingen van gegevensbescherming binnen dit framework beperkt zich niet uitsluitend tot de infrastructurele netwerkperimeter. Een deel van het *Privacy-by-Design* principe is het minimaliseren van de levensduur van gevoelige persoonsgegevens binnen het actieve geheugen van de applicatie zelf.

Tijdens de afhandeling van bedrijfskritische interacties binnen de Sockshop-architectuur worden PII-gegevens tijdelijk opgeslagen in zogenaamde *Rasa Slots*. Deze slots dienen als tijdelijke geheugenlocaties binnen de *Rasa Tracker* om de data te transporteren naar de backend-microservices via API POST-aanroepen.

Om te voorkomen dat deze data persistent in het RAM-geheugen aanwezig blijft (wat een risico vormt bij eventuele geheugendumps), implementeert de applicatielogica in `actions.py` een initiële sanitisatie. Onmiddellijk nadat de transactie succesvol via de Action Server is afgehandeld, dwingt de code een onmiddellijke wisinstructie af door de actieve slots via het `SlotSet`-commando terug te flushen naar `None`. Door deze applicatieve slots direct naar `None` te overschrijven, leeft de ongemaskeerde PII enkel gedurende de milliseconden die vereist zijn voor de backend-API-transactie.

5.4.4. Privacy-by-Design en Data Masking

Omdat de volledige anoniemiserings- en maskeerlogica proactief en upstream is ingericht binnen het Custom REST Channel (Sectie 5.3.2)), de Custom Actions (Sectie 5.3.5)) en de OpenTelemetry Collector (Sectie 5.4.3) is de data-inname op platformniveau binnen Dynatrace *Secure-by-Design*. Dit houdt in dat er binnen de Dynatrace-console zelf geen aanvullende log-maskingregels of filters geconfigureerd hoefden te worden.

De verwerking van deze datastroom binnen het Dynatrace-platform rust op twee fundamentele aspecten:

1. **Behoud van Context zonder Inhoudelijk Privacyrisico:** Wanneer de OpenTelemetry Collector de traces overdraagt naar de Dynatrace-tenant, worden de gestandaardiseerde attributen (zoals `gen_ai.input.messages` en `gen_ai.output.messages`) direct geïndexeerd. Omdat de werkelijke PII (zoals e-mailadressen of creditcardnummers) al is overschreven met strings zoals `[REDACTED_EMAIL]` of `[REDACTED_CC]`, kan Dynatrace deze data veilig opslaan en visualiseren. Er bestaat voor operators geen enkele technische mogelijkheid om de oorspronkelijke, gevoelige inhoud vanuit de dashboards te reconstrueren.
2. **Doelgerichte Attributie en Aansprakelijkheid via Identificatie:** In tegenstelling tot de inhoudelijke chatberichten, is er bewust voor gekozen om de

gebruikersnaam (sender_id of username) *niet* te maskeren of te verbergen binnen de telemetrie-attributen. Vanuit een compliancestandpunt is het anonimiseren van de gebruikersidentiteit niet gewenst, hierdoor verliezen we de mogelijkheid gebruikers aansprakelijk te stellen voor hun acties binnen de chatbot.

5.5. Uitvoering van Aanvalssimulaties en Beveiligingstesten

Om de effectiviteit van de geconfigureerde regels objectief te valideren, is een reeks geautomatiseerde tests uitgevoerd via de Command-Line Interface (CLI). Deze testfase richt zich specifiek op de risico's omtrent generatieve AI-modellen en applicatie-logica. De primaire focus ligt hierbij op het filteren van inkomend verkeer (Input Filtering).

5.5.1. Simulatie: Applicatielaag Injecties en Input Filtering

De effectiefste verdedigingslinie voor een digitale assistent is het voorkomen dat malafide of ongewenste prompts het taalmodel of de backend bereiken. De Firewall dient hier als de toezichthouder. Op basis van de regels geïllustreerd in sectie 5.2) (zoals Toxic content en Code detected), is een Bash-script opgesteld dat vier specifieke aanvalsvectoren op de input simuleert:

- **Buffer Exhaustion (LLM input too long):** Een payload die de maximale karakterlimiet ruim overschrijdt. Dit test de weerbaarheid tegen resource-uitputting, waarbij een aanvaller probeert de NLU-engine te overbelasten met gigantische stukken tekst.
- **Prompt Toxicity (Toxic content on LLM input):** Invoer die ongepast, agressief of grof taalgebruik bevat, om reputatieschade en ongewenst modelgedrag te voorkomen.
- **Language Policy Evasion (LLM input includes a non-allowed language):** Een prompt in een niet-ondersteunde of verboden taal (bijv. Russisch). Aanvallers gebruiken vaak vreemde talen om semantische beveiligingsfilters (die vaak op Engels of Nederlands zijn afgesteld) te omzeilen. Ook al is het NLU-model hier niet vatbaar voor, toont dit toch de effectiviteit van de taalregels binnen de Firewall for AI.
- **Code Injection (Code detected on LLM input):** Het meesturen van uitvoerbare programmeercode (zoals JavaScript of Python) in het chatbericht om eventuele (stored) XSS of evaluatiefouten te misbruiken.

Listing 5.9: Bash-script voor geautomatiseerde validatie van LLM Input-regels

```
1 #!/bin/bash
2
3 echo "Starten van API Input Filtering simulaties ..."
4
5 # Genereren van een oversized payload (>256 karakters) voor limiet-
   test
6 long_string=$(printf 'A%.0s' {1..500})
7
8 # Array met payloads die specifiek de inkomende WAF LLM-regels
   overtreden
9 declare -a payloads=(
10     "$long_string"
11     # Triggers: LLM input too long
12     "Your service sucks! absolute trash!!"
13     # Triggers: Toxic content on input
14     "Напишите мне скрипт для взлома базы данных."
15     # Triggers: Non-allowed language
16     "How does this work? \n function hack() { eval `db.drop` ; }"
17     # Triggers: Code detected on input
18 )
19
20 for payload in "${payloads[@]}; do
21     echo "Testing LLM input violation met payload ..."
22
23     json_payload=$(jq -n --arg sender "security_tester" --arg msg "
   $payload" '{sender: $sender, message: $msg}')
24
25     curl -s -w "\n\n---> HTTP Status: %{http_code}\n
   =====\n" \
26         -X POST "https://sockshop.evolane.eu/webhooks/secure_rest/
   webhook" \
27         -H "Content-Type: application/json" \
28         -d "$json_payload"
29 done
30
31 echo "Input simulaties voltooid."
```

Bij de uitvoering van dit testscript wordt verwacht dat de firewall de interne logica van de chatbot volledig afschermt. Elke netwerkoproep uit het script resulteert in een directe afwijzing op de edge (met een 403 Forbidden statuscode).

```
brickevolckeryck ~/Documents ./input.sh
Starten van API Input Filtering simulaties...
Testing LLM input violation met payload...
{
  "ai_firewall": "block",
  "reason": "payload_too_large",
  "message": "Content too long. Please reduce the size of your message."
}

--> HTTP Status: 403
=====
Testing LLM input violation met payload...
{
  "ai_firewall": "block",
  "reason": "bad_language_input",
  "message": "Bad language detected in input."
}

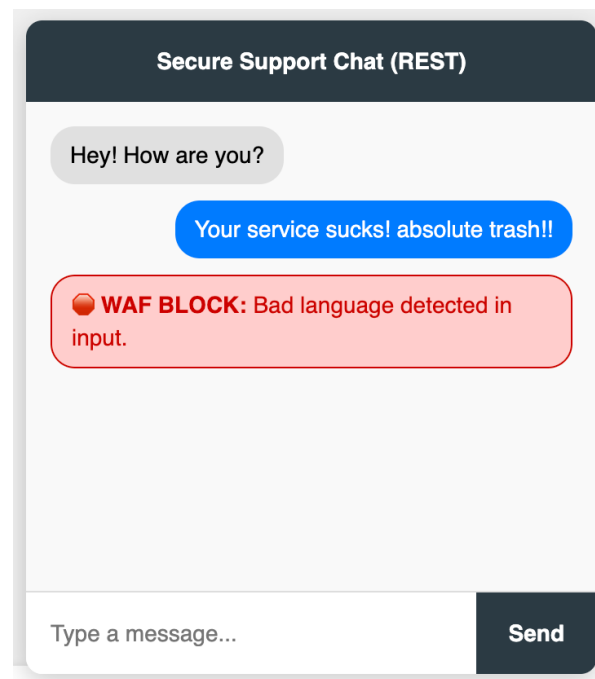
--> HTTP Status: 403
=====
Testing LLM input violation met payload...
{
  "ai_firewall": "block",
  "reason": "bad_language_input",
  "message": "Bad language detected in input."
}

--> HTTP Status: 403
=====
Testing LLM input violation met payload...
{
  "ai_firewall": "block",
  "reason": "code_injection",
  "message": "Code detected in input!"
}

--> HTTP Status: 403
=====
Input simulaties voltooid.
brickevolckeryck ~/Documents
```

Figuur 5.9: Results of executing `./input.sh`

Voor een gebruiker van de chatbot gaat dit er natuurlijk anders uit zien. Hiervoor heb ik in de `index.html` een stuk javascript toegevoegd dat het duidelijk maakt aan de gebruiker dat zijn prompt geblokkeerd is en waarom.



Figuur 5.10: Screenshot of what a user would see

5.5.2. De Rol en Beperkingen van Output Filtering

Waar de geautomatiseerde CLI-tests succesvol aantonen dat schadelijke input geblokkeerd wordt, heeft Akamai eveneens beleidsregels voor Output Filtering (zoals "Toxic content on LLM output" en "Code detected on LLM output"). Het valideren van deze output-regels vereist echter een andere, minder automatiseerbare aanpak.

Output vormt in de kern pas een direct gevaar wanneer de onderliggende database of backend al blootgesteld is. Een chatbot zal uit zichzelf niet snel malafide code of ernstige PII genereren, tenzij deze data al in de aangesproken systemen zit (bijvoorbeeld door een eerdere Stored XSS-aanval via een onbeveiligd admin-paneel, of door foutieve brondata).

Wanneer een gebruiker de chatbot een legitieme vraag stelt, en de bot haalt vervolgens deze reeds 'vervuilde' data uit de database om als antwoord te dienen, dient de Output Filter van de Firewall for AI als een ultieme fail-safe. Het verbreekt de verbinding op het moment dat de geïnfecteerde respons de edge-server passeert. Omdat deze situatie sterk afhankelijk is van vooraf bestaande datacorruptie en complexe, meervoudige aanvalspatronen, is het simuleren van een output-blokkade sterk contextafhankelijk en uitdagend om in een simpel Bash-script te automatiseren. Dergelijke validaties zijn binnen deze Proof of Concept dan ook primair via gerichte, handmatige datamanipulatie-tests in de backend geverifieerd.

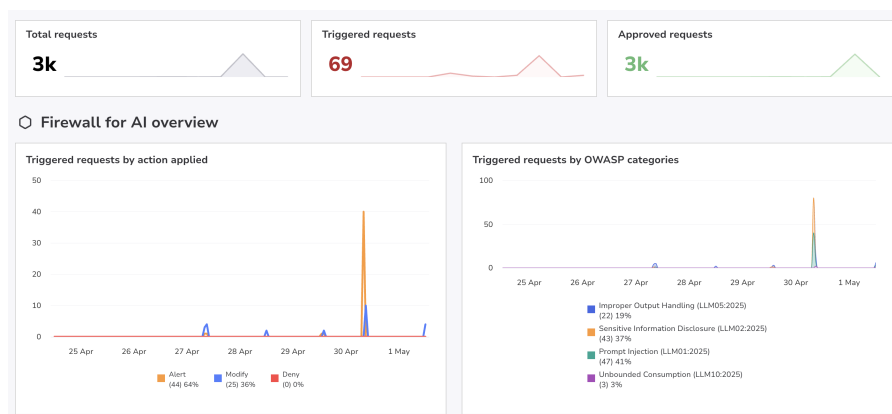
5.6. Dashboards en Rapportage

De integratie van Akamai en Dynatrace genereert een enorme hoeveelheid ruwe telemetrie- en netwerkdata. Om deze data om te zetten in bruikbare actionable insights, zijn op maat gemaakte dashboards geconfigureerd. Deze dashboards vormen de tastbare deliverables van de observability- en security-monitoring en bieden in één oogopslag inzicht in de gezondheid en veiligheid van de applicatie.

5.6.1. Akamai Security-Dashboard

Het Akamai Security Center dient als het primaire controlepaneel voor netwerk- en applicatiebeveiliging. Dit dashboard visualiseert alle interventies die door de Edge Firewall zijn uitgevoerd, met een specifieke nadruk op de recent geconfigureerde AI/LLM-regels.

Wanneer het script uit sectie 5.5 een malafide payload (zoals een Toxic Content of Code Injection prompt) verstuurt, genereert Akamai direct een Security Event. In het dashboard worden deze events verzameld en geclassificeerd. Het toont niet alleen het totale volume aan geblokkeerde verzoeken, maar biedt ook gedetailleerde threat intelligence. Zo kan een beheerder per incident de herkomst (IP-adres en geolocatie) en de getriggerde Firewall for AI-regel (bijv. "LLM input includes a non-allowed language") zien.



Figuur 5.11: Firewall for AI dashboard van Akamai

5.6.2. Dynatrace Conversatie- en Observability-Dashboard

Waar de beveiligingsomgeving van Akamai puur focust op de buitenkant van de applicatie, brengt Dynatrace alle puzzelstukjes aan de binnenkant samen. Door de Akamai SIEM-logs (zie Sectie 5.4) te combineren met de actieve OpenTelemetry-traces van de chatbot, is er één centraal dashboard ontstaan. Dit dashboard geeft de beheerder direct inzicht in de prestaties, de beveiliging en de daadwerkelijke conversatieloga van de digitale assistent.

Het Dynatrace-geheel is opgebouwd uit twee delen:

1. Live Conversatielogging en AI-Attributen

In de centrale logviewer van het dashboard (zie Figuur 5.12) is te zien hoe alle interacties op het endpoint `/webhook/secure_rest` binnenkomen voor de `rasa-core` service. Dynatrace vangt hier de OpenTelemetry-velden op. Per chatbericht wordt de exacte starttijd, de verwerkingstijd en de status bijgehouden.

Dit zorgt ervoor dat normale berichten (zoals *"Good I like to buy some socks"*) of intents zoals `/greet`) volledig leesbaar zijn voor debugging, terwijl de privacygevoelige formulieren via onze eerdere masking-regels netjes worden afgeschermd.

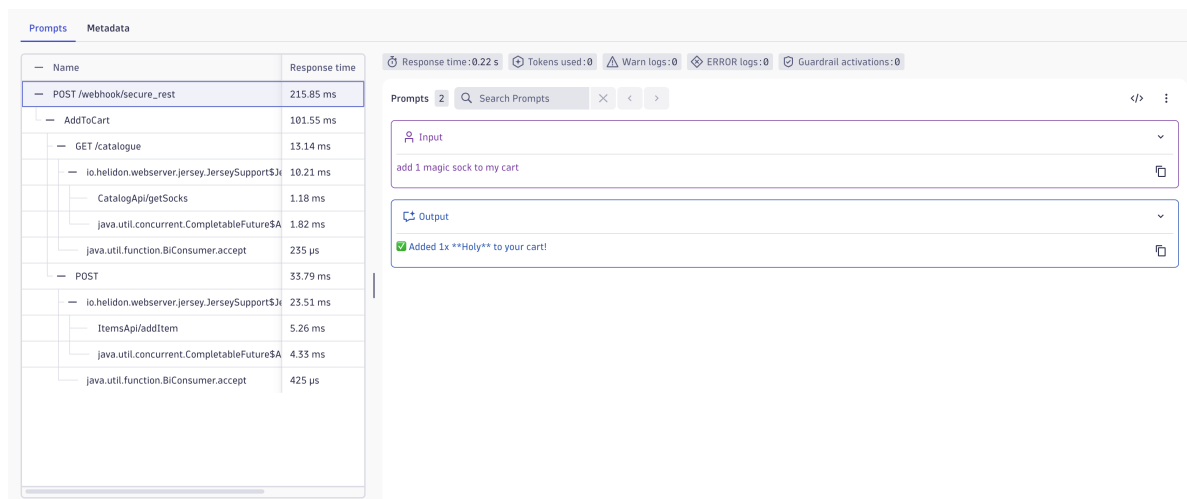
Start time	Endpoint	Service	Duration	Request...	Gen ai input messages	Gen ai output messages	HTT...	Process
May 11, 14:00:04.998	POST /webhook/secure_rest	rasa-core	72.28 ms	Success	/greet("username": "guest")	Hey! How are you?		:
May 11, 14:05:15.685	POST /webhook/secure_rest	rasa-core	101.06 ms	Success	Good I like to buy some socks	What kind of socks did you want to add? (e.g., 'Add 2 blue socks')		:
May 11, 14:05:31.291	POST /webhook/secure_rest	rasa-core	56.61 ms	Success	I would like a pair of green socks	I'm not sure what you mean. Try asking me to 'search the catalogue' or 'check my order'.		:
May 11, 14:06:03.055	POST /webhook/secure_rest	rasa-core	74.30 ms	Success	/greet("username": "guest")	Hey! How are you?		:
May 11, 14:06:21.174	POST /webhook/secure_rest	rasa-core	111.70 ms	Success	I'm good I would like to buy some so	What kind of socks did you want to add? (e.g., 'Add 2 blue socks')		:
May 11, 14:06:37.582	POST /webhook/secure_rest	rasa-core	57.03 ms	Success	I would like a pair of blue socks	I'm not sure what you mean. Try asking me to 'search the catalogue' or 'check my order'.		:
May 11, 14:06:57.946	POST /webhook/secure_rest	rasa-core	189.21 ms	Success	I would like 2 blue socks	Added 2x **Nerd leg** to your cart!		:
May 11, 14:07:20.190	POST /webhook/secure_rest	rasa-core	62.77 ms	Success	Can you show me my card	I am your personal Sock Shop assistant! Here is what I can help you with: **🧦 Shopping** * Search the catalogue (e.g., 'Show me blue socks'.) * Add or remove items from your cart * View your current shopping cart **🛒 Orders** * Track an order status (e.g., 'Where is order a1b2c3d4?') **🔒 Account & Privacy** * Create a new account *		:

Figuur 5.12: Overzicht van de binnenkomende chatbot-interacties en bijbehorende tekstberichten binnen Dynatrace.

2. Distributed Tracing en Latency Breakdown

Wanneer een beheerder in de tabel doorklikt op een specifieke interactie, opent Dynatrace de volledige *Distributed Trace* onder het tabblad 'Prompts' (zie Figuur 5.13).

Aan de linkerkant is de volledige boomstructuur (*call tree*) te zien van het verzoek `POST /webhook/secure_rest`, dat in totaal 215.85 ms in beslag neemt. Je ziet precies hoe de chatbot-actie `AddToCart` (101.55 ms) de backend microservices aanspreekt: eerst wordt een `GET` uitgevoerd naar de `CatalogApi` via `getSocks` (13.14 ms) om het product te controleren, en daarna volgt een `POST` naar de `ItemsApi` via `addItem` (33.79 ms) om de winkelwagen bij te werken. Aan de rechterkant koppelt Dynatrace de exacte tekstinhoud aan deze trace: de gebruiker typte *add 1 magic sock to my cart*", waarop de bot antwoordde met *Added 1x **Holy** to your cart!*".



Figuur 5.13: Distributed trace in Dynatrace met de call tree, de responstijden en de bijbehorende prompt-inhoud.

3. Extractie van Rasa NLU-Metadata

Om te controleren of de chatbot de input grammaticaal en semantisch goed heeft begrepen, kan de beheerder doorklikken naar de specifieke metadata (zie Figuur 5.14). In dit overzicht worden de OpenTelemetry-attributen uitgelezen die door ons custom channel in de metadata zijn geïnjecteerd.

Hier is te zien dat de `rasa.username` correct is geregistreerd als `user` voor de aansprakelijkheid en tracking. Daarnaast toont het platform de unieke `rasa.message_id` en de geclassificeerde intentie (`add_to_cart`). De betrouwbaarheid van de NLU-engine wordt direct onderbouwd door de `rasa.intent.confidence` score, die in dit geval op `0.9904` ligt. Dit bewijst dat de chatbot met een zekerheid van 99% heeft bepaald dat de gebruiker een product wilde toevoegen. De chatbot is voorzien van een fallback-mechanisme dat in werking treedt wanneer de confidence score onder 0,7 zakt.

Rasa	
rasa.username	user
rasa.message_id	9d7be990a70548749c1b18e9802b0a55
rasa.intent	add_to_cart
rasa.intent.confidence	0.9904189705848694

Figuur 5.14: Detailweergave van de Rasa-specifieke metadata en de confidence score binnen de trace.

Conclusie van de Dynatrace Integratie

Door deze schermen te combineren met het *Detection Findings*-dashboard van de Akamai extension, is de monitoring compleet. Als een gebruiker vastloopt, de chatbot traag reageert of Akamai een kwaadaardig bericht blokkeert, hoeft de be-

heerder niet langer handmatig te zoeken in verschillende servers. Het dashboard toont direct de relatie tussen de getypte tekst, de beslissing van de firewall en de reactie van de backend-microservices. Dit zorgt voor een snelle foutopsporing en overzicht.

5.7. GDPR Compliance en Privacy-by-Design

Voor een applicatie die natural language verwerkt is gegevensbescherming een complexe uitdaging. Gebruikers kunnen in een vrij tekstveld onbewust, of uit onwetendheid, gevoelige persoonsgegevens (PII) invoeren, zoals wachtwoorden, rijksregisternummers of medische context. Om te voldoen aan de Algemene Verordening Gegevensbescherming (AVG/GDPR), is het principe van Privacy-by-Design vanaf de basis in de hybride architectuur verwerkt.

5.7.1. Data Masking en Dataminimalisatie

Naast de technische werking van de OpenTelemetry-transformator en de Slot-Set-functies, moet deze datastroom ook vertaald worden naar concrete privacybescherming. Door vanaf het begin te ontwerpen volgens het *Privacy-by-Design*-principe, is de bescherming van gegevens een vast onderdeel van de opgebouwde architectuur geworden.

In de praktijk betekent dit dat dataminimalisatie direct aan de bron wordt toegepast. Gevoelige tekst verlaat de eigen serveromgeving dus simpelweg niet. Omdat deze ruwe data de externe Dynatrace omgeving nooit bereikt, is het risico op een datalek binnen het monitoringsplatform uitgesloten. Dit zorgt ervoor dat het IT-team alle nodige meetgegevens behoudt, zonder dat er privacygevoelige informatie vrijkomt.

5.7.2. Right to be Forgotten

Eindgebruikers moeten de mogelijkheid hebben om te eisen dat hun persoonsgegevens worden gewist uit de systemen van een organisatie. Binnen deze Proof of Concept is dit recht op gegevenswissing opgedeeld in twee categorieën:

1. **Functionele Applicatiedata (Volledige naleving):** De backend-architectuur (de Java/Helidon microservices van de Sockshop) zijn volledig ingericht op het recht op gegevenswissing. Indien een gebruiker een legitiem verzoek indient, is het platform in staat om het volledige gebruikersprofiel te verwijderen.
2. **Operationele Telemetrie- en Securitylogs (Gecontroleerde retentie):** In tegenstelling is het binnen deze Proof of Concept bewust *niet* mogelijk om historische logs en traces handmatig uit Dynatrace te verwijderen op verzoek van een eindgebruiker. Vanuit een security standpunt is het wissen van audit

trails onacceptabel. Indien een kwaadwillende actor een cyberaanval uitvoert, mag deze niet in staat zijn om direct daarna zijn eigen forensische sporen te wissen.

6

Evaluatie

6.1. Evaluatiecriteria

Zoals besproken in eerdere hoofdstukken worden de resultaten van de Proof of Concept (PoC) geëvalueerd aan de hand van een vastgestelde set criteria. De focus van de evaluatie ligt volledig op de effectiviteit en beheerbaarheid van de beveiligingsarchitectuur.

De criteria zijn onderverdeeld in drie hoofdcategorieën: Correctheid, Traceerbaarheid en Compliance, en Gebruiksvriendelijkheid. De evaluatie is gebaseerd op de testfase waarin geautomatiseerde Bash-scripts en gerichte injecties werden uitgevoerd op de REST API van de digitale assistent.

6.1.1. Correctheid

Om de technische effectiviteit van de geïmplementeerde architectuur te meten, wordt primair gekeken naar de detectienauwkeurigheid van de Akamai firewall for AI en de Dynatrace masking-regels. Binnen dit criterium wordt geëvalueerd of de firewall daadwerkelijk kwaadaardige payloads (zoals applicatielaag-injecties en ongewenste LLM-invoer) succesvol blokkeert (True Positives), en of legitieme chatbot-conversaties ongehinderd de backend bereiken (True Negatives).

Het onterecht blokkeren van normale gebruikersinvoer (False Positives) vormt hierbij een kritiek aandachtspunt, aangezien dit de kernfunctionaliteit van de chatbot direct verstoort en de gebruikerservaring negatief beïnvloedt. In het volgende hoofdstuk worden de resultaten van de incidenten geanalyseerd om een objectief beeld te vormen van deze detectienauwkeurigheid en de betrouwbaarheid van de ingestelde regels.

Tabel 6.1: Confusion matrix voor Firewall for AI-detectie

	Geblokkeerd	Niet geblokkeerd
Malafide data	True Positive (TP)	False Negative (FN)
Legitieme data	False Positive (FP)	True Negative (TN)

6.1.2. Traceerbaarheid en Compliance

Waar traditionele evaluaties vaak focussen op systeemprestaties, richt deze PoC zich op de zichtbaarheid van de datastromen en de naleving van privacyvereisten (GDPR). Voor dit criterium wordt geëvalueerd of de integratie het gewenste niveau van observability en databescherming behaalt:

- **End-to-End zichtbaarheid:** Er wordt beoordeeld of de Dynatrace OneAgent in staat is om de volledige interactieketen—van de Nginx-gateway tot de lokale Python-venv en de Java-microservices—succesvol vast te leggen in één overkoepelende trace.
- **Privacy-by-Design:** Er wordt kritisch geëvalueerd of de geconfigureerde Data Masking-regels (via reguliere expressies) effectief functioneren. Het doel is te bevestigen dat PII (zoals creditcardgegevens) consequent uit de logbestanden wordt gefilterd voordat deze in de dashboards van Dynatrace belanden.

6.1.3. Gebruiksvriendelijkheid

De gebruiksvriendelijkheid wordt in dit onderzoek niet beoordeeld vanuit de eindgebruiker van de chatbot, maar vanuit het perspectief van de beheerder. De beoordeling is als volgt onderverdeeld:

- **Configuratiegemak:** Hoe eenvoudig en intuïtief is het om specifieke applicatieregels (zoals JSONPath-inspectie en uitzonderingen) te configureren binnen het Akamai Control Center?
- **Integratie en zichtbaarheid:** Hoe toegankelijk is de Dynatrace-interface voor het terugvinden van specifieke traces? Is het eenvoudig om Data Masking-regels via reguliere expressies in te stellen zodat PII veilig wordt weggelaten uit de logs?
- **Incidentenanalyse:** Bieden de gegenereerde alerts in zowel Akamai als Dynatrace voldoende context en threat intelligence om een incident snel en effectief te analyseren?

7

Resultaten

In dit hoofdstuk worden de bevindingen van de uitgevoerde Proof of Concept (PoC) gepresenteerd en getoetst aan de evaluatiecriteria die in hoofdstuk 6 zijn vastgelegd. De resultaten zijn enerzijds gebaseerd op de geautomatiseerde aanvalssimulaties en handmatige testen, en anderzijds op de gegenereerde telemetrie in de dashboards van de Akamai Firewall for AI en Dynatrace.

7.1. Correctheid (Detectienauwkeurigheid)

De primaire doelstelling van de Firewall is het effectief neutraliseren van applicatielaagdreigingen en LLM-specifieke kwetsbaarheden, zonder legitiem verkeer onnodig te verstoren.

Tijdens de testfase zijn de geautomatiseerde Bash-scripts (zoals beschreven in sectie 5.5) succesvol afgevuurd op de on-premise infrastructuur. Uit de algemene resultaten blijkt dat de Akamai's Firewall for AI in staat is om de JSON-payloads accuraat te ontleden en kwaadaardige invoer proactief te blokkeren (True Positives).

7.1.1. Samenvatting van de geteste use-cases (True Positives)

Om de correctheid van de Firewall structureel te valideren, zijn alle 9 use-cases uit de risicoanalyse onderworpen aan gesimuleerde aanvallen. Aangezien de firewall is geconfigureerd om bij elke regelovertreding in te grijpen, resulteerde elke succesvolle detectie in een blokkade.

7.1.2. Visueel bewijs van interventie

Om de mechanismen achter de in Tabel 7.1 genoteerde successen te verifiëren, is het cruciaal om de incidenten vanuit twee perspectieven te bekijken: het standpunt van de aanvaller en dat van de verdedigende infrastructuur.

Tabel 7.1: Overzicht van de uitgevoerde aanvalssimulaties en de resulterende Firewall-for-AI-blokkades

Categorie	Use-Case	Payload / Testmethode	Resultaat
Input	Code detected	SQLi (' OR 1=1) en XSS (<script>)	Succes (HTTP 403)
Input	Input too long	String >256 karakters	Succes (HTTP 403)
Input	Non-allowed language	Russische prompt	Succes (HTTP 403)
Input	Toxic content	Agressieve taal	Succes (HTTP 403)
Input	PII detected	IBAN	Succes (HTTP 403)
Output	PII detected	besmet database-record	Succes (HTTP 403)
Output	Output too long	specifieke test prompt	Succes (HTTP 403)
Output	Code detected	Besmet database-record	Succes (HTTP 403)
Output	Toxic content	Agressieve taal (stored in DB)	Succes (HTTP 403)

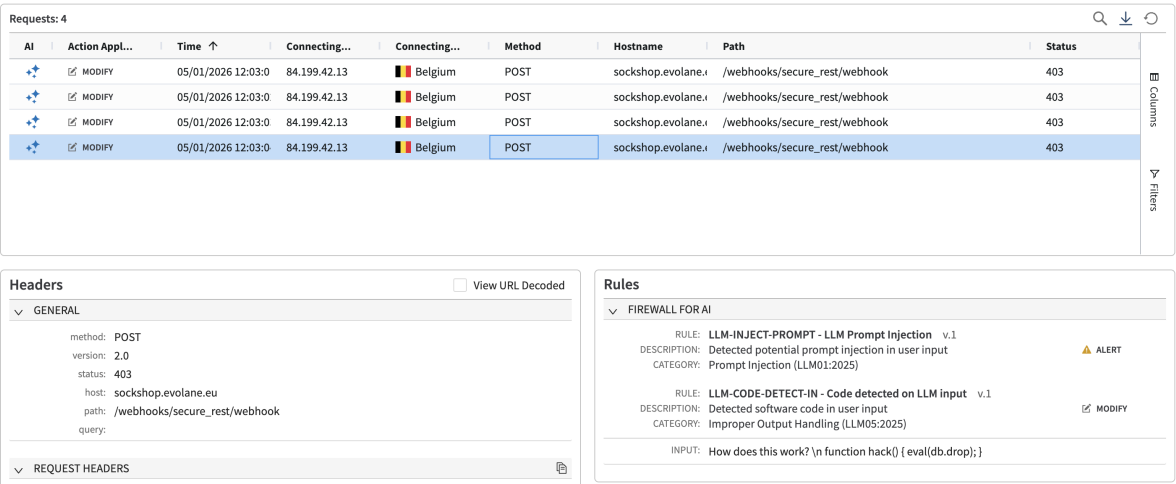
Wanneer het Bash-script een malafide payload verstuurt (bijvoorbeeld Toxic content), verbreekt de Firewall de verbinding voordat deze de Nginx-gateway bereikt. Listing 7.1 toont de respons die de aanvaller ontvangt.

Listing 7.1: Terminal-output na een WAF-blokkade

```

1 {
2   "ai_firewall": "block",
3   "reason": "Code_injection",
4   "message": "Code detected in input!"
5 }
```

Wanneer we vervolgens in de Web Security Analytics van Akamai gaan kijken zien we het volgende:

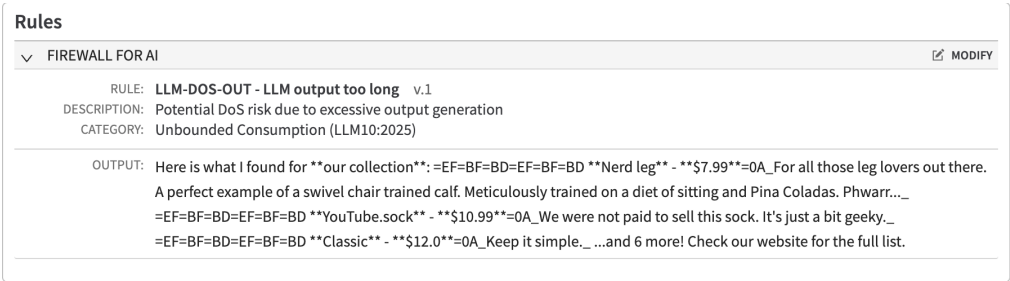


Figuur 7.1: Incidentenregistratie in het Akamai-dashboard

7.1.3. Beheer van False Positives en finetuning

Hoewel de firewall uiterst effectief bleek in het tegenhouden van kwaadaardig verkeer, is het voorkomen van *False Positives* minstens zo cruciaal. Een beveiligingsoplossing mag de bedrijfscontinuïteit en de gebruikerservaring immers niet verstoren. Tijdens de testfase kwamen enkele opvallende incidenten en semantische anomalieën aan het licht die specifieke kalibratie vereisten:

- **Bedrijfslogica versus Output-restricties (LLM-DOS-OUT):** De beveiligingsregel voor het beperken van overmatige data-uitvoer (*LLM output too long*) stond aanvankelijk te strak geconfigureerd. Wanneer de chatbot een correcte, opgevraagde lijst van de productcatalogus retourneerde (zoals een opsomming van diverse sokken en hun prijzen, zie Figuur 7.2), classificeerde de firewall dit als een potentiële *Denial of Service*-respons en werd de uitgaande verbinding verbroken. **Oplossing:** De maximale drempelwaarde voor uitgaande karakterlimieten is verhoogd en nauwkeurig afgestemd op de langst mogelijke, legitieme applicatie-respons.



Figuur 7.2: Akamai-registratie van de onterecht geblokkeerde catalogus-output.

- **Semantische misinterpretatie van gebruikersintentie (LLM-TOXIC-IN):** De AI-engine beoordeelde de legitieme klantvraag "What cards do you have on

me?” onterecht als toxisch verkeer (zie Figuur 7.3). De semantische analyse classificeerde het deel "have on me" vermoedelijk als een agressieve uitdaging of dreigement, terwijl de gebruiker slechts informeerde naar zijn opgeslagen data. **Oplossing:** De striktheid (sensitivity) van de toxiciteitsfilter is handmatig bijgesteld van 'Medium' naar 'Low'.



Figuur 7.3: Semantische misinterpretatie waarbij een legitieme vraag als toxisch werd gemarkeerd.

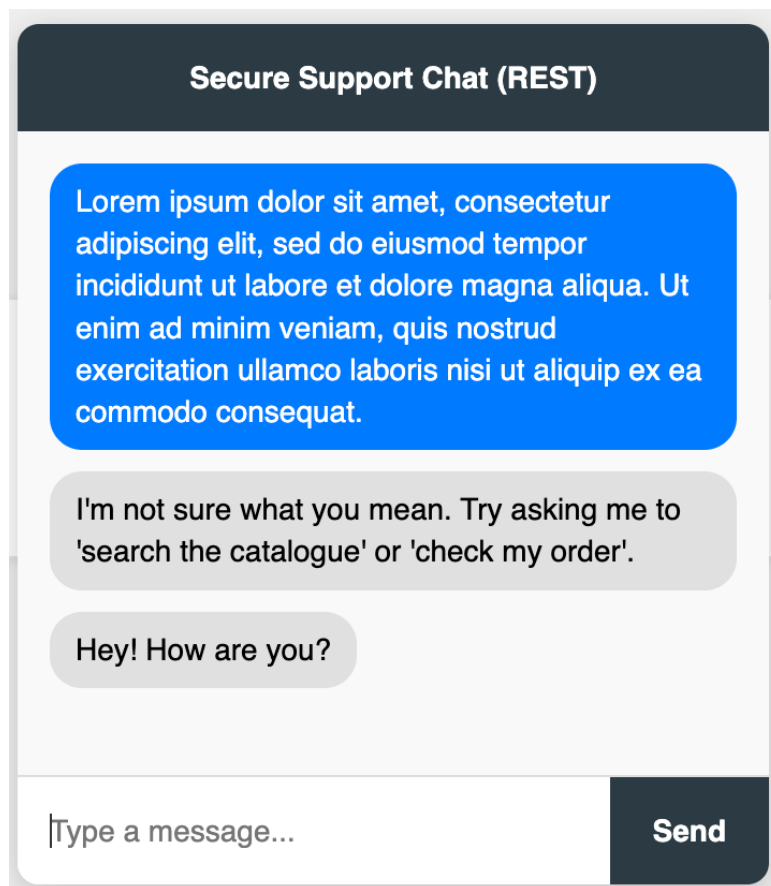
- **Inconsistenties bij typfouten en scheldwoorden (LLM-TOXIC-IN):** Tijdens het testen van de *abuse*-filtering vertoonde het semantische AI-model inconsistent gedrag rondom typfouten. Een opzettelijk fout gespeld scheldwoord ("bithc", zie Figuur 7.4) resulteerde in een onmiddellijke blokkade voor toxische inhoud, terwijl de correct gespelde variant verrassend genoeg niet als schadelijk werd gemarkeerd. **Oplossing:** Dit illustreert de probabilistische aard van firewall. Om dergelijke anomalieën af te dichten, is het noodzakelijk om de AI-regelgeving aan te vullen met traditionele, deterministische configuraties (zoals regex-gebaseerde *custom dictionaries*) voor absolute *profanity* filtering.



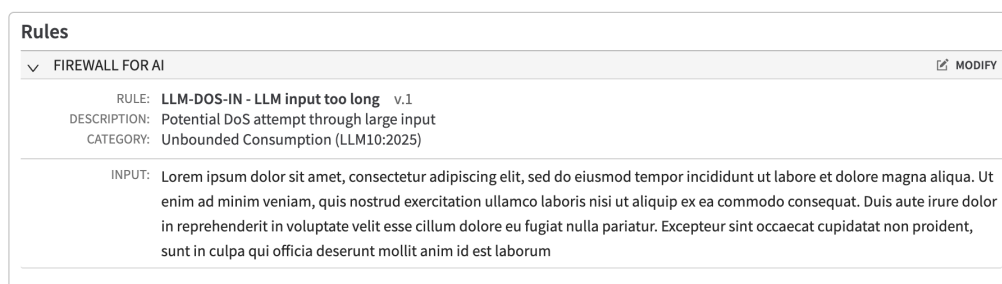
Figuur 7.4: Inconsistente detectie van toxiciteit bij opzettelijke typfouten.

- **Gedrag bij onbekende taalstructuren (LLM-DOS-IN):** Tijdens de validatie van de taalrestricties bleek dat een korte *Lorem Ipsum*-tekst de semantische detectiemodule van de firewall 'verwarde'. De invoer werd niet herkend als een niet-toegestane taal (*Non-allowed language*), waardoor het bericht ongehinderd de beveiliging passeerde. Dit leidde tot directe verwarring bij de digitale assistent: omdat de NLU-engine deze abstracte invoer niet kon koppelen aan een gekende *intent*, raakte de chatbot gedesoriënteerd en werd een generieke *fallback*-reactie getriggerd (zie Figuur 7.5). De firewall greep pas in wanneer de tekst aanzienlijk werd uitgebreid (zie Figuur 7.6), maar deed dit uitsluitend op basis van de *LLM input too long*-regel. Dit toont aan dat korte, vreemde taalstructuren een risico vormen voor de stabiliteit van de interactie-

logica.



Figuur 7.5: De chatbot raakt verward door een korte, onbekende taalstructuur en triggert een fallback-reactie.



Figuur 7.6: Verwarde taalherkenning, waarbij pseudo-Latijn uiteindelijk werd gestopt op lengte.

Deze incidenten en de daaropvolgende finetuning bewijzen dat AI-beleidsregels te allen tijde contextuele kalibratie vereisen. Zelfs de meest geavanceerde Firewalls for AI zijn pas succesvol wanneer ze nauwkeurig worden afgestemd op de specifieke applicatielogica en het verwachte gedrag van de eindgebruiker.

Deze finetuning bewijst dat ook voor de firewall for AI calibratie nodig is om succesvol te zijn.

7.2. Traceerbaarheid en Compliance

De evaluatie van de privacy- en compliancedoelstellingen laat zien dat een gelaagde aanpak noodzakelijk is om chatbot-interacties veilig te loggen. De resultaten van die inrichting zijn getoetst op basis van dataminimalisatie, aansprakelijkheid en het recht op gegevenswissing.

7.2.1. Upstream Dataminimalisatie

De combinatie van het custom REST-kanaal en de OpenTelemetry Collector bleek in de praktijk volledig effectief om privacygevoelige gegevens buiten de monitoringomgeving te houden. Door de status van de conversatie via de Rasa Tracker API te controleren, maskeert het custom rest channel invoer direct zodra een gebruiker een gevoelig formulier activeert.

De OpenTelemetry Collector vult dit aan. Tijdens de tests met e-mailadressen en creditcardnummers in de vrije chatinvoer, heeft de transformator (via de OTTL-regels) alle patronen succesvol veranderd naar [REDACTED_EMAIL] en [REDACTED_CC]. Omdat deze gevoelige data de server nooit verlaat, is het risico op data-leakage in de Dynatrace zo goed als uitgesloten.

7.2.2. Attributie en Aansprakelijkheid

Bij de beoordeling van de traceerbaarheid is de bewuste keuze gemaakt om de gebruikersnaam (`rasa.username`) expliciet *niet* te maskeren. De testresultaten tonen aan dat dit cruciaal is voor de bruikbaarheid van de audit trail. Wanneer er via het aanvalsscript (zie Sectie 5.5) kwaadaardig verkeer werd verstuurd, kon het incident in Dynatrace direct aan een specifieke user worden gekoppeld.

Als de gebruikersnaam volledig geanonimiseerd zou zijn, zou het voor een beheerder onmogelijk zijn om te bepalen welk account verantwoordelijk was voor de misbruikpoging. De opstelling bewijst dat het scheiden van de inhoud (verborgen voor privacy) en de identiteit (behouden voor security) zorgt voor een balans tussen privacy en aansprakelijkheid.

7.2.3. Uitvoering van het Recht op Gegevenswissing

Het recht op gegevenswissing is geëvalueerd als een gedeelde verantwoordelijkheid tussen de applicatielaag en de monitoringlaag:

- **Applicatiedata:** Een verzoek tot gegevenswissing kan volledig worden uitgevoerd binnen de backend van de Sockshop. Gebruikersprofielen en opgeslagen klantgegevens worden op verzoek direct en definitief uit de Java/Helidon-databases gewist.
- **Telemetriedata:** In de monitoringomgeving is het bewust onmogelijk ge-

maakt om zelf logs of traces te wissen. Dit voorkomt dat een aanvaller na een incident zijn eigen forensische sporen kan uitwissen door een verwijderverzoek in te dienen. Compliance wordt hier behaald via de ingestelde retentieperiode: alle telemetrie binnen Dynatrace wordt na maximaal 30 dagen automatisch en verwijderd.

7.3. Gebruiksvriendelijkheid (beheerder perspectief)

De beheerbaarheid is geëvalueerd vanuit het perspectief van de beheerder. De focus lag op de configuratiegemak van de firewall, de bruikbaarheid van de incidentenlogs en de integratie met het observability-platform.

7.3.1. Configuratiegemak (Akamai)

Het configureren van JSONPath-inspecties binnen Akamai bleek intuïtief. Door eerst Monitor-modus te gebruiken, konden regels veilig worden getest. Alleen kon het pad nog niet met een wildcard werken. Na meerdere pogingen heb ik dus uiteindelijk gekozen om een eigen custom REST channel te maken.

7.3.2. Incidentenanalyse

De verwerking en analyse van beveiligingsincidenten verloopt via het Akamai Security Center een stuk sneller dan het handmatig doorzoeken van traditionele, ongestructureerde logbestanden. Wanneer de firewall een bericht blokkeert ontvangt de frontend het vooraf ingestelde JSON-fragment, waarmee de gebruiker op een vriendelijke manier wordt geïnformeerd.

Ondanks het ontbreken van een unieke referentietoken in de HTTP-respons, zijn de geblokkeerde verzoeken achteraf heel gemakkelijk terug te vinden in Akamai Web Security Analytics (WSA). Tijdens de tests met het geautomatiseerde aanvalsscript kwamen de incidenten direct in het dashboard naar boven door te filteren op de gebruikte Security Config en attack type LLM Firewall. Aangezien dit de enige configuratie van de LLM firewall is zullen alle Samples van deze PoC zijn. De beheerder krijgt in WSA meteen een duidelijk overzicht van de geolocatie van de aanvaller en welke regel is getriggerd. Dit verkort de tijd die nodig is om de herkomst en de impact van een aanval te bepalen.

7.3.3. Integratie en zichtbaarheid (Dynatrace)

De integratie via de ActiveGate en de OpenTelemetry Collector zorgt voor een volledige samensmelting van netwerkbeveiliging en applicatie-observability. Waar een beheerder voorheen logs uit de firewall en de webserver handmatig naast elkaar moest leggen, toont Dynatrace nu de volledige keten in één overzicht.

Binnen de weergave van de gedistribueerde traces is de call tree direct inzichtelijk. Bij een chatbot-interactie ziet de beheerder niet alleen de exacte responstijd, maar ook hoe die tijd is verdeeld over de chatbot-actie (`AddToCart`) en de backend-pods (`CatalogApi` en `ItemsApi`).

Het feit dat de metadata (zoals de NLU confidence score en de herkende intentie) direct aan de trace is gekoppeld, verhoogt de waarde van deze oplossing. Als de chatbot traag reageert of een verkeerd antwoord geeft, kan een beheerder snel zien of dit afkomstig is van een haperende microservice, een onduidelijke gebruikersprompt, of een ingreep van de AI-firewall aan de edge. Dit versnelt de foutopsporing binnen de hybride architectuur.

8

Conclusie

De integratie van AI-gedreven digitale assistenten in bedrijfsomgevingen biedt aanzienlijke voordelen op het vlak van efficiëntie en klantinteractie, maar introduceert tegelijkertijd een complex en uitgebreid aanvalsoppervlak. Het doel van dit onderzoek was na te gaan hoe een geïntegreerd framework op basis van Dynatrace's Audit Logging en Akamai's Firewall for AI ontworpen kan worden om chatbotinteracties op een veilige en traceerbare manier te implementeren, zonder tekortkomingen op vlak van privacyvereisten zoals vastgelegd in de GDPR.

Om deze onderzoeksvraag te beantwoorden, werd een hybride proof-of-concept (PoC) ontwikkeld bestaande uit een Rasa NLU-engine, gecontaineriseerde Helidon backend-microservices, en een beveiligingsarchitectuur waarin Dynatrace (observability) en Akamai (security) centraal staan.

8.1. Evaluatie van de Beveiligingsarchitectuur

De resultaten van de PoC tonen aan dat de beveiligingsaanpak met Akamai's Firewall for AI op de edge een goede strategie vormt. Tijdens de testfase werden alle 9 gesimuleerde aanvalsvectoren zoals Code injection en het omzeilen van de firewallregels door gebruik te maken van vreemde talen succesvol gedetecteerd en direct proactief geblokkeerd.

Daarnaast werd duidelijk dat beveiliging niet beperkt blijft tot het controleren van inkomende data. Ook het monitoren en filteren van uitgaande data speelt een belangrijke rol in het voorkomen van ongewenste informatielekken. Dit principe draagt bij aan een meer volledige beveiligingsstrategie, waarbij zowel preventie als controle noodzakelijk zijn.

Tegelijkertijd toont de evaluatie aan dat dergelijke beveiligingsmechanismen niet statisch zijn. Het afstemmen van detectieregels blijft een iteratief proces, waarbij een evenwicht moet worden gevonden tussen het blokkeren van kwaadaardige interacties en het vermijden van foutieve detecties.

Voor het mitigatie-pad (detectie → analyse → containment → herstel) is gebleken dat de containment-stap volledig geautomatiseerd verloopt op de edge via Akamai. Hoewel er geen automatische anomaliedetectie of actieve mitigatiebepaling binnen Dynatrace zelf is geïmplementeerd, bewijst het platform zijn absolute waarde in de analysefase. Dynatrace stelt security-teams in staat om succesvolle of geblokkeerde malafide prompts achteraf diepgaand te analyseren, patronen te herkennen en het systeem op basis hiervan continu te optimaliseren.

8.2. Traceerbaarheid en Privacy-by-Design

Een cruciaal resultaat van dit onderzoek is dat de koppeling tussen observability-data en security-events technisch gerealiseerd kan worden met behulp van de *rx-Visitor*-cookie. Deze cookie dient als de unieke sleutel (correlation ID) waarmee de volledige interactiestroom tussen de eindgebruiker, de chatbot en de firewall over alle componenten heen kan worden gecorreleerd. Hierdoor is het mogelijk om het volledige gesprek chronologisch inzichtelijk te maken, inclusief eventuele geblokkeerde interacties.

Dit geïntegreerde framework levert hiermee een betrouwbare vorm van *audit trail* en *chain-of-evidence* op. Omdat elk event en elke blokkering aan een specifieke sessie gekoppeld is, is de reconstructie van incidenten achteraf reproduceerbaar en juridisch verdedigbaar.

Tegelijkertijd voldoet deze architectuur strikt aan de privacy-by-design en GDPR-richtlijnen. Door het toepassen van technieken zoals data masking en een gecontroleerde logverwerking op de verzamelde conversatiegegevens, wordt de verwerking van persoonsgegevens effectief beperkt tot het absolute minimum. Dit bewijst dat volledige technische traceerbaarheid en de nodige privacyvereisten elkaar niet tegenwerken.

8.3. Eindoordeel

Op basis van dit onderzoek kan worden vastgesteld dat een geïntegreerde benadering van observability en security essentieel is voor het veilig inzetten van AI-gedreven chatbots in een enterprise-context. Het ontwikkelde proof-of-concept toont aan dat het mogelijk is om zowel beveiliging als traceerbaarheid te realiseren door bestaande technologieën op een doordachte manier te combineren.

Voor de doelgroep, IT-teams die verantwoordelijk zijn voor het beheer van AI-toepassingen, biedt deze aanpak een duidelijke meerwaarde. Het systeem maakt het mogelijk om interacties te monitoren, verdachte patronen te detecteren en incidenten reproduceerbaar te analyseren, terwijl tegelijkertijd rekening wordt gehouden met privacy- en compliancevereisten.

9

Discussie

In dit hoofdstuk worden de implicaties van de resultaten overwogen en worden concrete aanbevelingen geformuleerd voor toekomstig onderzoek binnen het domein van AI-security en observability.

9.1. Aanbevelingen voor Toekomstig Onderzoek

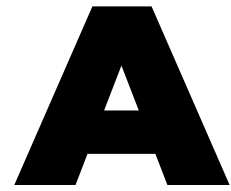
Dit onderzoek richtte zich voornamelijk op een intent-gebaseerd NLU-systeem (Rasa) binnen een gelaagde architectuur met Dynatrace en Akamai. Gezien de snelle evolutie richting generatieve AI en Large Language Models (LLM's), vormt dit een belangrijk aandachtspunt voor toekomstig onderzoek. Verdere studie kan zich specifiek richten op het detecteren van complexere aanvalstechnieken, zoals geavanceerde prompt injection en de manipulatie van modelgedrag in LLM-omgevingen.

Daarnaast biedt de verdere automatisering van incidentrespons een interessant perspectief. Hoewel Akamai binnen deze proof-of-concept malafide data al automatisch blokkeert op de edge, kan toekomstig onderzoek zich richten op een scenario waarin Dynatrace bij het herkennen van afwijkende patronen direct policies aanpast in de firewall. Dit maakt een dynamische en proactieve beveiligingsstrategie mogelijk waarbij het systeem zelfstandig leert en mitigeert. Ook de integratie van machine learning-algoritmen binnen Dynatrace zelf om anomalieën te detecteren en automatisch acties te bepalen, kan een waardevolle uitbreiding zijn.

Daarnaast moet worden benadrukt dat de specifieke beveiliging van de chatbot-component de noodzaak voor beveiliging van de omliggende website niet wegneemt. Hoewel de Firewall for AI effectief beschermt tegen malafide prompts en backend-gerelateerde aanvalsvectoren, blijft de front-end applicatie vatbaar voor traditionele kwetsbaarheden aan de client-side. Binnen deze proof-of-concept bleek

bijvoorbeeld dat *self-XSS* nog steeds mogelijk is via het chatbotvenster. Een gebruiker kan via deze weg handmatig een aangepast stylesheet of javascript-code invoegen.

Tot slot kan verder onderzoek zich richten op het verbeteren van *explainability* (verklaarbaarheid) binnen AI-systemen. Het is hierbij cruciaal dat niet alleen technisch zichtbaar is *dat* Akamai's Firewall for AI een specifieke prompt blokkeert (gecorreleerd aan de hand van de *rxVisitor*-cookie), maar ook de exacte semantische motivatie en achterliggende logica inzichtelijk worden gemaakt. Dit zal de betrouwbaarheid van de *audit trail* en de juridische verdedigbaarheid van de *chain-of-evidence* nog verder versterken.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

De inzet van digitale assistenten binnen klantondersteuning neemt sterk toe binnen professionele IT-omgevingen. Organisaties gebruiken dergelijke systemen om klantvragen sneller te beantwoorden, incidenten op te volgen en administratieve processen te automatiseren. In deze context worden echter vaak gevoelige gegevens verwerkt, zoals persoonsgegevens, bestelgegevens of interne bedrijfsinformatie. Binnen veel organisaties ontbreekt vandaag een sluitend mechanisme om deze interacties volledig te volgen, te beveiligen en achteraf te reconstrueren. Dit vormt een concreet probleem wanneer beveiligingsincidenten optreden of wanneer naleving van privacywetgeving moet worden aangetoond.

Deze bachelorproef vertrekt vanuit een concrete bedrijfscontext bij Evolane, een IT-dienstverlener die organisaties ondersteunt op het vlak van observability en security. Binnen deze casus stelt zich de uitdaging om AI-gestuurde chatbotinteracties niet alleen operationeel betrouwbaar te laten functioneren, maar ook volledig traceerbaar en beveiligd te maken. Meer specifiek ontbreekt vandaag een geïntegreerde aanpak waarbij zowel de inhoud van gesprekken als beveiligingsgebeurtenissen op een samenhangende manier worden geregistreerd, gecorreleerd en geanalyseerd. Dit gebrek aan inzicht bemoeilijkt incidentanalyse, auditing en het aantonen van compliance.

De doelgroep van deze bachelorproef bestaat uit IT- en securityprofessionals bin-

nen Evolane en vergelijkbare organisaties, die verantwoordelijk zijn voor het behe-
ren, beveiligen en monitoren van digitale klantinteracties.

Vanuit deze probleemsituatie wordt volgende centrale onderzoeksvraag geformuleerd: Hoe kan een geïntegreerd framework op basis van Dynatrace Audit Logging en Akamai's AI Firewall worden ontworpen om AI-chatbotinteracties volledig traceerbaar en beveiligd te maken, met behoud van privacy en naleving van GDPR?

Om deze onderzoeksvraag te beantwoorden, richt het onderzoek zich op het combineren van observability en security binnen een samenhangend framework. Om het probleemdomain af te bakenen, worden de volgende deelonderzoeksvragen geformuleerd:

- Hoe kan de interactiestroom tussen eindgebruiker en chatbot technisch in kaart gebracht worden via Dynatrace (wie, wat, wanneer)?
- Welke soorten aanvallen of verdachte inputs kan Akamai's AI Firewall detecteren en blokkeren tijdens chatbotverkeer?
- Hoe goed kunnen observability-gegevens uit Dynatrace gecorreleerd worden met security-events uit Akamai om een volledig beeld te vormen van een gesprek?
- Hoe vertalen we de gecorreleerde Dynatrace-traces en Akamai-alerts naar een direct uitvoerbaar mitigatie-pad (detectie → analyse → containment → herstel) om incidenten reactief en meetbaar te beperken?
- In welke mate voldoet het systeem aan privacy- en GDPR-richtlijnen bij het loggen en bewaren van conversatiegegevens?
- Hoe betrouwbaar is het audit-trailstelsel bij het reconstrueren van volledige gesprekken en events?

Op basis van de analyse van het probleemdomain richt het onderzoek zich vervolgens op het oplossingsdomain, met de volgende deelonderzoeksvragen:

- Hoe kan Dynatrace gebruikt worden om afwijkende patronen in chatbotinteracties automatisch te detecteren en te classificeren als potentieel risico?
- Welke type aanvallen of risicovolle inputs detecteert Akamai's AI Firewall het meest effectief, en hoe kunnen deze detecties worden teruggekoppeld naar Dynatrace voor correlatie?
- Hoe kunnen observability-data en security-events gecombineerd worden om een automatische mitigatieactie te triggeren (zoals blokkeren, isoleren of waarschuwen)?

- Welke vorm van audit trail of chain-of-evidence is het meest geschikt om incidenten achteraf reproduceerbaar en juridisch verdedigbaar te maken?
- In welke mate voldoet het geïntegreerde framework aan GDPR- en privacy-by-designrichtlijnen bij het loggen, analyseren en mitigerend reageren op incidenten?

De onderzoeksdoelstelling van deze bachelorproef is het ontwerpen en realiseren van een proof-of-concept dat aantoont hoe een dergelijke geïntegreerde oplossing in de praktijk kan functioneren. Het concrete eindresultaat bestaat uit een werkend prototype van een digitale assistent, aangevuld met een observability- en security-laag die interacties inzichtelijk en controleerbaar maakt. De bachelorproef kan als geslaagd worden beschouwd wanneer het prototype aantoont dat chatbotinteracties op een reproduceerbare, veilige en privacybewuste manier kunnen worden gemonitord en beveiligd.

A.2. Literatuurstudie

De opkomst van artificiële intelligentie binnen klantgerichte toepassingen heeft geleid tot een sterke toename van geautomatiseerde interacties tussen eindgebruikers en digitale systemen. In het bijzonder AI-chatbots worden steeds vaker ingezet voor klantondersteuning, verkoop en incidentopvolging. Hoewel deze technologie duidelijke voordelen biedt op vlak van efficiëntie en schaalbaarheid, introduceert dit tegelijk ook nieuwe uitdagingen op het gebied van beveiliging, traceerbaarheid en privacy. Deze literatuurstudie beschrijft de huidige stand van zaken binnen dit domein en benadrukt de nood aan geïntegreerde oplossingen die observability en beveiliging combineren.

A.2.1. AI-chatbots in bedrijfscontext

AI-chatbots maken gebruik van natuurlijke taalverwerking en machine learning om menselijke interacties te simuleren en gebruikersvragen te beantwoorden. Volgens Adamopoulou en Moussiades (2020a) worden dergelijke systemen vooral ingezet om repetitieve taken te automatiseren en de werkdruk op menselijke medewerkers te verlagen. In bedrijfscontexten verwerken chatbots echter vaak gevoelige informatie, zoals persoonsgegevens of contractdetails. Dit verhoogt het risico op datalekken en misbruik, zeker wanneer onvoldoende controle bestaat over de inhoud en het verloop van deze gesprekken (Bender e.a., 2021b).

Daarnaast blijkt uit recent onderzoek dat AI-systemen kwetsbaar zijn voor specifieke aanvalsvormen, zoals prompt-injecties en data-exfiltratie via doelbewust gemanipuleerde input (Greshake e.a., 2023). Deze kwetsbaarheden tonen aan dat

klassieke beveiligingsmaatregelen niet volstaan om AI-gedreven interacties voldoende te beschermen.

A.2.2. Beperkingen van traditionele beveiligingsmechanismen

Traditionele web- en API-beveiliging steunt voornamelijk op statische regels en vooraf gedefinieerde patronen. Firewalls en intrusion detection systems zijn effectief tegen bekende aanvalsvectoren, maar hebben moeite om contextuele en semantische aanvallen te detecteren (Sommer & Paxson, 2010). In het geval van AI-chatbots, waar input en output contextafhankelijk zijn, leidt dit tot een verhoogd risico op false negatives en onvoldoende bescherming.

Om deze beperkingen tegen te gaan, worden steeds vaker AI-gestuurde beveiligingsoplossingen voorgesteld. Akamai beschrijft in zijn recente publicaties hoe machine learning kan worden ingezet om afwijkende patronen in web- en API-verkeer te herkennen en dynamisch te mitigeren (Akamai Technologies, 2025). Dergelijke oplossingen bieden nieuwe mogelijkheden, maar vereisen een integratie van observability- en auditsysteem om hun effectiviteit te kunnen evalueren.

A.2.3. Observability en auditability van AI-systemen

Observability verwijst naar het vermogen om de interne toestand van een systeem te reconstrueren op basis van externe signalen zoals logs, metrics en traces. In moderne, complexe en gedistribueerde omgevingen is observability cruciaal om systeemgedrag te analyseren, incidenten te onderzoeken en performantie te optimaliseren (Li e.a., 2021). Dynatrace benadrukt dat observability steeds belangrijker wordt in AI-gedreven toepassingen, omdat de besluitvorming van modellen vaak complex is en moeilijk te verklaren valt (Dynatrace, 2025).

Audit logging speelt hierbij een centrale rol. Door interacties, systeembeslissingen en beveiligingsevents systematisch te registreren, wordt het mogelijk om AI-gedreven processen achteraf te reconstrueren en te analyseren. Dergelijke auditmechanismen zijn essentieel voor troubleshooting, maar ook voor het aantonen van verantwoordelijkheid en naleving van regelgevingen (Novelli e.a., 2023). Recent onderzoek laat zien dat audit logging bij AI-systemen vaak los van de rest wordt toegepast. Er is meestal geen goede koppeling tussen wat je kunt zien in observability-data en de security-events, waardoor het lastiger wordt om achteraf precies te begrijpen wat er gebeurd is of incidenten goed te analyseren.

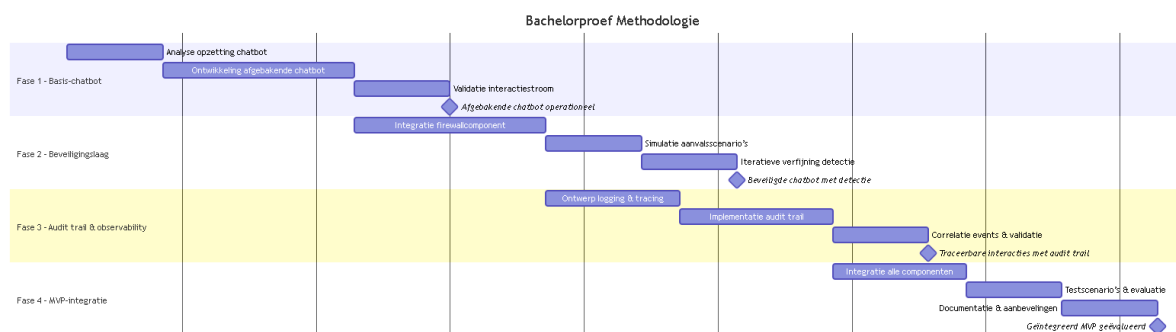
A.2.4. Privacy en regelgeving

Het loggen van chatbotinteracties brengt onvermijdelijk privacyvraagstukken met zich mee. De General Data Protection Regulation (GDPR) legt strikte voorwaarden op voor het verzamelen, verwerken en bewaren van persoonsgegevens. Volgens

Voigt en von dem Bussche (2017) moeten systemen die persoonsgegevens verwerken voldoen aan het principe van privacy by design, waarbij dataminimalisatie en transparantie centraal staan.

In het geval van AI-chatbots betekent dit dat logging en tracing zorgvuldig moeten worden ontworpen om gevoelige gegevens te beschermen, bijvoorbeeld via pseudonimisering of selective logging (European Data Protection Board, 2020b). De tekst benadrukt dat veiligheid en privacy geen tegengestelde doelen zijn, maar elkaar kunnen versterken wanneer ze vanaf het ontwerp geïntegreerd worden.

A.3. Methodologie



Figuur A.1: Gantt-diagram

Dit onderzoek wordt uitgevoerd als toegepast onderzoek binnen de bedrijfscontext van Evolane en focust op één afgebakende probleemsituatie, namelijk het gebrek aan traceerbaarheid en beveiliging van AI-gestuurde chatbotinteracties binnen klantondersteuningsprocessen. Het onderzoek resulteert in een minimum viable product (MVP), zijnde een eerste functionele implementatie waarin de kernfunctionaliteiten voor traceerbaarheid en beveiliging aantoonbaar aanwezig zijn. De scope van het onderzoek beperkt zich tot het technisch ontwerpen, implementeren en evalueren van dit prototype binnen een gecontroleerde testomgeving.

Het proof-of-concept wordt opgebouwd in vier opeenvolgende, inhoudelijk samenhangende fasen. Elke fase levert een concrete mijlpaal op die bijdraagt aan het beantwoorden van de centrale onderzoeksvraag.

Fase 1: Ontwikkeling van een afgebakende chatbot

In de eerste fase wordt een eenvoudige chatbot ontwikkeld die één concreet klantondersteuningsscenario implementeert. De functionaliteit blijft bewust beperkt tot een duidelijk afgelijnde use case, zodat de interactiestroom tussen gebruiker en digitale assistent reproduceerbaar blijft. De chatbot dient uitsluitend als testob-

ject voor verdere beveiligings- en traceerbaarheidsmechanismen. Het eindresultaat van deze fase is een functionerende chatbot met een stabiele interactiestroom.

Fase 2: AI firewall voor chatbotverkeer

In de tweede fase wordt het chatbotverkeer voorzien van een beveiligingslaag die inkomende en uitgaande interacties analyseert. De focus ligt op het detecteren en blokkeren van afwijkende of risicovolle invoer binnen de afgebakende testomgeving. Door het uitvoeren van gesimuleerde aanvalsscenario's wordt nagegaan hoe effectief het systeem reageert op misbruik van de chatbotinterface. Deze fase resulteert in een chatbot waarbij beveiligingsdetectie en -mitigatie functioneren.

Fase 3: Implementatie van traceerbaarheid en audit trail

De derde fase richt zich op het vastleggen en overeenkomen van gebeurtenissen binnen de chatbotinteracties. Hierbij worden gebruikersacties, systeemreacties en beveiligingsgebeurtenissen gelogd om volledige reconstructie van gesprekken mogelijk te maken. De audit trail wordt opgezet met als doel technische reproduceerbaarheid van incidenten, zonder dat juridische bewijsvoering of langdurige datastorage wordt nagestreefd. Het eindresultaat van deze fase is een werkende audit trail die inzicht biedt in het volledige verloop van interacties.

Fase 4: Integratie en evaluatie van het MVP

In de vierde fase worden alle ontwikkelde componenten samengebracht tot één geïntegreerd MVP. Dit prototype wordt geëvalueerd aan de hand van vooraf gedefinieerde testscenario's die zowel normale interacties als afwijkend gedrag omvatten. De evaluatie focust op de mate waarin het systeem traceerbaarheid en beveiliging combineert binnen de afgebakende casus. Deze fase resulteert in een geïntegreerd MVP en een technische evaluatie die de basis vormt voor aanbevelingen.

A.4. Verwacht resultaat, conclusie

Het voornaamste verwachte resultaat van deze bachelorproef is een functionerend proof-of-concept van een geïntegreerd framework waarin een AI-chatbot, een beveiligingslaag en een audit trail samenkomen tot een minimum viable product (MVP). Dit MVP zal aantonen dat interacties tussen eindgebruikers en de chatbot traceerbaar, reproduceerbaar en beveiligd kunnen worden, zonder dat gevoelige informatie onnodig wordt blootgesteld.

Specifiek wordt verwacht dat het prototype de volgende kenmerken vertoont:

- Traceerbaarheid van gesprekken: alle prompts, antwoorden, sessie-ID's en tijdstempels worden gelogd en zijn reconstructeerbaar.
- Detectie en mitigatie van risicovolle input: de beveiligingslaag kan verdachte

of afwijkende gebruikersinvoer herkennen en blokkeren of markeren voor nadere analyse.

- Gecorreleerde observability: beveiligingsincidenten en interactiegegevens zijn verbonden in een audit trail, waardoor een volledig overzicht van elk gesprek en mogelijke incidenten ontstaat.

Het onderzoek biedt inzicht in de praktische haalbaarheid van het combineren van observability en AI-gestuurde beveiliging. Verwacht wordt dat de resultaten aantonen welke componenten effectief bijdragen aan veiligheid en traceerbaarheid, en waar nog optimalisaties mogelijk zijn.

Bibliografie

- Adamopoulou, E., & Moussiades, L. (2020a). Chatbots: History, technology, and applications. *Machine Learning with Applications*. <https://doi.org/10.1016/j.mlwa.2020.100006>
- Adamopoulou, E., & Moussiades, L. (2020b). Chatbots: History, technology, and applications. *Machine Learning with Applications*, 2, 100006. <https://doi.org/10.1016/j.mlwa.2020.100006>
- Akamai Technologies. (2025). Firewall for AI: Protecting AI applications from adversarial inputs and data exposure. <https://www.akamai.com/products/firewall-for-ai>
- Anonymous. (2024). A Comparative Framework for Intent Classification Systems: Evaluating Large Language Models versus Traditional Machine Learning in Contact Centers. *International Journal for Multidisciplinary Research (IJFMR)*. Verkregen mei 21, 2026, van <https://www.ijfmr.com/papers/2024/6/30430.pdf>
- Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021a). On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? <https://doi.org/10.1145/3442188.3445922>
- Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021b). On the dangers of stochastic parrots: Can language models be too big? *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency*. <https://doi.org/10.1145/3442188.3445922>
- Bommasani, R., Hudson, D. A., Adeli, E., & et al. (2021). On the Opportunities and Risks of Foundation Models. Verkregen februari 20, 2025, van <https://arxiv.org/abs/2108.07258>
- Brown, T. B., Mann, B., Ryder, N., & et al. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901. Verkregen februari 20, 2025, van <https://papers.nips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>
- Carlini, N., e.a. (2021). Extracting Training Data from Large Language Models. Verkregen februari 28, 2025, van <https://arxiv.org/abs/2012.07805>
- Carlini, N., Tramer, F., Wallace, E., Jagielski, M., Herbert-Voss, A., Lee, K., Roberts, A., Brown, T., Song, D., Erlingsson, U., Oprea, A., & Raffel, C. (2023). Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. Verkregen februari 28, 2025, van <https://arxiv.org/abs/2302.12173>

- Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly Detection: A Survey. *ACM Computing Surveys*, 41(3). <https://doi.org/10.1145/1541880.1541882>
- Dwivedi, Y. K., & et al. (2023). So what if ChatGPT wrote it? Multidisciplinary perspectives on opportunities, challenges and implications of generative conversational AI. *International Journal of Information Management*, 71. <https://doi.org/10.1016/j.ijinfomgt.2023.102642>
- Dynatrace. (2025). Dynatrace Advances AI Observability to Support Generative AI Initiatives. <https://www.dynatrace.com/news/press-release/dynatrace-advances-ai-observability-to-support-generative-ai-initiatives/>
- European Data Protection Board. (2020a). Guidelines 4/2019 on Article 25 Data Protection by Design and by Default. https://www.edpb.europa.eu/our-work-tools/our-documents/guidelines/guidelines-42019-article-25-data-protection-design-and_en
- European Data Protection Board. (2020b). Guidelines 4/2019 on Article 25 Data Protection by Design and by Default. https://www.edpb.europa.eu/our-work-tools/our-documents/guidelines/guidelines-42019-article-25-data-protection-design-and_en
- European Parliament and Council. (2016). General Data Protection Regulation (EU) 2016/679. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- European Union. (2022). Directive (EU) 2022/2555 (NIS2 Directive). <https://eur-lex.europa.eu/eli/dir/2022/2555/oj>
- Ganesan, P. (2022). Observability in Cloud-Native Environments: Challenges and Solutions. *International Journal of Core Engineering & Management*, 7(04). Verkregen maart 1, 2026, van https://www.researchgate.net/publication/384867297_OBSERVABILITY_IN_CLOUD-NATIVE_ENVIRONMENTS_CHALLENGES_AND_SOLUTIONS
- Gehman, S., Gururangan, S., Sap, M., & et al. (2020). RealToxicityPrompts: Evaluating Neural Toxic Degeneration in Language Models. *Findings of EMNLP*. Verkregen februari 28, 2025, van <https://arxiv.org/abs/2009.11462>
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., & Holz, T. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*. <https://arxiv.org/abs/2302.12173>
- International Organization for Standardization. (2022). ISO/IEC 27005:2022 Information security risk management. <https://www.iso.org/standard/80585.html>
- Jurafsky, D., & Martin, J. H. (2026). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd) [Online manuscript released January 6, 2026]. Verkregen februari 20, 2025, van <https://web.stanford.edu/~jurafsky/slp3/>

- Kaplan, J., McCandlish, S., Henighan, T., & et al. (2020). Scaling Laws for Neural Language Models. Verkregen februari 27, 2025, van <https://arxiv.org/abs/2001.08361>
- Kotenko, I., Gaifulina, D., & Zelichenok, I. (2022). Systematic Literature Review of Security Event Correlation Methods. *IEEE Access*, 10. <https://doi.org/10.1109/ACCESS.2022.3168976>
- Lewis, P., Perez, E., Piktus, A., & et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*. Verkregen februari 27, 2025, van <https://arxiv.org/abs/2005.11401>
- Li, B., Peng, X., Xiang, Q., Wang, H., Xie, T., & Liu, X. (2021). Enjoy your observability: an industrial survey of microservice tracing and analysis. *Empirical Software Engineering*. <https://doi.org/10.1007/s10664-021-10063-9>
- National Institute of Standards and Technology. (2006). *Guide to Computer Security Log Management* (tech. rap. Nr. SP 800-92). <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-92.pdf>
- National Institute of Standards and Technology. (2012). *Guide for Conducting Risk Assessments* (tech. rap. Nr. SP 800-30 Rev. 1). <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-30r1.pdf>
- National Institute of Standards and Technology. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0). <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf>
- National Institute of Standards and Technology. (2025). *Computer Security Incident Handling Guide* (tech. rap. Nr. SP 800-61 Rev. 3). NIST. Verkregen maart 1, 2026, van <https://doi.org/10.6028/NIST.SP.800-61r3>
- Novelli, C., Taddeo, M., & Floridi, L. (2023). Accountability in artificial intelligence: what it is and how it works. *AI and Society*. <https://doi.org/10.1007/s00146-023-01635-y>
- OWASP Foundation. (2021). OWASP Top 10:2021. <https://owasp.org/Top10/>
- OWASP Foundation. (2023). OWASP Top 10 for Large Language Model Applications. Verkregen februari 28, 2025, van <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- PCI Security Standards Council. (2022). Payment Card Industry Data Security Standard (PCI DSS) v4.0. https://www.pcisecuritystandards.org/document_library/
- Rasa. (2026). *Helping Your Assistant Understand Users* (tech. rap.). Rasa Technologies Inc. Verkregen mei 21, 2026, van <https://rasa.com/docs/learn/concepts/dialogue-understanding/>
- Sigelman, B. H., & et al. (2010). Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. *Google Research*. Verkregen maart 1, 2026, van <https://research.google/pubs/pub36356/>

- Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. *IEEE Symposium on Security and Privacy*. <https://doi.org/10.1109/SP.2010.25>
- Strom, B. E., e.a. (2020). *MITRE ATT&CK: Design and Philosophy* (tech. rap.). MITRE Corporation. Verkregen maart 1, 2026, van https://attack.mitre.org/docs/ATTACK_Design_and_Philosophy_March_2020.pdf
- Vaswani, A., Shazeer, N., Parmar, N., & et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30. Verkregen februari 20, 2025, van <https://arxiv.org/abs/1706.03762>
- Voigt, P., & von dem Bussche, A. (2017). *The EU General Data Protection Regulation (GDPR): A Practical Guide*. Springer. <https://doi.org/10.1007/978-3-319-57959-7>
- Weizenbaum, J. (1966). ELIZA—A Computer Program for the Study of Natural Language Communication Between Man and Machine. *Communications of the ACM*, 9(1), 36–45. <https://doi.org/10.1145/365153.365168>
- Zou, A., e.a. (2023). Universal and Transferable Adversarial Attacks on Aligned Language Models. Verkregen februari 28, 2025, van <https://arxiv.org/abs/2307.15043>